

Youtube-Shorts-Automation

Complete Repository Documentation & Line-by-Line Code Explanation

Twitch → YouTube Shorts / Long-form + Facebook Automation Pipeline

This document explains **every file** in the repository: its purpose, the full source code with line numbers, and a detailed **why / when / where** explanation of each meaningful block. It ends with an end-to-end execution walkthrough and a set of interview-style questions & answers derived from the codebase.

Scope: 17 tracked Python/config files, GitHub Actions workflow, channels.json, and the runtime data/ artefacts. No API keys, OAuth tokens, or access tokens are included — only structural placeholders are shown.

Table of Contents

1. Introduction & Purpose
2. Overall Architecture & Data Flow
3. Repository File Tree
4. Configuration & Secrets Model
5. Per-File Deep Dive (source + explanation)
 - 5.1 requirements.txt
 - 5.2 config.py
 - 5.3 channels.json
 - 5.4 twitch_finder.py
 - 5.5 twitch_downloader.py
 - 5.6 video_finder.py
 - 5.7 downloader.py (YouTube downloader)
 - 5.8 audio_analyzer.py
 - 5.9 highlight_detector.py
 - 5.10 video_processor.py
 - 5.11 metadata_generator.py
 - 5.12 youtube_uploader.py
 - 5.13 facebook_uploader.py
 - 5.14 notifier.py
 - 5.15 gen_token.py
 - 5.16 pipeline.py
 - 5.17 .github/workflows/upload_short.yml
 - 5.18 README.md (summary)
 - 5.19 data/ runtime artefacts (seen_videos.txt, upload_log.json)
6. End-to-End Execution Walkthrough (one full run)
7. Interview Questions & Answers
8. Environment Variables & Secrets Reference
9. Troubleshooting & Operational Notes

1. Introduction & Purpose

The repository automates a 24/7 content factory for short-form and long-form gaming video. It pulls trending clips from the **Twitch Helix API**, downloads them, analyses audio, cuts a vertical Short (and stitches several clips into a long-form compilation), generates AI metadata via **Groq**, and uploads to **YouTube** (or, for one channel, **Facebook**) on a schedule. Everything runs inside **GitHub Actions** on a daily cron plus manual dispatch, so no always-on server is needed.

Four YouTube channels (ExampleGamingChannel, ExistingChannel, GAMES, MinecraftShorts) and one Facebook channel (FacebookGaming) are configured. The Facebook channel was recently modified so it publishes **only videos longer than 3 minutes** (no Shorts) — that behaviour is documented in section 5.13 and 5.3.

Key design pillars:

- **Free tier only** — GitHub Actions runners, Groq free model, YouTube Data API (free quota), FFmpeg (open source).
- **Secrets never committed** — OAuth tokens live in GitHub Actions secrets or local token.json files (gitignored).
- **Idempotent / resumable** — a per-channel seen-list prevents re-uploading the same clip; uploads log daily counts against a quota.
- **Defensive** — Groq model auto-selection, Facebook 413 fall-back to resumable upload, duration gates, and auth/network abort logic.

2. Overall Architecture & Data Flow

The pipeline is a single Python program (pipeline.py) that orchestrates a chain of specialist modules. Each module has one job and is imported by the orchestrator. The GitHub Actions workflow is the scheduler/executor, not part of the runtime logic.

2.1 Module dependency graph (who imports whom)

```

1 | pipeline.py
2 |   ■■ config.py                (paths, secrets, limits, ffmpeg resolver)
3 |   ■■ twitch_finder.py         (find_twitch_clips, mark_video_seen)
4 |   ■■ twitch_downloader.py     (download_twitch_clip)
5 |   ■■ video_finder.py          (find_videos, find_playlist_videos, mark_video_seen)
6 |   ■■ downloader.py            (download_video - YouTube source)
7 |   ■■ audio_analyzer.py        (analyze_audio)
8 |   ■■ highlight_detector.py     (get_highlights, get_full_clip_range, get_video_duration)
9 |   ■■ video_processor.py       (process_video, process_compilation_video, get_video_duration)
10 |  ■■ metadata_generator.py     (generate_metadata, generate_video_metadata)
11 |  ■■ youtube_uploader.py       (upload_short, upload_video, get_upload_count_today)
12 |  ■■ facebook_uploader.py      (upload_fb_video)
13 |  ■■ notifier.py               (notify_report)

```

2.2 End-to-end data flow for one clip (Shorts path)

```

1 | channels.json ■■■ run_multi_channel()
2 |   ■
3 |   ■■[per channel] run_channel_pipeline()
4 |     ■
5 |     ■   ■■ find_twitch_clips()      (Twitch Helix API, recency windows)
6 |     ■   ■■ download_twitch_clip()   (CDN or yt-dlp)
7 |     ■   ■■ analyze_audio()          (voice? silent? -> mute decision)
8 |     ■   ■■ get_highlights()         (cut 35s window)
9 |     ■   ■■ process_video()          (9:16 crop, speedup, BGM -> Short)
10 |    ■   ■■ generate_metadata()       (Groq -> title/desc/tags)
11 |    ■   ■■ upload_short() / upload_fb_video() (OAuth2 / Graph API)
12 |    ■
13 |    ■■[video phase] process_compilation_video() (stitch N clips)
14 |    ■■ upload_video() / upload_fb_video(is_short=False)

```

2.3 Two upload destinations

- **YouTube** (default): a clip becomes a vertical Short (#shorts) and several clips become a 16:9 long-form video. OAuth2 with a refresh token.
- **Facebook** (platform=facebook): same processed media is sent to a Facebook Page via the Graph API. Long-form videos use *resumable* chunked upload; Reels are published immediately (FB cannot schedule Reels).

2.4 Scheduling

The GitHub Actions workflow `.github/workflows/upload_short.yml` triggers daily at **00:30 UTC** (06:00 IST) and can also be triggered manually with an optional single channel input. Each channel runs as a separate matrix job (max 4 parallel, 350-minute timeout). After each run the workflow commits the updated `seen_videos.txt` and `upload_log.json` back to the repo so state persists across ephemeral runners.

3. Repository File Tree

```

1 | Youtube-Shorts-Automation/
2 |   ├── config.py           # central config: paths, secrets, limits, ffmpeg
3 |   ├── channels.json       # per-channel definitions (YT + FB)
4 |   ├── requirements.txt    # pinned-ish Python deps
5 |   ├── pipeline.py         # MAIN orchestrator
6 |   ├── twitch_finder.py    # Twitch clip discovery + seen-list
7 |   ├── twitch_downloader.py # Twitch clip downloader (CDN + yt-dlp)
8 |   ├── video_finder.py     # YouTube source discovery (search/playlist)
9 |   ├── downloader.py       # YouTube video downloader (yt-dlp)
10 |  ├── audio_analyzer.py    # voice / silence detection
11 |  ├── highlight_detector.py # highlight cut + duration probe
12 |  ├── video_processor.py   # Short + compilation video rendering (ffmpeg)
13 |  ├── metadata_generator.py # Groq AI title/description/tags
14 |  ├── youtube_uploader.py  # YouTube OAuth2 uploader
15 |  ├── facebook_uploader.py # Facebook Graph API uploader
16 |  ├── notifier.py          # Telegram / Twilio report
17 |  ├── gen_token.py         # local OAuth token generator
18 |  ├── README.md           # quick-start readme
19 |  ├── assets/
20 |  │   ├── background_music/*.mp3 # NCS royalty-free BGM (10 tracks)
21 |  │   ├── fonts/roboto.ttf      # font used for any text overlay
22 |  │   └── data/
23 |  │       ├── channels/<Name>/    # per-channel seen_videos.txt, upload_log.json, token.json
24 |  │       ├── raw_videos/        # temp working dirs (gitignored)
25 |  │       ├── processed/
26 |  │       └── upload_log.json    # legacy global log
27 |  ├── .github/workflows/upload_short.yml # scheduler + executor

```

4. Configuration & Secrets Model

Two layers of configuration exist:

- **channels.json** — declarative per-channel settings (niche, source, schedule counts, platform).
- **Environment / Secrets** — credentials injected at runtime (local `.env` or GitHub Actions secrets). `config.py` reads them via `os.getenv` and, on import, materialises token files from env vars (`setup_secret_files()`).

Credential values are **never** written into this document. Only the names and the mechanism are described. The repository ships no `token.json` or `client_secret.json`; they are gitignored and reconstructed in CI from secrets.

Secret resolution order (CI):

1		YOUTUBE_TOKEN_JSON_<CHANNEL_UPPER>	->	data/channels/<Channel>/token.json
2		YOUTUBE_CLIENT_SECRET_JSON	->	client_secret.json (root)
3		YOUTUBE_TOKEN_JSON	->	token.json (legacy/global)
4		TWITCH_CLIENT_ID / TWITCH_CLIENT_SECRET		
5		YOUTUBE_API_KEY	->	YouTube Data API (search/playlist source)
6		GROQ_API_KEY	->	Groq metadata generation
7		FACEBOOK_PAGE_ID / FACEBOOK_PAGE_ACCESS_TOKEN	->	FB Page upload

5. Per-File Deep Dive

Each subsection shows the **verbatim source code with line numbers** (read from disk at generation time) followed by a detailed explanation organised by block, citing the exact line ranges and answering *why* it exists, *when* it runs, and *where* it is used.

5.1 requirements.txt

5.x Source Code — requirements.txt

```
1 | requests>=2.31.0
2 | python-dotenv>=1.0.0
3 | yt-dlp>=2024.3.10
4 | google-api-python-client>=2.100.0
5 | google-auth>=2.23.0
6 | google-auth-oauthlib>=1.1.0
7 | google-auth-httpplib2>=0.1.1
8 | groq>=0.4.0
9 | scenedetect>=0.6.2
10 | opencv-python-headless>=4.8.0
11 | ffmpeg-python>=0.2.0
12 | pydub>=0.25.1
13 | librosa>=0.10.1
14 | SpeechRecognition>=3.10.0
15 | Pillow>=10.0.0
16 | numpy>=1.26.0
17 | scipy>=1.11.0
18 | streamlink>=6.5.0
19 |
```

Explanation

Lines 1-18 — Dependency manifest

Pinned minimum versions for reproducible CI installs. They fall into groups: (a) HTTP/client — requests, python-dotenv; (b) media download — yt-dlp, streamlink; (c) Google APIs — google-api-python-client, google-auth*; (d) AI — groq; (e) video/audio — ffmpeg-python, pydub, librosa, scenedetect, opencv-python-headless, numpy, scipy; (f) speech — SpeechRecognition; (g) image — Pillow. **Why:** CI runs `pip install -r requirements.txt` so the runner has every import available. **Note:** ffmpeg-python is the Python wrapper; the actual ffmpeg/ffprobe binaries are installed separately via `apt-get install ffmpeg` in the workflow. **Where:** consumed only by the Actions job step 'Install Dependencies'.

5.2 config.py

5.x Source Code — config.py

```

1 | import os
2 | import shutil
3 | from dotenv import load_dotenv
4 |
5 | # Load environment variables
6 | load_dotenv()
7 |
8 | # API Keys & Auth Secrets (strip accidental newlines/whitespace)
9 | YOUTUBE_API_KEY = (os.getenv("YOUTUBE_API_KEY") or "").strip()
10 | GROQ_API_KEY = (os.getenv("GROQ_API_KEY") or "").strip()
11 | TWITCH_CLIENT_ID = (os.getenv("TWITCH_CLIENT_ID") or "").strip()
12 | TWITCH_CLIENT_SECRET = (os.getenv("TWITCH_CLIENT_SECRET") or "").strip()
13 |
14 | # Notification Configuration
15 | NOTIFICATION_PROVIDER = (os.getenv("NOTIFICATION_PROVIDER") or "TELEGRAM").upper().strip()
16 | TWILIO_ACCOUNT_SID = (os.getenv("TWILIO_ACCOUNT_SID") or "").strip()
17 | TWILIO_AUTH_TOKEN = (os.getenv("TWILIO_AUTH_TOKEN") or "").strip()
18 | TWILIO_FROM_NUMBER = (os.getenv("TWILIO_FROM_NUMBER") or "").strip()
19 | USER_PHONE_NUMBER = (os.getenv("USER_PHONE_NUMBER") or "").strip()
20 | TELEGRAM_BOT_TOKEN = (os.getenv("TELEGRAM_BOT_TOKEN") or "").strip()
21 | TELEGRAM_CHAT_ID = (os.getenv("TELEGRAM_CHAT_ID") or "").strip()
22 |
23 |
24 | # Directory Paths
25 | BASE_DIR = os.path.dirname(os.path.abspath(__file__))
26 | DATA_DIR = os.path.join(BASE_DIR, "data")
27 |
28 | ACTIVE_CHANNEL_NAME = None
29 |
30 | def set_active_channel(channel_name):
31 |     global ACTIVE_CHANNEL_NAME
32 |     ACTIVE_CHANNEL_NAME = channel_name
33 |
34 | def get_channel_dir():
35 |     if ACTIVE_CHANNEL_NAME:
36 |         sanitized = "".join([c if c.isalnum() else "_" for c in ACTIVE_CHANNEL_NAME])
37 |         path = os.path.join(DATA_DIR, "channels", sanitized)
38 |         os.makedirs(path, exist_ok=True)
39 |         return path
40 |     return DATA_DIR
41 |
42 | def get_raw_videos_dir():
43 |     d = os.path.join(get_channel_dir(), "raw_videos")
44 |     os.makedirs(d, exist_ok=True)
45 |     return d
46 |
47 | def get_clips_dir():
48 |     d = os.path.join(get_channel_dir(), "clips")
49 |     os.makedirs(d, exist_ok=True)
50 |     return d
51 |
52 | def get_processed_dir():
53 |     d = os.path.join(get_channel_dir(), "processed")
54 |     os.makedirs(d, exist_ok=True)
55 |     return d
56 |
57 | def get_thumbnails_dir():
58 |     d = os.path.join(get_channel_dir(), "thumbnails")
59 |     os.makedirs(d, exist_ok=True)
60 |     return d
61 |
62 | def get_seen_videos_file():
63 |     return os.path.join(get_channel_dir(), "seen_videos.txt")
64 |
65 | def get_upload_log_file():
66 |     return os.path.join(get_channel_dir(), "upload_log.json")
67 |
68 | def get_queue_file():
69 |     return os.path.join(get_channel_dir(), "upload_queue.json")
70 |
71 | def get_token_file():
72 |     return os.path.join(get_channel_dir(), "token.json")
73 |
74 | CLIENT_SECRETS_FILE = os.path.join(BASE_DIR, "client_secret.json")
75 |
76 | # Default fallbacks
77 | TARGET_NICHES = ["gaming"]
78 | MAX_SUBS = 100000
79 |
80 | def __getattr__(name):
81 |     if name == "RAW_VIDEOS_DIR":

```

```

82 |         return get_raw_videos_dir()
83 |     if name == "PROCESSED_DIR":
84 |         return get_processed_dir()
85 |     if name == "CLIPS_DIR":
86 |         return get_clips_dir()
87 |     if name == "THUMBNAILS_DIR":
88 |         return get_thumbnails_dir()
89 |     if name == "SEEN_VIDEOS_FILE":
90 |         return get_seen_videos_file()
91 |     if name == "UPLOAD_LOG_FILE":
92 |         return get_upload_log_file()
93 |     if name == "QUEUE_FILE":
94 |         return get_queue_file()
95 |     raise AttributeError(f"module {__name__} has no attribute {name}")
96 |
97 | ASSETS_DIR = os.path.join(BASE_DIR, "assets")
98 | BGM_DIR = os.path.join(ASSETS_DIR, "background_music")
99 | FONTS_DIR = os.path.join(ASSETS_DIR, "fonts")
100 |
101 | # Limits & Requirements
102 | # 6 Shorts + 3 long-form videos per channel per day (2 channels = 18 max),
103 | # plus headroom for manual/dispatch runs.
104 | MAX_UPLOADS_PER_DAY = 24
105 |
106 | # Target Gaming Categories (Strictly GTA 5, GTA 6, Indian Bike Driving 3D, and Call of Duty ONLY)
107 | TARGET_GAMES = [
108 |     "Grand Theft Auto V",
109 |     "Grand Theft Auto VI",
110 |     "Indian Bike Driving 3D",
111 |     "Call of Duty: Warzone",
112 |     "Call of Duty: Modern Warfare III",
113 |     "Call of Duty: Black Ops 6"
114 | ]
115 |
116 | def setup_secret_files():
117 |     """Auto-populates secret files if environment variables are provided (e.g. in GitHub Actions)."""
118 |     token_json_env = os.getenv("YOUTUBE_TOKEN_JSON")
119 |     token_path = os.path.join(BASE_DIR, "token.json")
120 |     if token_json_env:
121 |         with open(token_path, "w", encoding="utf-8") as f:
122 |             f.write(token_json_env)
123 |
124 |     client_secret_env = os.getenv("YOUTUBE_CLIENT_SECRET_JSON")
125 |     if client_secret_env:
126 |         with open(CLIENT_SECRETS_FILE, "w", encoding="utf-8") as f:
127 |             f.write(client_secret_env)
128 |
129 |     # Resolve channels.json path to map uppercase env keys to exact case-sensitive channel names
130 |     channels_file = "channels.json"
131 |     import sys
132 |     if "--channels" in sys.argv:
133 |         try:
134 |             idx = sys.argv.index("--channels")
135 |             if idx + 1 < len(sys.argv):
136 |                 channels_file = sys.argv[idx + 1]
137 |         except Exception:
138 |             pass
139 |
140 |     channel_name_map = {}
141 |     if os.path.exists(channels_file):
142 |         try:
143 |             import json
144 |             with open(channels_file, "r", encoding="utf-8") as f:
145 |                 channels_data = json.load(f)
146 |                 if isinstance(channels_data, list):
147 |                     for ch in channels_data:
148 |                         name = ch.get("channel_name")
149 |                         if name:
150 |                             channel_name_map[name.upper()] = name
151 |         except Exception as e:
152 |             print(f"[!] Error loading channels configuration in setup_secret_files: {e}")
153 |
154 |     # Support channel-specific tokens from environment variables on GitHub Actions
155 |     for key, value in os.environ.items():
156 |         if key.startswith("YOUTUBE_TOKEN_JSON_"):
157 |             token_val = (value or "").strip()
158 |             if not token_val:
159 |                 print(f"[!] Warning: Environment variable {key} is empty. Skipping writing to token file.")
160 |                 continue
161 |             channel_name = key[len("YOUTUBE_TOKEN_JSON_"):]
162 |             actual_channel_name = channel_name_map.get(channel_name.upper(), channel_name)
163 |             sanitized = "".join([c if c.isalnum() else "_" for c in actual_channel_name])
164 |             path = os.path.join(DATA_DIR, "channels", sanitized, "token.json")
165 |             os.makedirs(os.path.dirname(path), exist_ok=True)
166 |             with open(path, "w", encoding="utf-8") as f:

```

```

167 |             f.write(token_val)
168 |
169 | def get_ffmpeg_path():
170 |     """
171 |     Returns the path to FFmpeg executable.
172 |     Prioritizes system PATH (works seamlessly on Linux/GitHub Actions & Windows),
173 |     and falls back to local ffmpeg binary if present.
174 |     """
175 |     system_ffmpeg = shutil.which("ffmpeg")
176 |     if system_ffmpeg:
177 |         return system_ffmpeg
178 |
179 |     local_win_ffmpeg = os.path.join(BASE_DIR, "ffmpeg", "bin", "ffmpeg.exe")
180 |     if os.path.exists(local_win_ffmpeg):
181 |         return local_win_ffmpeg
182 |
183 |     return "ffmpeg"
184 |
185 | # Initialize secrets on import if running in environment with secret env vars
186 | setup_secret_files()
187 |

```

Explanation

Lines 1-6 — Imports & env load

Loads `dotenv` and calls `load_dotenv()` so a local `.env` file (never committed) populates `os.environ` before config constants are read. This is the single entry point for local development secrets.

Lines 8-21 — API key constants

Every credential is read with `os.getenv(...).strip()` and defaults to empty string. **Why `.strip()`:** GitHub Actions secrets occasionally carry trailing newlines/whitespace that break JSON parsing or auth; stripping defends against that. Keys cover Twitch (client id/secret), Groq, YouTube Data API, and the notification providers (Twilio + Telegram).

Lines 24-27 — Base & data directories

`BASE_DIR` is the repo root (directory of this file). `DATA_DIR` is `BASE_DIR/data`, the single tree for all per-channel state.

Lines 28-72 — Channel-scoped path helpers

`ACTIVE_CHANNEL_NAME` is a module global set by `set_active_channel()`. Every helper (`get_raw_videos_dir`, `get_clips_dir`, `get_processed_dir`, `get_thumbnails_dir`, `get_seen_videos_file`, `get_upload_log_file`, `get_queue_file`, `get_token_file`) resolves under `data/channels/<sanitized-name>/. Why: isolating each channel's files means concurrent matrix jobs never collide on a shared path, and seen-lists/quotas are per channel. sanitized replaces non-alphanumerics with underscores so channel names with spaces/specials become safe folder names.`

Lines 74-99 — Misc constants & asset dirs

`CLIENT_SECRETS_FILE` points at root `client_secret.json`. `TARGET_NICHES/MAX_SUBS` are fallbacks for the YouTube source path. `ASSETS_DIR/BGM_DIR/FONTS_DIR` point at the bundled music and roboto font.

Lines 101-114 — Upload limits & target games

`MAX_UPLOADS_PER_DAY` = 24 caps total uploads per channel per UTC day (protects against YouTube's daily quota and runaway loops). `TARGET_GAMES` is the default game list used when a channel does not specify `target_games`.

Lines 116-167 — `setup_secret_files()`

Called automatically at import (line 186). **What it does:** if env vars `YOUTUBE_TOKEN_JSON` or `YOUTUBE_CLIENT_SECRET_JSON` exist, it writes them to `token.json` / `client_secret.json`. More importantly it iterates **all** env vars prefixed `YOUTUBE_TOKEN_JSON_`, maps the suffix back to the real (case-sensitive) channel name via `channels.json`, and writes each channel's token to its own folder. **Why:** GitHub Actions stores secrets in `SHOUTING_CASE`, but folders must match channel names exactly (e.g. `GAMES` not `GAMES_...`); this mapping fixes that. This is the bridge between CI secrets and the filesystem layout the uploaders expect.

Lines 169-183 — `get_ffmpeg_path()`

Returns the ffmpeg executable: prefers system PATH (works on Linux CI and Windows), falls back to a bundled `ffmpeg/bin/ffmpeg.exe`, else the bare name `ffmpeg`. Used by every media module so `ffmpeg` is never hard-coded.

Line 186 — `setup_secret_files()` invoked on import

Guarantees token files exist *before* the pipeline imports uploaders.

5.3 channels.json

5.x Source Code — channels.json

```

1 | [
2 |   {
3 |     "channel_name": "ExampleGamingChannel",
4 |     "niche": "Minecraft gaming highlights and funny moments",
5 |     "upload_schedule": {
6 |       "interval_hours": 1.2
7 |     },
8 |     "videos_per_run": 3,
9 |     "youtube_oauth_credentials": "data/channels/ExampleGamingChannel/token.json",
10 |    "source_configuration": {
11 |      "source_type": "twitch",
12 |      "target_games": [
13 |        "Minecraft"
14 |      ]
15 |    },
16 |    "shorts_per_run": 18
17 |  },
18 |  {
19 |    "channel_name": "ExistingChannel",
20 |    "niche": "gaming",
21 |    "upload_schedule": {
22 |      "interval_hours": 2
23 |    },
24 |    "videos_per_run": 3,
25 |    "youtube_oauth_credentials": "data/channels/ExistingChannel/token.json",
26 |    "source_configuration": {
27 |      "source_type": "twitch",
28 |      "target_games": [
29 |        "Minecraft"
30 |      ]
31 |    },
32 |    "shorts_per_run": 18
33 |  },
34 |  {
35 |    "channel_name": "GAMES",
36 |    "niche": "Minecraft gaming highlights and funny moments",
37 |    "upload_schedule": {
38 |      "interval_hours": 1.2
39 |    },
40 |    "videos_per_run": 3,
41 |    "youtube_oauth_credentials": "data/channels/GAMES/token.json",
42 |    "source_configuration": {
43 |      "source_type": "twitch",
44 |      "target_games": [
45 |        "Minecraft"
46 |      ]
47 |    },
48 |    "shorts_per_run": 18
49 |  },
50 |  {
51 |    "channel_name": "MinecraftShorts",
52 |    "niche": "Minecraft gaming highlights and funny moments",
53 |    "upload_schedule": {
54 |      "interval_hours": 1.6
55 |    },
56 |    "videos_per_run": 3,
57 |    "youtube_oauth_credentials": "data/channels/MinecraftShorts/token.json",
58 |    "source_configuration": {
59 |      "source_type": "twitch",
60 |      "target_games": [
61 |        "Minecraft"
62 |      ]
63 |    },
64 |    "shorts_per_run": 15
65 |  },
66 |  {
67 |    "channel_name": "FacebookGaming",
68 |    "niche": "Minecraft gaming highlights and funny moments",
69 |    "platform": "facebook",
70 |    "upload_schedule": {
71 |      "interval_hours": 2
72 |    },
73 |    "videos_per_run": 3,
74 |    "min_video_seconds": 195,
75 |    "clips_per_video": 6,
76 |    "facebook": {
77 |      "page_id_env": "FACEBOOK_PAGE_ID",
78 |      "access_token_env": "FACEBOOK_PAGE_ACCESS_TOKEN"
79 |    },
80 |    "source_configuration": {
81 |      "source_type": "twitch",

```

```

82 |         "target_games": [
83 |             "Minecraft"
84 |         ],
85 |     },
86 |     "shorts_per_run": 0
87 | }
88 | ]
89 |

```

Explanation

Overall structure (lines 1-88)

A JSON array of channel objects. Each object controls one automation target. The pipeline loops over this array (or a filtered single channel).

Common fields

- `channel_name` — unique key; also the folder name under `data/channels/`.
- `niche` — passed to metadata generation (affects titles/hashtags).
- `upload_schedule.interval_hours` — gap between scheduled releases (e.g. 1.2h, 2h).
- `videos_per_run` — how many long-form compilations to make this run (3 for all channels).
- `shorts_per_run` — how many Shorts to make this run.
- `source_configuration` — `source_type`: `twitch` + `target_games` (all channels use Minecraft clips).
- `youtube_oauth_credentials` — path to that channel's `token.json` (used by YouTube uploader).

YouTube channels (ExampleGamingChannel, ExistingChannel, GAMES, MinecraftShorts)

These four keep `shorts_per_run` of 15-18 and `videos_per_run`: 3, platform defaults to YouTube. They are untouched by the Facebook change.

FacebookGaming (lines 66-87) — the modified channel

- `"platform": "facebook"` — routes uploads to `facebook_uploader.py` instead of YouTube.
- `"shorts_per_run": 0` — was 18; now zero because Facebook publishes **only long-form videos**.
- `"min_video_seconds": 195` — the >3-minute (180s) requirement plus ~15s headroom; enforced as a hard gate in `pipeline.py`.
- `"clips_per_video": 6` — minimum clips stitched so a video comfortably exceeds 3 minutes (Twitch clips ≤60s each).
- `"facebook": { page_id_env, access_token_env }` — names the env vars holding the Page ID and long-lived Page Access Token.
- `videos_per_run: 3` — three >3-min videos per run.

Why these knobs exist

Keeping the schedule/counts in JSON (not code) means you can tune throughput, niche, and platform without editing Python. The Facebook block demonstrates a fully data-driven channel type switch.

5.4 twitch_finder.py

5.x Source Code — twitch_finder.py

```

1 | import os
2 | import random
3 | import time
4 | import requests
5 | import logging
6 | from datetime import datetime, timezone, timedelta
7 | import config
8 |
9 | logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
10 |
11 | # Max number of seen clip IDs we remember per channel. The Twitch "trending"
12 | # clips endpoint only returns a few hundred clips, so an ever-growing seen
13 | # list eventually covers the ENTIRE pool and find_twitch_clips returns [] forever
14 | # (this is what left ExampleGamingChannel with "No candidate videos"). Capping it
15 | # lets old clips rotate back into the candidate pool.
16 | MAX_SEEN = 500
17 |
18 | _TWITCH_TOKEN = None
19 |
20 | def get_twitch_access_token():
21 |     """
22 |     Obtains an App Access Token using Twitch Client ID and Client Secret.
23 |     Automatically strips accidental trailing newlines or whitespace.
24 |     """
25 |     global _TWITCH_TOKEN
26 |     if _TWITCH_TOKEN:
27 |         return _TWITCH_TOKEN
28 |
29 |     client_id = (config.TWITCH_CLIENT_ID or "").strip()
30 |     client_secret = (config.TWITCH_CLIENT_SECRET or "").strip()
31 |
32 |     if not client_id or not client_secret:
33 |         logging.error("TWITCH_CLIENT_ID or TWITCH_CLIENT_SECRET environment variable is missing!")
34 |         raise ValueError("TWITCH_CLIENT_ID and TWITCH_CLIENT_SECRET must be set in environment variables.")
35 |
36 |     url = "https://id.twitch.tv/oauth2/token"
37 |     params = {
38 |         "client_id": client_id,
39 |         "client_secret": client_secret,
40 |         "grant_type": "client_credentials"
41 |     }
42 |
43 |     try:
44 |         response = requests.post(url, params=params, timeout=10)
45 |         response.raise_for_status()
46 |         data = response.json()
47 |         _TWITCH_TOKEN = data.get("access_token")
48 |         logging.info("[+] Twitch OAuth Authentication Successful!")
49 |         return _TWITCH_TOKEN
50 |     except Exception as e:
51 |         logging.error(f"[!] Twitch Auth Error: {e}")
52 |         return None
53 |
54 | def get_seen_videos():
55 |     """Reads seen video/clip IDs from the persistence file."""
56 |     seen_file = config.get_seen_videos_file()
57 |     if not os.path.exists(seen_file):
58 |         return set()
59 |     with open(seen_file, "r", encoding="utf-8") as f:
60 |         return set(line.strip() for line in f.readlines() if line.strip())
61 |
62 | def mark_video_seen(video_id):
63 |     """Appends a clip ID to the seen videos file (capped to MAX_SEEN entries)."""
64 |     seen_file = config.get_seen_videos_file()
65 |     parent_dir = os.path.dirname(os.path.abspath(seen_file))
66 |     if parent_dir:
67 |         os.makedirs(parent_dir, exist_ok=True)
68 |     # Cap the file so old clips rotate back into the candidate pool.
69 |     existing = []
70 |     if os.path.exists(seen_file):
71 |         with open(seen_file, "r", encoding="utf-8") as f:
72 |             existing = [l.strip() for l in f.readlines() if l.strip()]
73 |     if video_id in existing:
74 |         return
75 |     existing.append(video_id)
76 |     if len(existing) > MAX_SEEN:
77 |         existing = existing[-MAX_SEEN:]
78 |     with open(seen_file, "w", encoding="utf-8") as f:
79 |         f.write("\n".join(existing) + "\n")
80 |
81 | def get_game_id(game_name, headers):

```

```

82 |     """Resolves a Twitch game name to its Helix game ID."""
83 |     url = f"https://api.twitch.tv/helix/games?name={game_name}"
84 |     try:
85 |         res = requests.get(url, headers=headers, timeout=10)
86 |         res.raise_for_status()
87 |         data = res.json().get("data", [])
88 |         if data:
89 |             return data[0]["id"]
90 |     except Exception as e:
91 |         logging.warning(f"[!] Could not resolve game ID for '{game_name}': {e}")
92 |     return None
93 |
94 | def _collect_unseen(clips, seen, candidates, game_name):
95 |     """Append clips whose id is not yet seen (helper for both query modes)."""
96 |     for clip in clips:
97 |         clip_id = clip.get("id")
98 |         if not clip_id or clip_id in seen:
99 |             continue
100 |         candidates.append({
101 |             "video_id": clip_id,
102 |             "title": clip.get("title", f"{game_name} Twitch Clip"),
103 |             "url": clip.get("url"),
104 |             "thumbnail_url": clip.get("thumbnail_url"),
105 |             "game_name": game_name,
106 |             "duration": clip.get("duration", 30)
107 |         })
108 |
109 |
110 | def find_twitch_clips(target_games=None):
111 |     """
112 |     Queries Twitch Helix API for fresh clips across target game categories.
113 |
114 |     Fix for "No candidate videos" exhaustion: the default clips endpoint only
115 |     returns the CURRENT top ~hundred trending clips, so once a channel's
116 |     seen-list covers that pool it returns [] forever. We now ALSO query a
117 |     RECENCY window (clips created in the last N days) and paginate further,
118 |     so there is always fresh content to upload.
119 |     """
120 |     token = get_twitch_access_token()
121 |     if not token:
122 |         logging.error("Failed to authenticate with Twitch API.")
123 |         return []
124 |
125 |     client_id = (config.TWITCH_CLIENT_ID or "").strip()
126 |     headers = {
127 |         "Client-ID": client_id,
128 |         "Authorization": f"Bearer {token}"
129 |     }
130 |
131 |     seen = get_seen_videos()
132 |     games = list(target_games) if target_games is not None else list(config.TARGET_GAMES)
133 |     random.shuffle(games)
134 |
135 |     candidates = []
136 |
137 |     # Walk back through recent time windows to find fresh, unseen clips.
138 |     # Twitch's Get Clips API requires RFC3339 UTC timestamps WITHOUT fractional
139 |     # seconds (e.g. 2026-08-24T21:55:36Z) and each window must stay small, so
140 |     # we step back in 2h increments across the last ~2 days. The trailing
141 |     # (None, None) keeps the original plain-trending behaviour as a fallback.
142 |     now = datetime.now(timezone.utc)
143 |     windows = []
144 |     for h in range(0, 48, 2): # 2h steps, ~2 days back
145 |         end = now - timedelta(hours=h)
146 |         start = end - timedelta(hours=2)
147 |         windows.append((start, end))
148 |     windows.append((None, None))
149 |
150 |     for game_name in games:
151 |         logging.info(f"[*] Searching Twitch for category: '{game_name}'...")
152 |         game_id = get_game_id(game_name, headers)
153 |         if not game_id:
154 |             continue
155 |
156 |         for (start, end) in windows:
157 |             if candidates:
158 |                 break
159 |             try:
160 |                 cursor = None
161 |                 for page in range(4): # up to 4 pages (400 clips) per window
162 |                     query_url = f"https://api.twitch.tv/helix/clips?game_id={game_id}&first=100"
163 |                     if start is not None:
164 |                         # RFC3339 UTC, seconds precision, 'Z' suffix (no frac/offset)
165 |                         fmt = lambda dt: dt.strftime("%Y-%m-%dT%H:%M:%SZ")
166 |                         query_url += f"&started_at={fmt(start)}&ended_at={fmt(end)}"

```



```

167 |             if cursor:
168 |                 query_url += f"&after={cursor}"
169 |                 res = requests.get(query_url, headers=headers, timeout=10)
170 |                 res.raise_for_status()
171 |                 res_data = res.json()
172 |                 clips = res_data.get("data", [])
173 |                 _collect_unseen(clips, seen, candidates, game_name)
174 |                 cursor = res_data.get("pagination", {}).get("cursor")
175 |                 if not cursor or len(clips) < 100:
176 |                     break
177 |             if candidates:
178 |                 logging.info(f"[+] Found {len(candidates)} unseen candidate clips in '{game_name}' (recency)
179 |                 break
180 |         except Exception as e:
181 |             logging.error(f"[!] Error fetching clips for game '{game_name}': {e}")
182 |             continue
183 |
184 |         if candidates:
185 |             break
186 |
187 |         random.shuffle(candidates)
188 |         return candidates
189 |
190 | if __name__ == "__main__":
191 |     clips = find_twitch_clips()
192 |     print(f"Found {len(clips)} candidate clips.")
193 |

```

Explanation

Lines 1-16 — Imports & MAX_SEEN cap

Standard libs + requests for the Helix API. `MAX_SEEN = 500` caps remembered clip IDs. **Why:** the trending-clips endpoint returns only a few hundred clips; an ever-growing seen-list would eventually cover the entire pool and `find_twitch_clips` would return `[]` forever (this is exactly what stalled `ExampleGamingChannel`). Capping lets old clips rotate back in.

Lines 18-52 — `get_twitch_access_token()`

Obtains a Twitch **App Access Token** via `client_credentials` grant, cached in module global `_TWITCH_TOKEN` (one token per process). Strips whitespace from secrets. Returns `None` on failure so the caller can degrade gracefully.

Lines 54-79 — Seen-list persistence

`get_seen_videos()` reads the channel's `seen_videos.txt` into a set; `mark_video_seen()` appends, de-duplicates, and trims to `MAX_SEEN` (keeps only the last 500). This is the dedup mechanism that prevents re-uploading clips.

Lines 81-92 — `get_game_id()`

Resolves a game name (e.g. 'Minecraft') to its numeric Twitch game id via Helix `/games`; needed because clip queries are by game id, not name.

Lines 94-107 — `_collect_unseen()`

Helper that appends only clips whose id is not in the seen-set, building a candidate dict with `video_id`, `title`, `url`, `thumbnail_url`, `game_name`, `duration`. Centralising this keeps both query modes consistent.

Lines 110-188 — `find_twitch_clips()` [THE CORE FIX]

- Builds OAuth token + headers, loads seen-set, shuffles the game list for variety.
- Builds a list of **recency windows**: 2-hour steps walking back ~2 days (lines 142-148). Twitch's Get-Clips API requires RFC3339 UTC timestamps *without* fractional seconds and small windows; the code formats them as `%Y-%m-%dT%H:%M:%SZ` (line 165).
- For each game it queries each window (up to 4 pages of 100 clips) and collects unseen clips (lines 156-182). The first window that yields candidates wins (line 157/177/184 break).
- This recency-walk replaced the old 'only current trending' query that caused 'No candidate videos' exhaustion. Combined with the 500-cap seen-list, there is now always fresh content.
- **Why RFC3339 with Z and seconds precision:** Twitch returns HTTP 400 on >2h windows or on timestamps with microseconds / offset suffixes; the fix matches the exact format Twitch accepts.

- Finally shuffles candidates and returns them (lines 187-188).

5.5 twitch_downloader.py

5.x Source Code — twitch_downloader.py

```

1 | import os
2 | import re
3 | import time
4 | import logging
5 | import requests
6 | import yt_dlp
7 | import config
8 |
9 | logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
10 |
11 | def get_direct_cdn_mp4_url(thumbnail_url):
12 |     """
13 |     Derives the direct Twitch CDN MP4 video URL from a clip's thumbnail_url.
14 |     Example:
15 |     'https://clips-media-assets2.twitch.tv/AT-cm%7C123456-preview-480x272.jpg'
16 |     -> 'https://clips-media-assets2.twitch.tv/AT-cm%7C123456.mp4'
17 |     """
18 |     if not thumbnail_url:
19 |         return None
20 |     # Strip preview dimensions (-preview-480x272.jpg or -preview.jpg)
21 |     mp4_url = re.sub(r"-preview(-\d+x\d+)?\.(jpg|png|jpeg)$", ".mp4", thumbnail_url)
22 |     if mp4_url != thumbnail_url and mp4_url.endswith(".mp4"):
23 |         return mp4_url
24 |     return None
25 |
26 | def download_file_http(url, out_path, retries=3):
27 |     """Downloads an MP4 file directly via HTTP stream with retries."""
28 |     for attempt in range(1, retries + 1):
29 |         try:
30 |             logging.info(f"[*] HTTP Downloading from CDN (Attempt {attempt}): {url}")
31 |             res = requests.get(url, stream=True, timeout=30)
32 |             res.raise_for_status()
33 |             with open(out_path, "wb") as f:
34 |                 for chunk in res.iter_content(chunk_size=1024 * 1024):
35 |                     if chunk:
36 |                         f.write(chunk)
37 |                     if os.path.exists(out_path) and os.path.getsize(out_path) > 100000:
38 |                         return True
39 |         except Exception as e:
40 |             logging.warning(f"[!] HTTP download attempt {attempt} failed: {e}")
41 |             time.sleep(2)
42 |     return False
43 |
44 | def download_twitch_clip(video_data):
45 |     """
46 |     Downloads a Twitch clip MP4 and saves it to raw videos directory.
47 |     video_data: dict with 'video_id', 'url', 'thumbnail_url'
48 |     Returns absolute path to downloaded MP4 file or None on failure.
49 |     """
50 |     raw_videos_dir = config.get_raw_videos_dir()
51 |     if not os.path.exists(raw_videos_dir):
52 |         os.makedirs(raw_videos_dir, exist_ok=True)
53 |
54 |     if isinstance(video_data, str):
55 |         clip_id = video_data
56 |         clip_url = f"https://clips.twitch.tv/{clip_id}"
57 |         thumbnail_url = None
58 |     else:
59 |         clip_id = video_data.get("video_id")
60 |         clip_url = video_data.get("url", f"https://clips.twitch.tv/{clip_id}")
61 |         thumbnail_url = video_data.get("thumbnail_url")
62 |
63 |     out_file = os.path.join(raw_videos_dir, f"{clip_id}.mp4")
64 |
65 |     # Method 1: Direct Twitch CDN Fast Download
66 |     direct_mp4 = get_direct_cdn_mp4_url(thumbnail_url)
67 |     if direct_mp4:
68 |         if download_file_http(direct_mp4, out_file):
69 |             logging.info(f"[+] Successfully downloaded Twitch clip via CDN: {out_file}")
70 |             return out_file
71 |
72 |     # Method 2: Fallback yt-dlp Native Twitch Extractor
73 |     logging.info(f"[*] Fallback yt-dlp downloading Twitch clip: {clip_url}")
74 |     out_tmpl = os.path.join(raw_videos_dir, f"{clip_id}.{ext}s")
75 |
76 |     ydl_opts = {
77 |         'format': 'best',
78 |         'outtmpl': out_tmpl,
79 |         'merge_output_format': 'mp4',
80 |         'quiet': True,
81 |         'no_warnings': True,

```

```

82 |         'socket_timeout': 30,
83 |         'retries': 5,
84 |     }
85 |
86 |     try:
87 |         with yt_dlp.YoutubeDL(ydl_opts) as ydl:
88 |             info = ydl.extract_info(clip_url, download=True)
89 |             base_filename = ydl.prepare_filename(info)
90 |             final_filename = base_filename.rsplit('.', 1)[0] + '.mp4'
91 |             if os.path.exists(final_filename):
92 |                 logging.info(f"[+] Downloaded Twitch clip via yt-dlp: {final_filename}")
93 |                 return final_filename
94 |             elif os.path.exists(base_filename):
95 |                 return base_filename
96 |     except Exception as e:
97 |         logging.error(f"[!] Error downloading Twitch clip {clip_id}: {e}")
98 |
99 |     return None
100 |
101 | if __name__ == "__main__":
102 |     test_clip = {"video_id": "AwkwardImportantPloverTTours", "url": "https://clips.twitch.tv/AwkwardImportantPloverTTours"}
103 |     res = download_twitch_clip(test_clip)
104 |     print("Download result:", res)
105 |

```

Explanation

Lines 1-9 — Imports & logging

Standard + requests + yt_dlp. Logging at INFO.

Lines 11-24 — get_direct_cdn_mp4_url()

Derives a direct Twitch CDN MP4 from a clip's thumbnail_url by regex-substituting -preview(-WxH).jpg → .mp4. This is the fast path: a plain HTTP GET of the CDN file, far quicker and more reliable than yt-dlp for Twitch clips.

Lines 26-42 — download_file_http()

Streams the CDN MP4 to disk in 1 MB chunks with up to 3 retries. Verifies the file is >100 KB before declaring success. Used by Method 1 below.

Lines 44-99 — download_twitch_clip()

- Normalises input (string clip id or dict), builds the output path under raw_videos.
- **Method 1** (lines 65-70): if a CDN URL was derived, download it directly via HTTP — fastest, no yt-dlp overhead.
- **Method 2** (lines 72-97): fallback to yt_dlp using the native Twitch extractor. Picks 'best' format, merges to mp4, quiet mode, 5 retries.
- Returns the absolute path or None on total failure. The pipeline treats None as 'mark seen and skip'.

Lines 101-104 — __main__ self-test

Lets you run the module directly to download one test clip for debugging.

5.6 video_finder.py

5.x Source Code — video_finder.py

```

1 | import os
2 | import random
3 | from googleapiclient.discovery import build
4 | import config
5 |
6 | def get_seen_videos():
7 |     seen_file = config.get_seen_videos_file()
8 |     if not os.path.exists(seen_file):
9 |         return set()
10 |     with open(seen_file, "r", encoding="utf-8") as f:
11 |         return set(line.strip() for line in f.readlines() if line.strip())
12 |
13 | def mark_video_seen(video_id):
14 |     seen_file = config.get_seen_videos_file()
15 |     parent_dir = os.path.dirname(os.path.abspath(seen_file))
16 |     if parent_dir:
17 |         os.makedirs(parent_dir, exist_ok=True)
18 |     with open(seen_file, "a", encoding="utf-8") as f:
19 |         f.write(f"{video_id}\n")
20 |
21 | def find_videos(niche_query=None, max_subs=None):
22 |     """
23 |     Finds recent unseen gaming videos by querying target niches.
24 |     Iterates through niches until fresh candidates are found.
25 |     """
26 |     api_key = config.YOUTUBE_API_KEY
27 |     if not api_key:
28 |         raise ValueError("YOUTUBE_API_KEY is missing!")
29 |
30 |     youtube = build('youtube', 'v3', developerKey=api_key)
31 |     seen_videos = get_seen_videos()
32 |
33 |     if niche_query:
34 |         niches = [niche_query] if isinstance(niche_query, str) else list(niche_query)
35 |     else:
36 |         niches = list(config.TARGET_NICHES)
37 |
38 |     random.shuffle(niches)
39 |
40 |     limit_subs = max_subs if max_subs is not None else config.MAX_SUBS
41 |     videos = []
42 |
43 |     for niche in niches:
44 |         print(f"[*] Searching YouTube for niche: '{niche}'...")
45 |         try:
46 |             search_response = youtube.search().list(
47 |                 q=niche,
48 |                 part='id,snippet',
49 |                 maxResults=50,
50 |                 type='video',
51 |                 videoDuration='medium',
52 |                 order='date'
53 |             ).execute()
54 |
55 |             for item in search_response.get('items', []):
56 |                 video_id = item['id'].get('videoId')
57 |                 if not video_id:
58 |                     continue
59 |                 channel_id = item['snippet']['channelId']
60 |                 title = item['snippet']['title']
61 |
62 |                 if video_id in seen_videos:
63 |                     continue
64 |
65 |                 # Check channel subscriber count
66 |                 try:
67 |                     channel_response = youtube.channels().list(
68 |                         part='statistics',
69 |                         id=channel_id
70 |                     ).execute()
71 |
72 |                     if channel_response.get('items'):
73 |                         stats = channel_response['items'][0].get('statistics', {})
74 |                         subs = int(stats.get('subscriberCount', 0))
75 |                         if subs >= limit_subs:
76 |                             continue
77 |                 except Exception:
78 |                     pass # If channel stats check fails, keep video as candidate
79 |
80 |                 videos.append({
81 |                     'video_id': video_id,

```

```

82 |             'title': title,
83 |             'channel_id': channel_id
84 |         })
85 |
86 |         if videos:
87 |             print(f"[+] Found {len(videos)} fresh candidate videos in niche '{niche}'")
88 |             break
89 |
90 |         except Exception as e:
91 |             print(f"[!] Search error for niche '{niche}': {e}")
92 |             continue
93 |
94 |         random.shuffle(videos)
95 |         return videos
96 |
97 | def find_playlist_videos(playlist_id):
98 |     """
99 |     Retrieves recent videos from a specified YouTube playlist.
100 |     """
101 |     api_key = config.YOUTUBE_API_KEY
102 |     if not api_key:
103 |         raise ValueError("YOUTUBE_API_KEY is missing!")
104 |
105 |     youtube = build('youtube', 'v3', developerKey=api_key)
106 |     seen_videos = get_seen_videos()
107 |     videos = []
108 |
109 |     print(f"[*] Fetching videos from YouTube Playlist: '{playlist_id}'...")
110 |     try:
111 |         response = youtube.playlistItems().list(
112 |             playlistId=playlist_id,
113 |             part='snippet',
114 |             maxResults=50
115 |         ).execute()
116 |
117 |         for item in response.get('items', []):
118 |             snippet = item.get('snippet', {})
119 |             res_id = snippet.get('resourceId', {})
120 |             video_id = res_id.get('videoId')
121 |             title = snippet.get('title', 'Playlist Video')
122 |
123 |             if video_id and video_id not in seen_videos:
124 |                 videos.append({
125 |                     'video_id': video_id,
126 |                     'title': title
127 |                 })
128 |             print(f"[+] Found {len(videos)} unseen videos in playlist '{playlist_id}'")
129 |         except Exception as e:
130 |             print(f"[!] Error fetching playlist items: {e}")
131 |
132 |         random.shuffle(videos)
133 |         return videos
134 |
135 | if __name__ == "__main__":
136 |     vids = find_videos()
137 |     print(f"Found {len(vids)} matching videos.")
138 |

```

Explanation

Purpose

Alternative **source** when a channel's `source_configuration.source_type` is `youtube_search` or `youtube_playlist` (instead of Twitch). Not used by the current `channels.json` (all use Twitch), but the pipeline imports and supports it.

Lines 1-19 — imports & seen-list

Same dedup pattern as `twitch_finder` but for YouTube video ids; `mark_video_seen` appends without the 500 cap (YouTube pool is effectively unlimited).

Lines 21-95 — `find_videos()`

- Builds the YouTube Data API client with `YOUTUBE_API_KEY`.
- For each niche it runs `search().list(... type=video, videoDuration=medium, order=date)` to get recent medium-length videos.
- For each result it checks the channel's subscriber count via `channels().list` and **skips channels with $\geq$ MAX_SUBS** (keeps content from smaller creators — avoids re-uploading already-huge channels).

- Skips already-seen ids, collects fresh candidates, shuffles, returns.

Lines 97-133 — `find_playlist_videos()`

Fetches items from a given playlist id (used when `source_type=youtube_playlist`), returning unseen video ids/titles.

5.7 downloader.py (YouTube downloader)

5.x Source Code — downloader.py

```

1 | import os
2 | import time
3 | import yt_dlp
4 | import config
5 |
6 | def download_video(video_id):
7 |     """
8 |     Downloads a public video using multi-stage client fallback endpoints
9 |     with cookie authentication logging & verbose diagnostics.
10 |     """
11 |     raw_dir = config.get_raw_videos_dir()
12 |     if not os.path.exists(raw_dir):
13 |         os.makedirs(raw_dir, exist_ok=True)
14 |
15 |     out_tmpl = os.path.join(raw_dir, f"{video_id}.{ext}s")
16 |
17 |     ydl_opts_base = {
18 |         'format': 'bestvideo[height<=1080]+bestaudio/best',
19 |         'outtmpl': out_tmpl,
20 |         'merge_output_format': 'mp4',
21 |         'postprocessor_args': {
22 |             'ffmpeg': ['-strict', 'experimental', '-c:v', 'copy', '-c:a', 'aac']
23 |         },
24 |         'quiet': False,
25 |         'no_warnings': False,
26 |         'verbose': True,
27 |         'socket_timeout': 30,
28 |         'retries': 10,
29 |         'fragment_retries': 10,
30 |         'file_access_retries': 5,
31 |         'concurrent_fragment_downloads': 4,
32 |         'sleep_interval': 2,
33 |         'max_sleep_interval': 4,
34 |     }
35 |
36 |     local_ffmpeg_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'ffmpeg', 'bin')
37 |     if os.path.exists(local_ffmpeg_dir):
38 |         ydl_opts_base['ffmpeg_location'] = local_ffmpeg_dir
39 |
40 |     cookies_env = os.getenv("COOKIES_PATH")
41 |     if cookies_env and os.path.exists(cookies_env):
42 |         cookies_path = os.path.abspath(cookies_env)
43 |     else:
44 |         cookies_path = os.path.join(config.BASE_DIR, "cookies.txt")
45 |
46 |     has_cookies = os.path.exists(cookies_path) and os.path.getsize(cookies_path) > 10
47 |
48 |     print("=== COOKIES DIAGNOSTIC INFO ===")
49 |     print("Cookie file:", cookies_path)
50 |     print("Exists:", os.path.exists(cookies_path))
51 |     if os.path.exists(cookies_path):
52 |         print("Size (bytes):", os.path.getsize(cookies_path))
53 |     print("=====")
54 |
55 |     stages = [
56 |         ("Android VR & Mobile API", ['android_vr', 'android'], True),
57 |         ("iOS & Mobile Web API", ['ios', 'mweb'], True),
58 |         ("Standard Client", ['web', 'mweb'], True)
59 |     ]
60 |
61 |     for stage_name, clients, use_cookies in stages:
62 |         print(f"\n[*] Downloading {video_id} ({stage_name})...")
63 |         opts = dict(ydl_opts_base)
64 |         if use_cookies and has_cookies:
65 |             opts['cookiefile'] = cookies_path
66 |
67 |         opts['extractor_args'] = {'youtube': {'player_client': clients}}
68 |
69 |         try:
70 |             with yt_dlp.YoutubeDL(opts) as ydl:
71 |                 info = ydl.extract_info(f"https://www.youtube.com/watch?v={video_id}", download=True)
72 |                 base_filename = ydl.prepare_filename(info)
73 |                 final_filename = base_filename.rsplit('.', 1)[0] + '.mp4'
74 |                 if os.path.exists(final_filename):
75 |                     print(f"[+] Download succeeded via {stage_name}!")
76 |                     return final_filename
77 |                 elif os.path.exists(base_filename):
78 |                     print(f"[+] Download succeeded via {stage_name}!")
79 |                     return base_filename
80 |             except Exception as e:
81 |                 print(f"[!] {stage_name} attempt failed: {e}")

```



```
82 |             time.sleep(2)
83 |             continue
84 |
85 |         return None
86 |
```

Explanation

Purpose

Downloads a full YouTube video when the source type is YouTube (mirror of `twitch_downloader` for the YouTube source path). Not exercised by current Twitch-only channels but kept for completeness.

Lines 6-38 — `download_video()` setup

Builds yt-dlp options: best 1080p video+audio merge to mp4, with retry/fragment settings, optional local ffmpeg, and cookie-file diagnostics. The cookie diagnostics (lines 48-53) print whether `cookies.txt` exists and its size — useful when downloads are blocked.

Lines 55-83 — multi-stage fallback

Tries three extractor client stages (Android VR/mobile → iOS/mweb → standard web), each with cookie auth, until one succeeds. This resilience handles YouTube's rotating client restrictions that often break a single client.

5.8 audio_analyzer.py

5.x Source Code — audio_analyzer.py

```

1 | import os
2 | import subprocess
3 | import librosa
4 | import numpy as np
5 | import speech_recognition as sr
6 | from config import get_ffmpeg_path
7 |
8 | def analyze_audio(video_path):
9 |     """
10 |     Analyzes audio for human voice AND overall silence.
11 |     Returns: (has_voice: bool, is_silent: bool)
12 |     """
13 |     print("[*] Analyzing audio for human voice and overall sound levels...")
14 |     wav_path = video_path.rsplit('.', 1)[0] + "_sample.wav"
15 |
16 |     try:
17 |         # 1. Extract first 60 seconds of audio for quick analysis
18 |         ffmpeg_path = get_ffmpeg_path()
19 |         cmd = [
20 |             ffmpeg_path, "-y", "-i", video_path,
21 |             "-t", "60", "-ac", "1", "-ar", "16000", wav_path
22 |         ]
23 |         subprocess.run(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
24 |
25 |         if not os.path.exists(wav_path):
26 |             # If FFmpeg failed to make a WAV, the video has no audio track
27 |             return False, True
28 |
29 |         # 2. Spectral Analysis (librosa)
30 |         y, sr_lib = librosa.load(wav_path, sr=16000)
31 |
32 |         # Check for absolute silence
33 |         rms = librosa.feature.rms(y=y)
34 |         mean_rms = np.mean(rms)
35 |
36 |         # 0.005 is a very low threshold. If below this, video is basically mute.
37 |         is_silent = mean_rms < 0.005
38 |
39 |         if is_silent:
40 |             print("[+] Audio is completely silent (no game sounds or voice).")
41 |             return False, True
42 |
43 |         # Continue with Voice Detection
44 |         S = np.abs(librosa.stft(y))
45 |
46 |         # Vocal frequency band is roughly 300Hz - 3000Hz
47 |         freqs = librosa.fft_frequencies(sr=sr_lib)
48 |         vocal_band = np.where((freqs > 300) & (freqs < 3000))[0]
49 |         vocal_energy = np.sum(S[vocal_band, :])
50 |         total_energy = np.sum(S)
51 |
52 |         vocal_ratio = vocal_energy / (total_energy + 1e-6)
53 |
54 |         # 3. Speech Recognition Check
55 |         recognizer = sr.Recognizer()
56 |         with sr.AudioFile(wav_path) as source:
57 |             audio_data = recognizer.record(source)
58 |             try:
59 |                 text = recognizer.recognize_google(audio_data)
60 |                 if len(text.strip()) > 0:
61 |                     print(f"[-] Voice detected via Speech Recognition: '{text}'")
62 |                     return True, False
63 |             except sr.UnknownValueError:
64 |                 pass # No recognizable speech
65 |             except sr.RequestError:
66 |                 pass # API error, fallback to spectral
67 |
68 |         # Spectral fallback if API fails or is ambiguous
69 |         if vocal_ratio > 0.65:
70 |             print(f"[-] Voice detected via Spectral Analysis (Ratio: {vocal_ratio:.2f})")
71 |             return True, False
72 |
73 |         print("[+] Good game sounds found. No human voice detected.")
74 |         return False, False
75 |
76 |     except Exception as e:
77 |         print(f"[!] Audio analysis error: {e}")
78 |         return True, False
79 |
80 |     finally:
81 |         if os.path.exists(wav_path):
82 |             try:

```

```
82 |             os.remove(wav_path)
83 |         except Exception:
84 |             pass
85 |
```

Explanation

Lines 1-7 — imports

Uses librosa (audio analysis), numpy, and speech_recognition (Google SR).

Lines 8-14 — analyze_audio() entry

Extracts the first 60s of audio to a temp WAV via ffmpeg, then analyses.

Lines 16-27 — extract & silence check

Runs ffmpeg to produce a 16 kHz mono WAV. If no WAV is produced the video has no audio track → returns (False, True) (no voice, silent). Computes RMS energy; if below 0.005 the clip is effectively mute → (False, True).

Lines 29-74 — voice vs game-audio detection

- Computes a short-time Fourier transform and measures energy in the vocal band 300-3000 Hz.
- If `vocal_ratio > 0.65` it treats the clip as containing human voice.
- Also tries Google Speech Recognition on the WAV; any recognised text confirms voice.
- Returns (True, False) when voice is detected, else (False, False) (clean game audio).

Lines 76-84 — finally

Always deletes the temp WAV to avoid disk bloat.

Where used

pipeline.py calls this to decide `mute_original`: voice → mute original + add BGM; silent → add BGM; clean game audio → keep original under low BGM.

5.9 highlight_detector.py

5.x Source Code — highlight_detector.py

```

1 | import os
2 | import subprocess
3 | from config import get_ffmpeg_path
4 |
5 | def get_video_duration(video_path):
6 |     """Probes the video file using ffprobe to retrieve duration in seconds."""
7 |     try:
8 |         ffmpeg_path = get_ffmpeg_path()
9 |         # Resolve ffprobe path by replacing ffmpeg with ffprobe
10 |         if "ffmpeg.exe" in ffmpeg_path:
11 |             ffprobe_path = ffmpeg_path.replace("ffmpeg.exe", "ffprobe.exe")
12 |         elif "ffmpeg" in ffmpeg_path:
13 |             ffprobe_path = ffmpeg_path.replace("ffmpeg", "ffprobe")
14 |         else:
15 |             ffprobe_path = "ffprobe"
16 |
17 |         cmd = [
18 |             ffprobe_path, "-v", "error", "-show_entries", "format=duration",
19 |             "-of", "default=noprint_wrappers=1:nokey=1", video_path
20 |         ]
21 |         result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True)
22 |         return float(result.stdout.strip())
23 |     except Exception as e:
24 |         print(f"[!] Error getting video duration: {e}")
25 |         return 0.0
26 |
27 | def get_highlights(video_path, num_clips=1):
28 |     """
29 |     Bypasses complex audio energy and scene detection.
30 |     Always cuts a reliable segment from the video to guarantee successful processing.
31 |     """
32 |     print("[*] Detecting highlights (Simplified Cut logic)...")
33 |     duration = get_video_duration(video_path)
34 |
35 |     if duration <= 0:
36 |         # Default fallback if duration could not be determined
37 |         print("[!] Could not determine video duration. Using fallback range.")
38 |         return [{"start": 10.0, "end": 45.0, "duration": 35.0}]
39 |
40 |     print(f"[+] Video duration detected: {duration:.1f}s")
41 |
42 |     # Target clip duration of 35 seconds (leads to a ~29.1s Short after 1.2x speedup)
43 |     target_clip_dur = 35.0
44 |
45 |     if duration <= 45.0:
46 |         # For short clips (already under 45s), use the full clip
47 |         start_time = 0.0
48 |         end_time = duration
49 |         clip_dur = duration
50 |     else:
51 |         # For longer videos, cut a 35s segment from the middle
52 |         start_time = (duration - target_clip_dur) / 2.0
53 |         end_time = start_time + target_clip_dur
54 |         clip_dur = target_clip_dur
55 |
56 |     return [{
57 |         "start": start_time,
58 |         "end": end_time,
59 |         "duration": clip_dur
60 |     }]
61 |
62 |
63 | def get_full_clip_range(video_path, max_duration=60.0):
64 |     """
65 |     Returns a start/end range covering the WHOLE clip for long-form compilation
66 |     videos. Twitch clips are short (typically < 60s) so the entire clip is used;
67 |     unexpectedly long sources get a middle cut instead.
68 |     """
69 |     duration = get_video_duration(video_path)
70 |
71 |     if duration <= 0:
72 |         # Default fallback if duration could not be determined
73 |         print("[!] Could not determine video duration. Using fallback range.")
74 |         return {"start": 0.0, "end": 45.0, "duration": 45.0}
75 |
76 |     if duration <= max_duration:
77 |         return {"start": 0.0, "end": duration, "duration": duration}
78 |
79 |     start_time = (duration - max_duration) / 2.0
80 |     return {"start": start_time, "end": start_time + max_duration, "duration": max_duration}
81 |

```

Explanation

Lines 1-3 — imports

Uses ffmpeg path resolver for probing.

Lines 5-25 — `get_video_duration()`

Probes any video with ffprobe and returns duration in seconds (0.0 on failure). **Note:** an identical-named helper also exists in `video_processor.py` (imported by pipeline for the Facebook >3-min gate). Both use ffprobe; the pipeline imports from `video_processor`.

Lines 27-60 — `get_highlights()`

Simplified cut logic for the **Short** path. It does NOT do heavy scene detection (that proved unreliable); instead it returns a reliable single 35s window (mid-clip for long videos, full clip if ≤ 45 s). At 1.2x speedup that yields a ~29s Short, comfortably under YouTube's 60s Shorts limit.

Lines 63-80 — `get_full_clip_range()`

Used by the **compilation** path: returns the whole clip (Twitch clips <60s) or a middle 60s cut for unexpectedly long sources. The pipeline stitches these full ranges together.

5.10 video_processor.py

5.x Source Code — video_processor.py

```

1 | import os
2 | import random
3 | import shutil
4 | import subprocess
5 | import ffmpeg
6 | import config
7 |
8 | def get_random_bgm():
9 |     if not os.path.exists(config.BGM_DIR): return None
10 |     files = [f for f in os.listdir(config.BGM_DIR) if f.endswith('.mp3')]
11 |     if not files: return None
12 |     return os.path.join(config.BGM_DIR, random.choice(files))
13 |
14 | def get_font_path():
15 |     if not os.path.exists(config.FONTS_DIR): return None
16 |     files = [f for f in os.listdir(config.FONTS_DIR) if f.endswith('.ttf')]
17 |     if not files: return None
18 |     return os.path.join(config.FONTS_DIR, files[0])
19 |
20 | def process_video(video_path, start, end, output_filename, mute_original=False):
21 |     """
22 |     Processes a landscape gameplay video into a high-quality YouTube Short.
23 |     Guarantees strict duration <= 45s for YouTube Shorts compliance.
24 |     """
25 |     processed_dir = config.get_processed_dir()
26 |     if not os.path.exists(processed_dir):
27 |         os.makedirs(processed_dir, exist_ok=True)
28 |
29 |     out_path = os.path.join(processed_dir, output_filename)
30 |
31 |     # Cap raw clip duration to 45s max (speedup 1.2x produces ~37s Short)
32 |     clip_duration = min(max(end - start, 15.0), 45.0)
33 |     print(f"[*] Processing Short clip ({start:.1f}s -> {start+clip_duration:.1f}s, target duration: {clip_duration:.1f}s)")
34 |
35 |     stream = ffmpeg.input(video_path, ss=start, t=clip_duration)
36 |     video = stream.video
37 |     audio = stream.audio
38 |
39 |     # 1. Scale down to 1920 height
40 |     v = video.filter("scale", -1, 1920, flags="lanczos")
41 |
42 |     # 2. Crop center to 9:16 (1080x1920)
43 |     v = v.filter("crop", w=1080, h=1920, x="(in_w-1080)/2", y=0)
44 |
45 |     # 3. Speed up video to 1.2x
46 |     v = v.filter("setpts", "0.833333*PTS")
47 |
48 |     # 4. Light denoise
49 |     v = v.filter("hqdn3d", 1.0, 1.0, 3.0, 3.0)
50 |
51 |     # 5. Color balance
52 |     v = v.filter("colorbalance", rs=0.18, gs=0.18, bs=0.18)
53 |
54 |     # 6. HDR-style curve
55 |     v = v.filter("curves", m="0/0 0.25/0.20 0.5/0.55 0.75/0.83 1/1")
56 |
57 |     # 7. Sharpen
58 |     v = v.filter("unsharp", 5, 5, 1.2, 5, 5, 0.0)
59 |
60 |     # ENCODER SETTINGS (Optimized for speed & multi-threading)
61 |     kwargs = {
62 |         "vcodec": "libx264",
63 |         "acodec": "aac",
64 |         "preset": "fast",
65 |         "crf": 20,
66 |         "pix_fmt": "yuv420p",
67 |         "movflags": "+faststart",
68 |         "strict": "experimental",
69 |         "loglevel": "error",
70 |         "threads": 0,
71 |         "shortest": None # Crucial: cut output at shortest stream (video length)
72 |     }
73 |
74 |     bgm_path = get_random_bgm()
75 |     if mute_original:
76 |         if bgm_path:
77 |             bgm = (
78 |                 ffmpeg
79 |                 .input(bgm_path)
80 |                 .audio
81 |                 .filter("atrim", duration=clip_duration)

```

```

82 |         .filter("volume", 0.8)
83 |     )
84 |     out = ffmpeg.output(v, bgm, out_path, **kwargs)
85 |     else:
86 |         out = ffmpeg.output(v, out_path, an=None, **kwargs)
87 |     else:
88 |         audio = audio.filter("atempo", 1.2)
89 |         if bgm_path:
90 |             bgm = (
91 |                 ffmpeg
92 |                 .input(bgm_path)
93 |                 .audio
94 |                 .filter("atrim", duration=clip_duration)
95 |                 .filter("volume", 0.15)
96 |             )
97 |             audio = ffmpeg.filter([audio, bgm], "amix", inputs=2, duration="first")
98 |
99 |         out = ffmpeg.output(v, audio, out_path, **kwargs)
100 |
101 |     try:
102 |         ffmpeg_cmd = config.get_ffmpeg_path()
103 |         out.run(overwrite_output=True, cmd=ffmpeg_cmd)
104 |         print(f"[+] YouTube Short saved: {out_path}")
105 |         return out_path
106 |     except ffmpeg.Error as e:
107 |         print("\n===== FFMPEG ERROR =====")
108 |         if e.stderr:
109 |             print(e.stderr.decode())
110 |         else:
111 |             print(str(e))
112 |         print("=====\n")
113 |         return None
114 |
115 |
116 | def get_video_duration(video_path):
117 |     """
118 |     Returns the actual duration (seconds, float) of a rendered video file using
119 |     ffprobe. Returns 0.0 if ffprobe is unavailable or the file can't be probed.
120 |     Used to HARD-VERIFY a minimum length requirement before upload (e.g. every
121 |     Facebook video must be > 3 minutes).
122 |     """
123 |     if not video_path or not os.path.exists(video_path):
124 |         return 0.0
125 |     probe = shutil.which("ffprobe") or os.path.join(
126 |         os.path.dirname(config.get_ffmpeg_path()), "ffprobe"
127 |     )
128 |     if not probe or not os.path.exists(probe):
129 |         # ffprobe missing – fall back to a best-effort estimate via ffmpeg
130 |         probe = config.get_ffmpeg_path()
131 |     try:
132 |         result = subprocess.run(
133 |             [probe, "-v", "error", "-show_entries", "format=duration",
134 |              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
135 |             stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=60
136 |         )
137 |         out = result.stdout.decode().strip()
138 |         if out:
139 |             return float(out)
140 |     except (subprocess.SubprocessError, ValueError, OSError):
141 |         pass
142 |     return 0.0
143 |
144 |
145 | def process_compilation_video(clip_segments, output_filename, max_duration=600, min_duration=0):
146 |     """
147 |     Builds a long-form YouTube video (16:9 1080p) by stitching several clip
148 |     segments together into a single file. Each segment receives the same color /
149 |     denoise / sharpen grade as the Shorts processor, original audio is preserved,
150 |     and BGM is laid underneath at low volume. Output is capped at max_duration
151 |     seconds (default 10 min, safe under YouTube's long-form limit).
152 |
153 |     clip_segments: list of (video_path, start, end) tuples.
154 |     Returns absolute path to the final MP4 or None on failure.
155 |     """
156 |     processed_dir = config.get_processed_dir()
157 |     os.makedirs(processed_dir, exist_ok=True)
158 |
159 |     ffmpeg_cmd = config.get_ffmpeg_path()
160 |     base = os.path.splitext(output_filename)[0]
161 |     part_paths = []
162 |     list_path = os.path.join(processed_dir, f"{base}_concat.txt")
163 |     merged_path = os.path.join(processed_dir, f"{base}_merged.mp4")
164 |     out_path = os.path.join(processed_dir, output_filename)
165 |
166 |     try:

```

```

167 |         # Step A: grade & normalize each segment into uniform 16:9 1080p parts
168 |         for i, (video_path, start, end) in enumerate(clip_segments):
169 |             dur = min(max(end - start, 5.0), 90.0)
170 |             part_path = os.path.join(processed_dir, f"{base}_part{i}.mp4")
171 |
172 |             stream = ffmpeg.input(video_path, ss=start, t=dur)
173 |             v = (
174 |                 stream.video
175 |                 .filter("scale", 1920, 1080, force_original_aspect_ratio="decrease")
176 |                 .filter("pad", 1920, 1080, "(ow-iw)/2", "(oh-ih)/2")
177 |                 .filter("hqdn3d", 1.0, 1.0, 3.0, 3.0)
178 |                 .filter("colorbalance", rs=0.18, gs=0.18, bs=0.18)
179 |                 .filter("curves", m="0/0 0.25/0.20 0.5/0.55 0.75/0.83 1/1")
180 |                 .filter("unsharp", 5, 5, 1.2, 5, 5, 0.0)
181 |             )
182 |
183 |             # Try with original audio (normalized params so concat can stream-copy)
184 |             has_audio = False
185 |             try:
186 |                 a = stream.audio.filter(
187 |                     "aformat",
188 |                     sample_fmts="fltp", sample_rates=44100, channel_layouts="stereo"
189 |                 )
190 |                 has_audio = True
191 |             except (AttributeError, ffmpeg.Error):
192 |                 pass
193 |
194 |             if has_audio:
195 |                 try:
196 |                     part = ffmpeg.output(
197 |                         v, a, part_path,
198 |                         vcodec="libx264", acodec="aac", preset="fast", crf=18,
199 |                         pix_fmt="yuv420p", movflags="+faststart",
200 |                         strict="experimental", loglevel="error", threads=0
201 |                     )
202 |                     part.run(overwrite_output=True, cmd=ffmpeg_cmd)
203 |                 except ffmpeg.Error:
204 |                     has_audio = False
205 |
206 |             if not has_audio:
207 |                 part = ffmpeg.output(
208 |                     v, part_path,
209 |                     vcodec="libx264", preset="fast", crf=18,
210 |                     pix_fmt="yuv420p", movflags="+faststart",
211 |                     loglevel="error", threads=0
212 |                 )
213 |                 part.run(overwrite_output=True, cmd=ffmpeg_cmd)
214 |
215 |             part_paths.append(part_path)
216 |
217 |         if len(part_paths) < 1:
218 |             return None
219 |
220 |         # Step B: concatenate parts (identical encoding params -> stream copy)
221 |         with open(list_path, "w", encoding="utf-8") as f:
222 |             for p in part_paths:
223 |                 escaped = p.replace("\\", "/").replace("'", "'\\'")
224 |                 f.write(f"file '{escaped}'\n")
225 |
226 |         concat_in = ffmpeg.input(list_path, f="concat", safe=0)
227 |         try:
228 |             concat_in.output(merged_path, c="copy", loglevel="error").run(
229 |                 overwrite_output=True, cmd=ffmpeg_cmd
230 |             )
231 |         except ffmpeg.Error:
232 |             concat_in.output(
233 |                 merged_path, vcodec="libx264", acodec="aac", preset="fast", crf=18,
234 |                 pix_fmt="yuv420p", loglevel="error", threads=0
235 |             ).run(overwrite_output=True, cmd=ffmpeg_cmd)
236 |
237 |         # Step C: lay BGM underneath the original audio
238 |         bgm_path = get_random_bgm()
239 |         if bgm_path:
240 |             merged = ffmpeg.input(merged_path)
241 |             bgm = (
242 |                 ffmpeg.input(bgm_path).audio
243 |                 .filter("atrim", duration=max_duration)
244 |                 .filter("volume", 0.15)
245 |             )
246 |             mixed = ffmpeg.filter([merged.audio, bgm], "amix", inputs=2, duration="first")
247 |             ffmpeg.output(
248 |                 merged.video, mixed, out_path,
249 |                 vcodec="copy", acodec="aac", preset="fast", t=max_duration,
250 |                 movflags="+faststart", loglevel="error", threads=0
251 |             ).run(overwrite_output=True, cmd=ffmpeg_cmd)

```



```

252 |         else:
253 |             os.replace(merged_path, out_path)
254 |
255 |         if os.path.exists(out_path) and os.path.getsize(out_path) > 100000:
256 |             print(f"[+] Long-form video saved: {out_path}")
257 |             return out_path
258 |         return None
259 |
260 |     except ffmpeg.Error as e:
261 |         print("\n===== FFMPEG ERROR (compilation) =====")
262 |         if e.stderr:
263 |             print(e.stderr.decode())
264 |         else:
265 |             print(str(e))
266 |         print("=====\n")
267 |         return None
268 |
269 |     finally:
270 |         # Always clean up intermediates (raw/processed dirs are wiped later too)
271 |         for p in part_paths + [list_path, merged_path]:
272 |             try:
273 |                 if os.path.exists(p):
274 |                     os.unlink(p)
275 |             except Exception:
276 |                 pass
277 |

```

Explanation

Lines 1-6 — imports

Includes the `ffmpeg` Python wrapper (used to build filter graphs) and `config`.

Lines 8-18 — BGM & font helpers

`get_random_bgm()` picks a random NCS track from `assets/background_music`; `get_font_path()` returns the bundled roboto font (reserved for overlays).

Lines 20-113 — `process_video()` (SHORT renderer)

- Creates the processed dir and an output path.
- Caps the cut to 45s max (line 32) so the sped-up Short stays <45s for YouTube compliance.
- Builds an `ffmpeg` filter chain (lines 35-58): scale to 1080p height, crop centre to 9:16 (1080x1920), 1.2x speedup, light denoise, colour balance, HDR-style curve, sharpen — a consistent 'premium' look.
- Encoder settings (lines 60-72): `libx264`, `aac`, fast preset, `crf 20`, `yuv420p`, `faststart`. BGM is mixed under original audio (low 0.15 vol) unless `mute_original` is set (then BGM at 0.8).
- Runs via the resolved `ffmpeg` binary; on `ffmpeg.Error` prints the `stderr` for debugging and returns `None`.

Lines 116-142 — `get_video_duration()`

Duplicated `fprobe` helper (the one pipeline actually imports). Returns 0.0 if the file is missing or probe fails. **This is the function that powers the Facebook >3-minute hard gate.**

Lines 145-277 — `process_compilation_video()` (LONG-FORM renderer)

- Takes a list of (`video_path`, `start`, `end`) segments.
- **Step A** (lines 166-215): grades & normalises each segment to a uniform 16:9 1080p part with the same colour/sharpen treatment, preserving original audio (normalised sample rate/channels so concat can stream-copy). If a segment has no audio it renders video-only.
- **Step B** (lines 217-235): writes a concat list file and concatenates the parts with stream-copy (fast); falls back to re-encode if the copy path errors.
- **Step C** (lines 237-253): mixes BGM under the merged audio at 0.15 volume; outputs the final MP4 capped at `max_duration` (default 600s/10min). If no BGM, just renames the merged file.
- Returns the final path only if it is >100 KB; cleans up all intermediate parts in `finally` (lines 269-276).

5.11 metadata_generator.py

5.x Source Code — metadata_generator.py

```

1 | import json
2 | import os
3 | import re
4 | from groq import Groq
5 | from config import GROQ_API_KEY
6 |
7 | # Families we prefer for chat/JSON generation, in priority order.
8 | # Used to rank models returned by the live /models list.
9 | _CHAT_PREFER = (
10 |     "llama-4", "llama-3.3", "llama-3.1", "deepseek", "qwen",
11 |     "gemma", "gpt-oss", "kimi", "mistral",
12 | )
13 | # Model id substrings that are NOT text-chat models (skip them).
14 | _SPEECH_HINTS = ("whisper", "tts", "playai", "distil-whisper")
15 |
16 | # Best-guess current live fallbacks, used only if the live /models list
17 | # cannot be fetched (e.g. key/network issue). Cycled through on failure.
18 | _FALLBACK_MODELS = [
19 |     "llama-4-scout-17b-16e-instruct",
20 |     "llama-3.3-70b-versatile",
21 |     "deepseek-r1-distill-llama-70b",
22 |     "gpt-oss-120b",
23 | ]
24 |
25 | _MODEL_CACHE = None # module-level cache so we query /models at most once per run
26 |
27 |
28 | def _available_chat_models(client):
29 |     """Return chat-capable Groq model ids, preferred families first.
30 |
31 |     Self-healing: queries the live /models endpoint so decommissioned models
32 |     are never used. Falls back to a static list if the API is unreachable.
33 |     """
34 |     global _MODEL_CACHE
35 |     if _MODEL_CACHE is not None:
36 |         return _MODEL_CACHE
37 |     ids = []
38 |     try:
39 |         resp = client.models.list()
40 |         ids = [m.id for m in resp.data]
41 |     except Exception as e:
42 |         print(f"[-] Could not list Groq models ({e}); using built-in list.")
43 |     text = [i for i in ids if not any(h in i for h in _SPEECH_HINTS)]
44 |     text.sort(key=lambda i: next(
45 |         (idx for idx, p in enumerate(_CHAT_PREFER) if p in i), len(_CHAT_PREFER)))
46 |     seen, ordered = set(), []
47 |     for i in text:
48 |         if i not in seen:
49 |             seen.add(i)
50 |             ordered.append(i)
51 |     _MODEL_CACHE = ordered or list(_FALLBACK_MODELS)
52 |     if _MODEL_CACHE and _MODEL_CACHE is not _FALLBACK_MODELS:
53 |         print(f"[*] Groq live models available ({len(_MODEL_CACHE)}); will use in order.")
54 |     return _MODEL_CACHE
55 |
56 |
57 | def _extract_json(text):
58 |     """Pull a JSON object out of a model response.
59 |
60 |     Models sometimes wrap JSON in ```json fences or add prose around it. With
61 |     response_format=json_object most return raw JSON, but a few models still
62 |     emit fenced output that Groq's server-side validator rejects – so we parse
63 |     defensively instead of trusting the forced mode.
64 |     """
65 |     if text is None:
66 |         return None
67 |     text = text.strip()
68 |     try:
69 |         return json.loads(text)
70 |     except Exception:
71 |         pass
72 |     # Strip ```json ... ``` fences
73 |     fenced = re.search(r"```(?:json)?\s*(\{.*?\})\s*```", text, re.DOTALL)
74 |     if fenced:
75 |         try:
76 |             return json.loads(fenced.group(1))
77 |         except Exception:
78 |             pass
79 |     # Last resort: first balanced-looking {...}
80 |     brace = re.search(r"\{.*\}", text, re.DOTALL)
81 |     if brace:

```

```

82 |         try:
83 |             return json.loads(brace.group(0))
84 |         except Exception:
85 |             pass
86 |     return None
87 |
88 |
89 | def _validate_metadata(obj):
90 |     """Ensure the parsed object has the fields pipeline expects."""
91 |     if not isinstance(obj, dict):
92 |         return False
93 |     if not obj.get("title") or not isinstance(obj.get("title"), str):
94 |         return False
95 |     if not obj.get("description") or not isinstance(obj.get("description"), str):
96 |         return False
97 |     return True
98 |
99 |
100 | def _call_groq(client, prompt):
101 |     """Try every available model until one returns valid JSON metadata.
102 |
103 |     Robust against Groq's `json_validate_failed` (HTTP 400): when a model
104 |     fails validation under forced json_object mode, we retry the SAME model
105 |     WITHOUT the json_object mode (most models emit clean JSON that way) before
106 |     moving on to the next model. This stops the noisy model-failed spam and
107 |     gets a result from the preferred model instead of always falling back.
108 |     """
109 |     models = _available_chat_models(client)
110 |     last_err = None
111 |     for model in models:
112 |         # Attempt 1: forced JSON mode (best for well-behaved models)
113 |         try:
114 |             response = client.chat.completions.create(
115 |                 messages=[{"role": "user", "content": prompt}],
116 |                 model=model,
117 |                 response_format={"type": "json_object"},
118 |                 temperature=0.7,
119 |             )
120 |             obj = _extract_json(response.choices[0].message.content)
121 |             if obj and _validate_metadata(obj):
122 |                 return obj, model
123 |         except Exception as e:
124 |             last_err = e
125 |             err_text = str(e)
126 |             # 'json_validate_failed' -> retry same model without json mode
127 |             if "json_validate_failed" in err_text:
128 |                 try:
129 |                     response = client.chat.completions.create(
130 |                         messages=[{"role": "user", "content": prompt}],
131 |                         model=model,
132 |                         temperature=0.7,
133 |                     )
134 |                     obj = _extract_json(response.choices[0].message.content)
135 |                     if obj and _validate_metadata(obj):
136 |                         return obj, model
137 |                 except Exception as e2:
138 |                     last_err = e2
139 |             # fall through to next model
140 |         # Attempt 2 (used if forced-JSON returned bad/empty content but didn't raise)
141 |         if True:
142 |             try:
143 |                 response = client.chat.completions.create(
144 |                     messages=[{"role": "user", "content": prompt}],
145 |                     model=model,
146 |                     temperature=0.7,
147 |                 )
148 |                 obj = _extract_json(response.choices[0].message.content)
149 |                 if obj and _validate_metadata(obj):
150 |                     return obj, model
151 |             except Exception as e:
152 |                 last_err = e
153 |                 print(f"[!] Groq model '{model}' failed: {last_err}; trying next model...")
154 |                 print(f"[!] All Groq models failed ({last_err}); using fallback metadata.")
155 |     return None, None
156 |
157 |
158 | def generate_metadata(video_title, niche="gaming"):
159 |     """Generate Shorts metadata (title, description, tags) via Groq."""
160 |     print(f"[*] Generating AI Metadata for niche '{niche}'...")
161 |     niche_clean = "".join(c for c in niche if c.isalnum())
162 |     fallback_data = {
163 |         "title": f"Epic {niche.capitalize()} Moment ■ #shorts",
164 |         "description": f"Wait for the end of this crazy {niche} video!\n\n■ Like and Subscribe for more amazing",
165 |         "hashtags": ["#shorts", f"#{niche_clean}", "#epic", "#moments", "#highlights", "#clip", "#viral"],
166 |         "tags": f"{niche},shorts,epic,funny,moments,highlights,clip,viral,video",

```

```

167 |     }
168 |
169 |     if not GROQ_API_KEY:
170 |         print("[-] GROQ_API_KEY missing. Using fallback metadata.")
171 |         return fallback_data
172 |
173 |     client = Groq(api_key=GROQ_API_KEY)
174 |     prompt = f"""
175 |     You are an expert YouTube Shorts creator. Generate metadata for a Shorts video in the "{niche}" niche, cura
176 |
177 |     Output strictly in this JSON format:
178 |     {{
179 |         "title": "<Click-worthy title under 80 characters with 1-2 emojis>",
180 |         "description": "<SEO friendly description under 300 characters, ending with a Call to Action>",
181 |         "hashtags": ["#shorts", ... 9 more relevant tags based on the niche],
182 |         "tags": "comma, separated, list, of, 20, youtube, seo, tags, related to this niche/video"
183 |     }}
184 |     """
185 |
186 |     data, used = _call_groq(client, prompt)
187 |     if not data:
188 |         return fallback_data
189 |
190 |     print(f"[+] Groq metadata generated with model '{used}'.")
191 |     if "#shorts" not in data.get("hashtags", []):
192 |         data["hashtags"] = ["#shorts"] + data.get("hashtags", [])[:9]
193 |     return data
194 |
195 |
196 | def generate_video_metadata(video_title, niche="gaming"):
197 |     """Generate LONG-FORM compilation metadata. Hashtags never include #shorts."""
198 |     print(f"[*] Generating AI Video Metadata for niche '{niche}'...")
199 |     niche_clean = "".join(c for c in niche if c.isalnum())
200 |     fallback_data = {
201 |         "title": f"INSANE {niche.upper()} MOMENTS - Best Plays Compilation 🎮",
202 |         "description": f"The craziest {niche} moments you need to see!\n\n👉 Like and Subscribe for more amazing
203 |         "hashtags": [f"{niche_clean}", "#gaming", "#highlights", "#bestmoments", "#epic", "#viral", "#compilat
204 |         "tags": f"{niche},gaming,highlights,compilation,best moments,epic,gameplay,twitch,viral,funny,top plays
205 |     }
206 |
207 |     if not GROQ_API_KEY:
208 |         print("[-] GROQ_API_KEY missing. Using fallback metadata.")
209 |         return fallback_data
210 |
211 |     client = Groq(api_key=GROQ_API_KEY)
212 |     prompt = f"""
213 |     You are an expert YouTube gaming channel creator. Generate metadata for a LONG-FORM compilation video in th
214 |     This is a regular YouTube video, NOT a Short, so hashtags must NOT include #shorts.
215 |
216 |     Output strictly in this JSON format:
217 |     {{
218 |         "title": "<Click-worthy title under 100 characters with 1-2 emojis>",
219 |         "description": "<SEO friendly description under 500 characters, ending with a Call to Action>",
220 |         "hashtags": ["... 9 relevant tags based on the niche, no #shorts],
221 |         "tags": "comma, separated, list, of, 20, youtube, seo, tags, related to this niche/video"
222 |     }}
223 |     """
224 |
225 |     data, used = _call_groq(client, prompt)
226 |     if not data:
227 |         return fallback_data
228 |
229 |     print(f"[+] Groq video metadata generated with model '{used}'.")
230 |     data["hashtags"] = [h for h in data.get("hashtags", []) if h.lower() != "#shorts"]
231 |     if len(data["hashtags"]) < 9:
232 |         data["hashtags"] += fallback_data["hashtags"][: 9 - len(data["hashtags"])]
233 |     return data
234 |

```

Explanation

Lines 1-25 — model preference tables

Defines preferred chat-model families (`_CHAT_PREFER`) and speech-model hints to skip (`_SPEECH_HINTS`), plus static `_FALLBACK_MODELS` used only if the live model list can't be fetched.

Lines 28-54 — `_available_chat_models()` [SELF-HEALING]

Queries Groq's live `/models` endpoint, filters out speech models, orders preferred families first, caches the result in `_MODEL_CACHE` (queried at most once per run). If the API is unreachable it returns the static fallback list. **Why:** Groq occasionally decommissions models; self-healing avoids hard-coding a model that may vanish.

Lines 57-86 — `_extract_json()`

Defensively parses JSON from a model response: tries raw parse, then strips ``json fences, then grabs the first balanced { . . . }. Models sometimes wrap JSON in prose, so defensive parsing prevents false failures.

Lines 89-97 — `_validate_metadata()`

Ensures the parsed object has non-empty title and description strings before accepting it.

Lines 100-155 — `_call_groq()` [RETRY LOGIC]

- Iterates available models. Attempt 1 uses forced `json_object` mode.
- If Groq raises `json_validate_failed` (HTTP 400), it RETRIES the SAME model WITHOUT json mode (most models emit clean JSON that way) before moving on — this stops noisy model-failed spam and gets a result from the preferred model.
- Attempt 2 (lines 141-152) is a plain call for cases where forced-JSON returned empty content but didn't raise.
- If all models fail it returns `(None, None)` so the caller uses fallback metadata.

Lines 158-193 — `generate_metadata()` (SHORTS)

Builds a Shorts prompt, calls `_call_groq`, and ensures `#shorts` is present in hashtags. Returns Groq output or a safe templated fallback when `GROQ_API_KEY` is missing.

Lines 196-233 — `generate_video_metadata()` (LONG-FORM)

Same idea but explicitly forbids `#shorts` in hashtags (it's a regular video) and tops up to 9 hashtags from the fallback list.

5.12 youtube_uploader.py

5.x Source Code — youtube_uploader.py

```

1 | import os
2 | import json
3 | import logging
4 | from datetime import datetime, timezone
5 | from googleapiclient.discovery import build
6 | from googleapiclient.http import MediaFileUpload
7 | from google_auth_oauthlib.flow import InstalledAppFlow
8 | from google.auth.transport.requests import Request
9 | from google.oauth2.credentials import Credentials
10 | from google.auth.exceptions import TransportError
11 | import config
12 |
13 | SCOPES = ['https://www.googleapis.com/auth/youtube.upload']
14 |
15 | def get_authenticated_service(token_info=None, token_path=None):
16 |     creds = None
17 |
18 |     # Resolve the token path to save refreshed/new credentials
19 |     target_token_path = token_path or config.get_token_file()
20 |
21 |     # 1. Try to load from provided token_info dict
22 |     if token_info and isinstance(token_info, dict):
23 |         try:
24 |             creds = Credentials.from_authorized_user_info(token_info, SCOPES)
25 |         except Exception as e:
26 |             logging.error(f"[!] Error building credentials from token_info dict: {e}")
27 |
28 |     # 2. Try to load from channel token file path if dict load failed or wasn't provided
29 |     if not creds:
30 |         if os.path.exists(target_token_path):
31 |             try:
32 |                 creds = Credentials.from_authorized_user_file(target_token_path, SCOPES)
33 |             except Exception as e:
34 |                 logging.error(f"[!] Error reading authorized user file {target_token_path}: {e}")
35 |         elif os.path.exists('token.json'):
36 |             # Backward compatibility fallback
37 |             try:
38 |                 creds = Credentials.from_authorized_user_file('token.json', SCOPES)
39 |                 target_token_path = 'token.json'
40 |             except Exception as e:
41 |                 logging.error(f"[!] Error reading fallback token.json: {e}")
42 |
43 |     # 3. Refresh or authenticate via flow
44 |     if not creds or not creds.valid:
45 |         if creds and creds.expired and creds.refresh_token:
46 |             try:
47 |                 logging.info("[*] Refreshing expired YouTube credentials...")
48 |                 creds.refresh(Request())
49 |                 logging.info("[+] YouTube credentials refreshed successfully!")
50 |             except TransportError as te:
51 |                 logging.error(f"[!] Network connection error during token refresh: {te}")
52 |                 raise te
53 |             except Exception as e:
54 |                 logging.error(f"[!] Failed to refresh credentials: {e}. Falling back to InstalledAppFlow.")
55 |                 creds = None
56 |
57 |         if not creds:
58 |             logging.error("[!] No valid credentials or refresh token available. Attempting InstalledAppFlow log
59 |             flow = InstalledAppFlow.from_client_secrets_file(config.CLIENT_SECRETS_FILE, SCOPES)
60 |             creds = flow.run_local_server(port=0)
61 |
62 |         # Ensure parent directory exists for token path
63 |         parent_dir = os.path.dirname(os.path.abspath(target_token_path))
64 |         if parent_dir:
65 |             os.makedirs(parent_dir, exist_ok=True)
66 |
67 |         with open(target_token_path, 'w', encoding='utf-8') as token:
68 |             token.write(creds.to_json())
69 |
70 |     return build('youtube', 'v3', credentials=creds)
71 |
72 | def get_upload_count_today():
73 |     log_file = config.get_upload_log_file()
74 |     if not os.path.exists(log_file):
75 |         return 0
76 |     with open(log_file, 'r', encoding='utf-8') as f:
77 |         try:
78 |             logs = json.load(f)
79 |             today = datetime.now(timezone.utc).strftime('%Y-%m-%d')
80 |             return sum(1 for log in logs if log.get('date') == today)
81 |         except json.JSONDecodeError:

```

```

82 |         return 0
83 |
84 | def log_upload(video_id):
85 |     log_file = config.get_upload_log_file()
86 |     logs = []
87 |     if os.path.exists(log_file):
88 |         with open(log_file, 'r', encoding='utf-8') as f:
89 |             try:
90 |                 logs = json.load(f)
91 |             except:
92 |                 pass
93 |
94 |     logs.append({
95 |         'video_id': video_id,
96 |         'date': datetime.now(timezone.utc).strftime('%Y-%m-%d'),
97 |         'time': datetime.now(timezone.utc).isoformat()
98 |     })
99 |
100 | # Ensure parent dir exists
101 | parent_dir = os.path.dirname(os.path.abspath(log_file))
102 | if parent_dir:
103 |     os.makedirs(parent_dir, exist_ok=True)
104 |
105 | with open(log_file, 'w', encoding='utf-8') as f:
106 |     json.dump(logs, f, indent=4)
107 |
108 | def _upload(video_path, metadata, schedule_time=None, token_info=None, token_path=None, is_short=True):
109 |     """
110 |     Uploads a video to YouTube via OAuth2.
111 |     If schedule_time is provided, sets privacyStatus to 'private' and publishAt to future ISO 8601 UTC timestamp.
112 |     is_short=True appends #shorts to the title/description (Shorts uploads);
113 |     is_short=False uploads a regular long-form video without #shorts.
114 |     """
115 |     content_kind = "Short" if is_short else "video"
116 |     logging.info(f"[*] Uploading {video_path} to YouTube {content_kind}...")
117 |     youtube = get_authenticated_service(token_info=token_info, token_path=token_path)
118 |
119 |     tags = [t.strip() for t in metadata['tags'].split(',')] if isinstance(metadata['tags'], str) else metadata['tags']
120 |     description = metadata['description'] + "\n\n" + " ".join(metadata['hashtags'])
121 |     if is_short and "#shorts" not in description.lower():
122 |         description += " #shorts"
123 |
124 |     title = metadata['title']
125 |     if is_short:
126 |         if "#shorts" not in title.lower():
127 |             title = f"{title[:75]} #shorts"
128 |     else:
129 |         title = title[:100]
130 |
131 |     status_payload = {
132 |         'privacyStatus': 'public',
133 |         'selfDeclaredMadeForKids': False
134 |     }
135 |
136 |     if schedule_time:
137 |         # Explicit ISO 8601 formatting required by YouTube API: YYYY-MM-DDTHH:MM:SS.000Z
138 |         formatted_publish_time = schedule_time.strftime('%Y-%m-%dT%H:%M:%S.000Z')
139 |         status_payload['privacyStatus'] = 'private'
140 |         status_payload['publishAt'] = formatted_publish_time
141 |         logging.info(f"[*] Scheduling YouTube Video Status Payload: {json.dumps(status_payload)}")
142 |
143 |     body = {
144 |         'snippet': {
145 |             'title': title,
146 |             'description': description,
147 |             'tags': tags,
148 |             'categoryId': '20' # Gaming
149 |         },
150 |         'status': status_payload
151 |     }
152 |
153 |     media = MediaFileUpload(video_path, mimetype='video/mp4', resumable=True)
154 |
155 |     request = youtube.videos().insert(
156 |         part='snippet,status',
157 |         body=body,
158 |         media_body=media
159 |     )
160 |
161 |     response = None
162 |     while response is None:
163 |         status, response = request.next_chunk()
164 |         if status:
165 |             logging.info(f"[-] Uploaded {int(status.progress() * 100)}%")
166 |

```

```

167 |     logging.info(f"[+] Upload Complete! Video ID: {response['id']} (Status: {status_payload['privacyStatus']})")
168 |     log_upload(response['id'])
169 |     return response['id']
170 |
171 |
172 | def upload_short(video_path, metadata, schedule_time=None, token_info=None, token_path=None):
173 |     """
174 |     Uploads a short to YouTube via OAuth2 (adds #shorts to title/description).
175 |     If schedule_time is provided, sets privacyStatus to 'private' and publishAt to future ISO 8601 UTC timestamp.
176 |     """
177 |     return _upload(
178 |         video_path, metadata,
179 |         schedule_time=schedule_time, token_info=token_info, token_path=token_path,
180 |         is_short=True
181 |     )
182 |
183 |
184 | def upload_video(video_path, metadata, schedule_time=None, token_info=None, token_path=None):
185 |     """
186 |     Uploads a regular (long-form, non-short) video to YouTube via OAuth2.
187 |     No '#shorts' is appended to the title/description.
188 |     If schedule_time is provided, sets privacyStatus to 'private' and publishAt to future ISO 8601 UTC timestamp.
189 |     """
190 |     return _upload(
191 |         video_path, metadata,
192 |         schedule_time=schedule_time, token_info=token_info, token_path=token_path,
193 |         is_short=False
194 |     )
195 |

```

Explanation

Lines 1-13 — imports & SCOPES

Google API client + OAuth2 libs. Scope is `youtube.upload` only (least privilege).

Lines 15-70 — `get_authenticated_service()`

- Loads credentials from a passed dict (`token_info`) or the channel `token.json` path, with a backward-compat fallback to root `token.json`.
- If expired but refreshable, refreshes via `creds.refresh()`.
- If no valid creds, falls back to `InstalledAppFlow` (opens a browser) — only used for local first-time auth.
- Persists refreshed creds back to the token file. Returns a YouTube API service object.

Lines 72-106 — quota logging

`get_upload_count_today()` counts today's entries in the channel `upload_log.json`; `log_upload()` appends a record. These feed the daily quota gate in `pipeline.py`.

Lines 108-169 — `_upload()` [CORE]

- Builds tags list and description (title + hashtags). For Shorts it appends `#shorts` to title (truncated to 75 chars) and description.
- If `schedule_time` is given, sets `privacyStatus=private` + `publishAt` (formatted exactly `YYYY-MM-DDTHH:MM:SS.000Z`, which YouTube requires).
- Uploads resumably (`MediaFileUpload(..., resumable=True)`) in a `next_chunk()` loop, logging progress.
- On success logs the upload and returns the video id.

Lines 172-194 — `upload_short()` / `upload_video()`

Thin wrappers setting `is_short=True/False`. The Facebook channel never calls these (it calls `upload_fb_video`).

5.13 facebook_uploader.py

5.x Source Code — facebook_uploader.py

```

1 | """
2 | Facebook Page video uploader (Graph API).
3 |
4 | Mirrors youtube_uploader.py: uploads the SAME processed short / long-form
5 | video to a Facebook Page instead of YouTube. Used when a channel in
6 | channels.json has "platform": "facebook".
7 |
8 | Auth model (kept simple, free-tier, no extra deps beyond `requests`):
9 | - A long-lived Facebook **Page Access Token** is provided via the
10 |   FACEBOOK_PAGE_ACCESS_TOKEN secret (GitHub Actions) or env var.
11 | - The target Page ID is provided via FACEBOOK_PAGE_ID.
12 | - Alternatively a per-channel "facebook" block can name custom
13 |   env-var keys (see channels.json).
14 |
15 | Behaviour:
16 | - Long-form videos: uploaded with scheduled_publish_time (matches the
17 |   channel's interval-based schedule, just like YouTube publishAt).
18 |   Because Facebook's simple /videos upload rejects large files
19 |   (HTTP 413 "Request Entity Too Large"), long-form videos use the
20 |   RESUMABLE (chunked) upload endpoint, which has a much higher limit.
21 | - Shorts / vertical clips: small files, so the simple /videos upload is
22 |   fine. Facebook does NOT support scheduling Reels via the Graph API,
23 |   so they are published immediately (published=true).
24 | - If scheduling fails, it transparently falls back to immediate publish
25 |   so the video is never lost.
26 | - All failures raise; the caller (pipeline.py) treats Facebook uploads as
27 |   BEST-EFFORT (never aborts the channel run).
28 | """
29 | import os
30 | import logging
31 | import requests
32 |
33 | # Graph API version. If you get a "Unsupported version" / 403, bump this
34 | # (e.g. v20.0, v21.0). Overridable via FACEBOOK_GRAPH_VERSION env var.
35 | GRAPH_VERSION = os.getenv("FACEBOOK_GRAPH_VERSION", "v19.0")
36 | GRAPH_BASE = f"https://graph.facebook.com/{GRAPH_VERSION}"
37 |
38 | # Above this size we switch to the resumable (chunked) upload endpoint,
39 | # which avoids Facebook's 413 "Request Entity Too Large" on big files.
40 | RESUMABLE_THRESHOLD = 80 * 1024 * 1024 # 80 MB
41 | CHUNK_SIZE = 4 * 1024 * 1024 # 4 MB per chunk (FB allows up to 8 MB)
42 |
43 |
44 | def get_fb_credentials(channel):
45 |     """Return (page_id, access_token) or (None, None) if FB is not configured."""
46 |     fb_cfg = (channel or {}).get("facebook")
47 |     if isinstance(fb_cfg, dict):
48 |         page_id = (
49 |             fb_cfg.get("page_id")
50 |             or os.getenv(fb_cfg.get("page_id_env", "FACEBOOK_PAGE_ID"))
51 |         )
52 |         token = os.getenv(fb_cfg.get("access_token_env", "FACEBOOK_PAGE_ACCESS_TOKEN"))
53 |     else:
54 |         # Fall back to global env vars
55 |         page_id = os.getenv("FACEBOOK_PAGE_ID")
56 |         token = os.getenv("FACEBOOK_PAGE_ACCESS_TOKEN")
57 |
58 |     page_id = (page_id or "").strip()
59 |     token = (token or "").strip()
60 |     if not page_id or not token:
61 |         return None, None
62 |     return page_id, token
63 |
64 |
65 | def _raise_for_error(resp, params_hint=""):
66 |     """Turn a non-200 Graph response into an actionable Exception."""
67 |     data = resp.json() if resp.content else {}
68 |     if resp.status_code == 200 and "id" in data:
69 |         return data["id"]
70 |     err = data.get("error", {}) if isinstance(data, dict) else {}
71 |     code = err.get("code")
72 |     subcode = err.get("error_subcode")
73 |     if code == 190 and subcode == 463:
74 |         raise Exception(
75 |             "Facebook token EXPIRED (code 190/subcode 463). The FACEBOOK_PAGE_ACCESS_TOKEN "
76 |             "secret holds a short-lived token. Regenerate a LONG-LIVED Page Access Token "
77 |             "(-60 days) and update the GitHub secret 'FACEBOOK_PAGE_ACCESS_TOKEN'."
78 |         )
79 |     if code == 190:
80 |         raise Exception(
81 |             "Facebook token invalid (code 190). Check FACEBOOK_PAGE_ACCESS_TOKEN / FACEBOOK_PAGE_ID."

```

```

82 |         )
83 |         raise Exception(f"Facebook API {resp.status_code}: {data}")
84 |
85 |
86 | def _post_video_simple(page_id, access_token, video_path, title, description,
87 |                       is_short, schedule_time=None):
88 |     """Simple /videos upload (best for small files: shorts/reels)."""
89 |     params = {
90 |         "access_token": access_token,
91 |         "title": title[:255],
92 |         "description": description,
93 |         "published": "false" if (schedule_time and not is_short) else "true",
94 |     }
95 |     if schedule_time and not is_short:
96 |         params["scheduled_publish_time"] = str(int(schedule_time.timestamp()))
97 |     if is_short:
98 |         params["video_reels_publish"] = "true"
99 |
100 |     url = f"{GRAPH_BASE}/{page_id}/videos"
101 |     with open(video_path, "rb") as f:
102 |         files = {"source": (os.path.basename(video_path), f, "video/mp4")}
103 |         resp = requests.post(url, data=params, files=files, timeout=900)
104 |         return _raise_for_error(resp)
105 |
106 |
107 | def _post_video_resumable(page_id, access_token, video_path, title, description,
108 |                           is_short, schedule_time=None):
109 |     """
110 |     Resumable (chunked) upload for LARGE files (compilation videos).
111 |
112 |     Steps (Facebook Graph resumable video upload):
113 |     1. POST /{page}/videos?upload_phase=start&access_token=... -> {upload_session_id, video_id, ...}
114 |     2. POST /{page}/videos?upload_phase=transfer&upload_session_id=...&start_offset=0
115 |        with the file chunk as multipart 'video_file_chunk' -> {start_offset, end_offset}
116 |     3. Repeat transfer until end_offset == file_size
117 |     4. POST /{page}/videos?upload_phase=finish&upload_session_id=...&title=...&description=...
118 |        [&published=false&scheduled_publish_time=...] -> {success: true}
119 |     """
120 |     file_size = os.path.getsize(video_path)
121 |
122 |     # 1. start
123 |     start = requests.post(
124 |         f"{GRAPH_BASE}/{page_id}/videos",
125 |         data={"access_token": access_token, "upload_phase": "start"},
126 |         timeout=120,
127 |     )
128 |     start_data = start.json() if start.content else {}
129 |     if start.status_code != 200 or "upload_session_id" not in start_data:
130 |         raise Exception(f"Facebook resumable START failed: {start_data}")
131 |     session_id = start_data["upload_session_id"]
132 |
133 |     # 2. transfer chunks
134 |     offset = 0
135 |     with open(video_path, "rb") as f:
136 |         while offset < file_size:
137 |             f.seek(offset)
138 |             chunk = f.read(CHUNK_SIZE)
139 |             transfer = requests.post(
140 |                 f"{GRAPH_BASE}/{page_id}/videos",
141 |                 data={
142 |                     "access_token": access_token,
143 |                     "upload_phase": "transfer",
144 |                     "upload_session_id": session_id,
145 |                     "start_offset": str(offset),
146 |                 },
147 |                 files={"video_file_chunk": ("chunk", chunk, "application/octet-stream")},
148 |                 timeout=900,
149 |             )
150 |             tdata = transfer.json() if transfer.content else {}
151 |             if transfer.status_code != 200 or "end_offset" not in tdata:
152 |                 raise Exception(f"Facebook resumable TRANSFER failed at offset {offset}: {tdata}")
153 |             offset = int(tdata["end_offset"])
154 |
155 |     # 3. finish
156 |     finish_params = {
157 |         "access_token": access_token,
158 |         "upload_phase": "finish",
159 |         "upload_session_id": session_id,
160 |         "title": title[:255],
161 |         "description": description,
162 |         "published": "false" if (schedule_time and not is_short) else "true",
163 |     }
164 |     if schedule_time and not is_short:
165 |         finish_params["scheduled_publish_time"] = str(int(schedule_time.timestamp()))
166 |     finish = requests.post(

```

```

167 |         f"{GRAPH_BASE}/{page_id}/videos",
168 |         data=finish_params,
169 |         timeout=300,
170 |     )
171 |     fdata = finish.json() if finish.content else {}
172 |     if finish.status_code != 200 or not fdata.get("success"):
173 |         # If finish gives a video_id anyway, treat as success.
174 |         if "video_id" in fdata:
175 |             return fdata["video_id"]
176 |         raise Exception(f"Facebook resumable FINISH failed: {fdata}")
177 |     return fdata.get("video_id") or session_id
178 |
179 |
180 | def _post_video(page_id, access_token, video_path, title, description,
181 |                is_short, schedule_time=None):
182 |     """Pick upload strategy.
183 |
184 |     - Shorts/Reels (small files): simple multipart upload, upgrade to resumable
185 |       if Facebook rejects it with 413.
186 |     - Long-form videos (compilations): ALWAYS use resumable chunked upload.
187 |       Facebook's simple /{page}/videos multipart endpoint returns 413
188 |       "Request Entity Too Large" even for moderate sizes (observed on ~9-80MB
189 |       compilation files), and resumable is the supported path for any video.
190 |     """
191 |     if is_short:
192 |         try:
193 |             return _post_video_simple(page_id, access_token, video_path,
194 |                                       title, description, is_short, schedule_time)
195 |         except Exception as e:
196 |             if "413" in str(e) or "Request Entity Too Large" in str(e):
197 |                 logging.warning("[!] Facebook simple upload 413 (file too large); switching to resumable...")
198 |                 return _post_video_resumable(page_id, access_token, video_path,
199 |                                              title, description, is_short, schedule_time)
200 |             raise
201 |     # Long-form video: always resumable (avoids the 413 on simple /videos).
202 |     try:
203 |         return _post_video_resumable(page_id, access_token, video_path,
204 |                                       title, description, is_short, schedule_time)
205 |     except Exception as e:
206 |         if "413" in str(e):
207 |             # Already resumable but still 413? nothing more we can do.
208 |             raise
209 |         logging.warning(f"[!] Facebook resumable upload failed ({e}); trying simple upload...")
210 |         return _post_video_simple(page_id, access_token, video_path,
211 |                                   title, description, is_short, schedule_time)
212 |
213 |
214 | def upload_fb_video(video_path, metadata, schedule_time=None,
215 |                    channel=None, is_short=True):
216 |     """
217 |     Upload a processed video to a Facebook Page.
218 |
219 |     Mirrors youtube_uploader.upload_short / upload_video signatures so the
220 |     pipeline can swap the destination based on channel["platform"].
221 |     Returns the Facebook post/video id.
222 |     """
223 |     if not os.path.exists(video_path):
224 |         raise Exception(f"Facebook upload: file not found: {video_path}")
225 |
226 |     page_id, token = get_fb_credentials(channel)
227 |     if not page_id or not token:
228 |         raise Exception(
229 |             "Facebook credentials not configured. Set FACEBOOK_PAGE_ID and "
230 |             "FACEBOOK_PAGE_ACCESS_TOKEN (GitHub secrets or env vars)."
231 |         )
232 |
233 |     raw_title = metadata.get("title", "Gaming Highlight")
234 |     # Facebook has no "#shorts" convention; strip it for cleanliness.
235 |     title = raw_title.replace("#shorts", "").replace("#Shorts", "").strip() or "Gaming Highlight"
236 |
237 |     # Strip any "#shorts"/"#Shorts" from incoming hashtags too (FB has no such tag).
238 |     def _clean_tags(t):
239 |         return " ".join(w for w in t.split() if w.lower() != "#shorts")
240 |
241 |     description = metadata.get("description", "")
242 |     if isinstance(metadata.get("hashtags"), list):
243 |         description += "\n\n" + _clean_tags(" ".join(metadata["hashtags"]))
244 |     elif isinstance(metadata.get("hashtags"), str):
245 |         description += "\n\n" + _clean_tags(metadata["hashtags"])
246 |     # Facebook has no #shorts tag convention; use neutral gaming tags.
247 |     description += "\n\n#gaming #minecraft #reels" if is_short else "\n\n#gaming #minecraft"
248 |
249 |     kind = "Short/Reel" if is_short else "video"
250 |     logging.info(f"[*] Uploading {video_path} to Facebook Page {page_id} ({kind})...")
251 |

```

```

252 |         try:
253 |             post_id = _post_video(page_id, token, video_path, title, description,
254 |                                   is_short, schedule_time)
255 |             if schedule_time and not is_short:
256 |                 logging.info(f"[+] Facebook video scheduled (post id {post_id}) for {schedule_time.strftime('%Y-%m-%d %H:%M:%S')}")
257 |             else:
258 |                 logging.info(f"[+] Facebook {kind} published (post id {post_id})")
259 |             return post_id
260 |         except Exception as e:
261 |             # Reels can't be scheduled; if a scheduled *video* call failed for a
262 |             # scheduling reason, retry immediately so the content still goes out.
263 |             if schedule_time and not is_short:
264 |                 logging.warning(f"[!] Facebook scheduled upload failed ({e}); retrying as immediate publish...")
265 |                 return _post_video(page_id, token, video_path, title, description,
266 |                                     is_short, None)
267 |             raise
268 |

```

Explanation

Lines 1-40 — module doc & constants

Documents the auth model (long-lived Page Access Token via FACEBOOK_PAGE_ACCESS_TOKEN) and the 413 problem. RESUMABLE_THRESHOLD=80MB and CHUNK_SIZE=4MB drive the chunked upload path.

Lines 44-62 — get_fb_credentials()

Reads Page ID + token from the channel's facebook block env-var names (or globals). Returns (None, None) if unconfigured so the pipeline can skip FB gracefully.

Lines 65-83 — _raise_for_error()

Turns a non-200 Graph response into an actionable exception. Notably detects token expiry code 190 / subcode 463 and tells the operator to regenerate a LONG-LIVED token — this is the exact failure mode that previously blocked Facebook uploads.

Lines 86-104 — _post_video_simple()

Multipart POST to /{page_id}/videos. For scheduled long-form videos sets published=false + scheduled_publish_time; Reels are published immediately (video_reels_publish=true).

Lines 107-177 — _post_video_resumable() [413 FIX]

Implements Facebook's 3-phase chunked upload: (1) start → get upload_session_id; (2) transfer chunks in a loop reading the file in 4MB pieces and tracking end_offset; (3) finish with title/description/schedule. **This path avoids the HTTP 413 'Request Entity Too Large' that the simple endpoint threw on compilation videos.**

Lines 180-211 — _post_video() router

Shorts try simple first, upgrade to resumable on 413. Long-form videos ALWAYS use resumable (the 413 was observed even at ~9-80MB). If resumable still fails with 413 there is nothing more to do; otherwise it falls back to simple.

Lines 214-267 — upload_fb_video()

- Entry point mirroring upload_short/upload_video signatures.
- Strips #shorts from title/description/hashtags (Facebook has no such tag), adds neutral gaming tags.
- Calls _post_video. If a *scheduled* long-form upload fails for any reason, it retries once as immediate publish so the content is never lost.
- Returns the Facebook post/video id. The pipeline treats FB uploads as best-effort (a failure never aborts the whole channel run).

Relationship to the >3-minute requirement

This module uploads whatever pipeline.py hands it. The >3-min guarantee lives in pipeline.py (it only calls this with videos that already passed the ffmpeg duration gate).

5.14 notifier.py

5.x Source Code — notifier.py

```

1 | import logging
2 | import requests
3 | from requests.auth import HTTPBasicAuth
4 | import config
5 |
6 | logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
7 |
8 | def format_report_message(report):
9 |     msg_lines = ["■ YouTube Shorts Automation Report", " "*30]
10 |     for r in report:
11 |         status_emoji = "■" if r['status'] == "Success" else "■■" if r['status'] == "Partial Success" else "■■■"
12 |         msg_lines.append(f"{status_emoji} {r['channel_name']}: {r['status']}")
13 |         msg_lines.append(f"    Shorts Created: {r['shorts_created']}")
14 |         msg_lines.append(f"    Videos Created: {r.get('videos_created', 0)}")
15 |         if r['error']:
16 |             msg_lines.append(f"    Error: {r['error'][:80]}")
17 |         msg_lines.append(" "*30)
18 |     return "\n".join(msg_lines)
19 |
20 | def send_twilio_sms(body):
21 |     if not (config.TWILIO_ACCOUNT_SID and config.TWILIO_AUTH_TOKEN and config.TWILIO_FROM_NUMBER and config.USER_PHONE_NUMBER):
22 |         logging.warning("Twilio credentials missing. Skipping SMS.")
23 |         return
24 |     url = f"https://api.twilio.org/2010-04-01/Accounts/{config.TWILIO_ACCOUNT_SID}/Messages.json"
25 |     data = {
26 |         "To": config.USER_PHONE_NUMBER,
27 |         "From": config.TWILIO_FROM_NUMBER,
28 |         "Body": body
29 |     }
30 |     try:
31 |         response = requests.post(url, data=data, auth=HTTPBasicAuth(config.TWILIO_ACCOUNT_SID, config.TWILIO_AUTH_TOKEN))
32 |         response.raise_for_status()
33 |         logging.info("[+] SMS notification sent successfully via Twilio.")
34 |     except Exception as e:
35 |         logging.error(f"[!] Failed to send Twilio SMS: {e}")
36 |
37 | def send_telegram_msg(body):
38 |     if not (config.TELEGRAM_BOT_TOKEN and config.TELEGRAM_CHAT_ID):
39 |         logging.warning("Telegram credentials missing. Skipping Telegram notification.")
40 |         return
41 |     url = f"https://api.telegram.org/bot{config.TELEGRAM_BOT_TOKEN}/sendMessage"
42 |     payload = {
43 |         "chat_id": config.TELEGRAM_CHAT_ID,
44 |         "text": body
45 |     }
46 |     try:
47 |         response = requests.post(url, json=payload, timeout=10)
48 |         response.raise_for_status()
49 |         logging.info("[+] Telegram notification sent successfully.")
50 |     except Exception as e:
51 |         logging.error(f"[!] Failed to send Telegram notification: {e}")
52 |
53 | def notify_report(report):
54 |     if config.NOTIFICATION_PROVIDER == "NONE":
55 |         return
56 |
57 |     body = format_report_message(report)
58 |
59 |     if config.NOTIFICATION_PROVIDER == "TWILIO":
60 |         send_twilio_sms(body)
61 |     elif config.NOTIFICATION_PROVIDER == "TELEGRAM":
62 |         send_telegram_msg(body)
63 |

```

Explanation

Lines 1-18 — format_report_message()

Builds a concise emoji-coded summary (Success/Partial/Failed/Skipped) per channel from the run report.

Lines 20-35 — send_twilio_sms()

Sends the summary via Twilio if all four Twilio secrets are set; otherwise skips silently.

Lines 37-51 — send_telegram_msg()

Sends via Telegram Bot API if bot token + chat id are set.

Lines 53-62 — `notify_report()`

Dispatches based on `NOTIFICATION_PROVIDER` (TELEGRAM / TWILIO / NONE). Called once at the end of every run so the owner gets a push notification.

5.15 gen_token.py

5.x Source Code — gen_token.py

```

1 | #!/usr/bin/env python3
2 | """
3 | Generate a YouTube OAuth token.json for one of your channels (local dev only).
4 |
5 | Usage:
6 |     python gen_token.py <CHANNEL_NAME>
7 |
8 | What happens:
9 |     1. Reads client_secret.json (root of this repo, gitignored).
10 |    2. Opens your browser -> login with the Google/YouTube account that OWNS
11 |       that channel -> approve the upload scope.
12 |    3. Writes data/channels/<CHANNEL_NAME>/token.json
13 |    4. Prints the JSON content so you can paste it into the GitHub secret
14 |       YOUTUBE_TOKEN_JSON-<CHANNEL_NAME> (or upload the file yourself).
15 |
16 | The token file is self-contained (includes its own client_id/client_secret),
17 | so it refreshes without the client_secret.json present in GitHub Actions.
18 | """
19 | import argparse
20 | import json
21 | import os
22 | import sys
23 |
24 | # Allow running from anywhere inside the repo
25 | BASE_DIR = os.path.dirname(os.path.abspath(__file__))
26 | sys.path.insert(0, BASE_DIR)
27 | os.chdir(BASE_DIR)
28 |
29 | SCOPES = ["https://www.googleapis.com/auth/youtube.upload"]
30 |
31 | def main():
32 |     parser = argparse.ArgumentParser(description=__doc__.split("Usage:")[0])
33 |     parser.add_argument("channel", help="exact channel_name from channels.json (e.g. GAMES)")
34 |     parser.add_argument("--client-secret", help="optional path to a specific client_secret.json "
35 |                        "(default: data/channels/<CHANNEL>/client_secret.json, then client_secret.json)")
36 |     args = parser.parse_args()
37 |
38 |     sanitized = "".join([c if c.isalnum() else "_" for c in args.channel])
39 |     token_path = os.path.join(BASE_DIR, "data", "channels", sanitized, "token.json")
40 |
41 |     # Prefer a channel-specific client secret, fall back to the shared root one.
42 |     candidates = [
43 |         args.client_secret,
44 |         os.path.join(BASE_DIR, "data", "channels", sanitized, "client_secret.json"),
45 |         os.path.join(BASE_DIR, "client_secret.json"),
46 |     ]
47 |     client_secret_path = next((c for c in candidates if c and os.path.exists(c)), None)
48 |
49 |     if not client_secret_path:
50 |         sys.exit(f"[!] No client_secret.json found. Put it in the repo root, in "
51 |                f"data/channels/{sanitized}/client_secret.json, or pass --client-secret.")
52 |
53 |     try:
54 |         from google_auth_oauthlib.flow import InstalledAppFlow
55 |     except ImportError:
56 |         sys.exit("[!] Missing Google libs. Install: pip install google-api-python-client google-auth-oauthlib g")
57 |
58 |     print(f"[*] Opening browser to authorize channel '{args.channel}'...")
59 |     print("    Sign in with the Google account that OWNS this YouTube channel,")
60 |     print("    approve the scope, then come back here.\n")
61 |
62 |     flow = InstalledAppFlow.from_client_secrets_file(client_secret_path, SCOPES)
63 |     creds = flow.run_local_server(port=0)
64 |
65 |     os.makedirs(os.path.dirname(token_path), exist_ok=True)
66 |     token_json = creds.to_json()
67 |     with open(token_path, "w", encoding="utf-8") as f:
68 |         f.write(token_json)
69 |
70 |     print(f"\n[+] Token saved to: {token_path}")
71 |     print(f"[+] Copy the ENTIRE JSON below into the GitHub secret "
72 |           f"YOUTUBE_TOKEN_JSON_{args.channel.upper()}\n")
73 |     print(json.dumps(json.loads(token_json), indent=2))
74 |
75 | if __name__ == "__main__":
76 |     main()

```

Explanation

Purpose

Local-only helper to create a YouTube OAuth token .json for a channel by opening a browser login. Not run in CI.

Lines 19-47 — arg & path setup

Takes a <CHANNEL_NAME> argument, computes the channel token path, and locates a client_secret.json (channel-specific, then root).

Lines 49-73 — main()

Runs InstalledAppFlow, writes the token to data/channels/<Name>/token.json, and prints the full JSON so the user can paste it into the GitHub secret YOUTUBE_TOKEN_JSON_<NAME>. The printed token is self-contained (includes client id/secret) so it refreshes in CI without the secret file.

5.16 pipeline.py

5.x Source Code — pipeline.py

```

1 | import os
2 | import sys
3 | import time
4 | import json
5 | import argparse
6 | import logging
7 | from datetime import datetime, timezone, timedelta
8 | import config
9 | from twitch_finder import find_twitch_clips, mark_video_seen as mark_twitch_seen
10 | from twitch_downloader import download_twitch_clip
11 | from video_finder import find_videos, find_playlist_videos, mark_video_seen as mark_youtube_seen
12 | from downloader import download_video
13 | from audio_analyzer import analyze_audio
14 | from highlight_detector import get_highlights, get_full_clip_range
15 | from video_processor import process_video, process_compilation_video, get_video_duration
16 | from metadata_generator import generate_metadata, generate_video_metadata
17 | from youtube_uploader import upload_short, upload_video, get_upload_count_today
18 | from facebook_uploader import upload_fb_video
19 | from notifier import notify_report
20 |
21 |
22 | logging.basicConfig(level=logging.INFO, format="%asctime)s [%levelname)s] %(message)s")
23 |
24 | def cleanup_disk():
25 |     """Cleans up temporary downloaded and processed video files to preserve storage."""
26 |     logging.info("[*] Cleaning up temporary video files...")
27 |     raw_dir = config.get_raw_videos_dir()
28 |     processed_dir = config.get_processed_dir()
29 |     for folder in [raw_dir, processed_dir]:
30 |         if os.path.exists(folder):
31 |             for filename in os.listdir(folder):
32 |                 if filename.startswith('.'):
33 |                     continue
34 |                 file_path = os.path.join(folder, filename)
35 |                 try:
36 |                     if os.path.isfile(file_path):
37 |                         os.unlink(file_path)
38 |                 except Exception as e:
39 |                     logging.warning(f"Failed to delete {file_path}: {e}")
40 |
41 | def run_channel_pipeline(channel, default_count=6, default_gap=2):
42 |     channel_name = channel["channel_name"]
43 |     niche = channel.get("niche", "gaming")
44 |     # Destination platform: "youtube" (default) or "facebook".
45 |     platform = (channel.get("platform") or "youtube").lower()
46 |     channel_fb_cfg = channel.get("facebook")
47 |
48 |     # Pause gate: if "paused_until": "YYYY-MM-DD" is set and today < that date,
49 |     # skip this channel entirely (no sourcing, no uploads). Resumes automatically
50 |     # on the date.
51 |     paused_until = channel.get("paused_until")
52 |     if paused_until:
53 |         try:
54 |             pause_date = datetime.strptime(paused_until, "%Y-%m-%d").date()
55 |             if datetime.now(timezone.utc).date() < pause_date:
56 |                 logging.info(f"[!] Channel '{channel_name}' is paused until {paused_until}. Skipping.")
57 |                 return {
58 |                     "channel_name": channel_name,
59 |                     "shorts_created": 0,
60 |                     "videos_created": 0,
61 |                     "uploads": [],
62 |                     "status": f"Paused until {paused_until}",
63 |                     "error": None,
64 |                 }
65 |         except ValueError:
66 |             logging.info(f"[*] Channel '{channel_name}': pause window ({paused_until}) over – resuming.")
67 |     except ValueError:
68 |         logging.warning(f"[!] Invalid paused_until for '{channel_name}': {paused_until!r} (expected YYYY-MM-DD)")
69 |
70 |     shorts_per_run = channel.get("shorts_per_run", default_count)
71 |     # Facebook-only channels: NO shorts. They publish videos > 3 minutes only.
72 |     if platform == "facebook":
73 |         shorts_per_run = 0
74 |     # Minimum rendered length for a Facebook video (seconds). Enforced in the
75 |     # video phase below – every uploaded FB video must exceed this (3 min + pad).
76 |     fb_min_video_seconds = channel.get("min_video_seconds", 195)
77 |
78 |     # Resolve schedule
79 |     schedule_config = channel.get("upload_schedule", {})
80 |     interval_hours = schedule_config.get("interval_hours", default_gap)
81 |

```

```

82 | # Set the active channel in config to isolate paths
83 | config.set_active_channel(channel_name)
84 |
85 | # 1. Handle credentials setup
86 | creds_config = channel.get("youtube_oauth_credentials")
87 | token_info = None
88 | token_path = None
89 |
90 | if isinstance(creds_config, dict):
91 |     # Inline token content: Write it to the channel's dynamic token file path if not already present
92 |     token_path = config.get_token_file()
93 |     if not os.path.exists(token_path):
94 |         parent_dir = os.path.dirname(os.path.abspath(token_path))
95 |         if parent_dir:
96 |             os.makedirs(parent_dir, exist_ok=True)
97 |             with open(token_path, "w", encoding="utf-8") as f:
98 |                 json.dump(creds_config, f, indent=4)
99 |             token_info = creds_config
100 | elif isinstance(creds_config, str):
101 |     # A filepath to the token
102 |     token_path = creds_config
103 | else:
104 |     # Fallback to default
105 |     token_path = config.get_token_file()
106 |
107 | logging.info(f"\n=====")
108 | logging.info(f"[*] Starting Pipeline for Channel: {channel_name}")
109 | logging.info(f"Niche: {niche} | Target: {shorts_per_run} Shorts | Interval: {interval_hours}h")
110 | logging.info(f"=====")
111 |
112 | # 2. Check upload quota today
113 | uploads_today = get_upload_count_today()
114 | logging.info(f"[*] Total uploads recorded today prior to run for '{channel_name}': {uploads_today}/{config.MAX_UPLOADS_PER_DAY}")
115 | if uploads_today >= config.MAX_UPLOADS_PER_DAY:
116 |     logging.info(f"[!] Daily quota limit reached ({uploads_today}/{config.MAX_UPLOADS_PER_DAY}). Skipping upload")
117 |     return {
118 |         "channel_name": channel_name,
119 |         "shorts_created": 0,
120 |         "videos_created": 0,
121 |         "uploads": [],
122 |         "status": "Skipped (Quota reached)",
123 |         "error": None
124 |     }
125 |
126 | # 3. Sourcing videos
127 | source_config = channel.get("source_configuration", {})
128 | source_type = source_config.get("source_type", "twitch").lower()
129 |
130 | videos = []
131 | if source_type == "twitch":
132 |     target_games = source_config.get("target_games")
133 |     videos = find_twitch_clips(target_games=target_games)
134 | elif source_type == "youtube_search" or source_type == "youtube_api":
135 |     query = source_config.get("query", niche)
136 |     max_subs = source_config.get("max_subs", 100000)
137 |     videos = find_videos(niche_query=query, max_subs=max_subs)
138 | elif source_type == "youtube_playlist":
139 |     playlist_id = source_config.get("playlist_id")
140 |     if playlist_id:
141 |         videos = find_playlist_videos(playlist_id=playlist_id)
142 |     else:
143 |         raise ValueError(f"Playlist ID is missing in source_configuration for channel {channel_name}")
144 | else:
145 |     raise ValueError(f"Unsupported source_type: {source_type}")
146 |
147 | if not videos:
148 |     logging.warning(f"[-] No suitable unseen videos/clips found for channel {channel_name}.")
149 |     return {
150 |         "channel_name": channel_name,
151 |         "shorts_created": 0,
152 |         "videos_created": 0,
153 |         "uploads": [],
154 |         "status": "No candidate videos",
155 |         "error": None
156 |     }
157 |
158 | now_utc = datetime.now(timezone.utc)
159 | base_publish_time = now_utc + timedelta(hours=interval_hours)
160 |
161 | uploaded_count = 0
162 | uploads_info = []
163 | upload_error = None
164 | abort_pipeline = False
165 |
166 | for video in videos:

```

```

167 |         if uploaded_count >= shorts_per_run:
168 |             logging.info(f"[+] Target run goal of {shorts_per_run} scheduled Shorts achieved for {channel_name}")
169 |             break
170 |
171 |         logging.info(f"\n>>> Processing Candidate [{uploaded_count + 1}/{shorts_per_run}]: {video['title']} (ID: {video['id']})")
172 |         time.sleep(2)
173 |
174 |         # 1. Download Video
175 |         video_path = None
176 |         if source_type == "twitch":
177 |             video_path = download_twitch_clip(video)
178 |         else:
179 |             video_path = download_video(video['video_id'])
180 |
181 |         if not video_path or not os.path.exists(video_path):
182 |             logging.error(f"[!] Downloading video failed for ID: {video['video_id']}")
183 |             if source_type == "twitch":
184 |                 mark_twitch_seen(video['video_id'])
185 |             else:
186 |                 mark_youtube_seen(video['video_id'])
187 |             continue
188 |
189 |         # 2. Analyze Audio
190 |         has_voice, is_silent = analyze_audio(video_path)
191 |         mute_audio = False
192 |         if has_voice:
193 |             logging.info("[!] Voice detected. Original audio will be muted.")
194 |             mute_audio = True
195 |         elif is_silent:
196 |             logging.info("[!] Audio is silent. BGM will be added.")
197 |             mute_audio = True
198 |         else:
199 |             logging.info("[+] Clear game audio found. Preserving game audio with BGM.")
200 |
201 |         # 3. Highlight Detection
202 |         highlights = get_highlights(video_path, num_clips=1)
203 |         if not highlights:
204 |             logging.info("[-] No exciting highlights found.")
205 |             if source_type == "twitch":
206 |                 mark_twitch_seen(video['video_id'])
207 |             else:
208 |                 mark_youtube_seen(video['video_id'])
209 |             cleanup_disk()
210 |             continue
211 |
212 |         scene = highlights[0]
213 |
214 |         # 4. Process into YouTube Short (9:16 vertical 1080p)
215 |         out_filename = f"short_{video['video_id']}.mp4"
216 |         processed_path = process_video(
217 |             video_path,
218 |             scene['start'],
219 |             scene['end'],
220 |             out_filename,
221 |             mute_original=mute_audio
222 |         )
223 |
224 |         if not processed_path or not os.path.exists(processed_path):
225 |             logging.error("[!] Video processing failed.")
226 |             if source_type == "twitch":
227 |                 mark_twitch_seen(video['video_id'])
228 |             else:
229 |                 mark_youtube_seen(video['video_id'])
230 |             cleanup_disk()
231 |             continue
232 |
233 |         # 5. Metadata Generation (Pass the dynamic Niche parameter!)
234 |         metadata = generate_metadata(video['title'], niche=niche)
235 |
236 |         # 6. Calculate Scheduled Publish Time
237 |         scheduled_time = base_publish_time + timedelta(hours=uploaded_count * interval_hours)
238 |
239 |         # 7. Upload & Schedule the Short (YouTube OR Facebook, based on platform)
240 |         try:
241 |             if platform == "facebook":
242 |                 # Facebook-only channel: upload to the FB Page, never YouTube.
243 |                 fb_id = upload_fb_video(
244 |                     processed_path, metadata,
245 |                     schedule_time=scheduled_time,
246 |                     channel=channel, is_short=True
247 |                 )
248 |                 uploaded_video_id = fb_id
249 |                 uploaded_count += 1
250 |                 logging.info(f"[+] ({uploaded_count}/{shorts_per_run}) Published short/reel for clip {video['title']}")
251 |                 uploads_info.append({

```

```

252 |         "video_id": uploaded_video_id,
253 |         "title": metadata['title'],
254 |         "publish_time": scheduled_time.strftime('%Y-%m-%dT%H:%M:%S.000Z')
255 |     })
256 |     else:
257 |         uploaded_video_id = upload_short(
258 |             processed_path,
259 |             metadata,
260 |             schedule_time=scheduled_time,
261 |             token_info=token_info,
262 |             token_path=token_path
263 |         )
264 |         uploaded_count += 1
265 |         logging.info(f"[+] ({uploaded_count}/{shorts_per_run}) Successfully uploaded & scheduled clip {video_id}")
266 |         uploads_info.append({
267 |             "video_id": uploaded_video_id,
268 |             "title": metadata['title'],
269 |             "publish_time": scheduled_time.strftime('%Y-%m-%dT%H:%M:%S.000Z')
270 |         })
271 |     if source_type == "twitch":
272 |         mark_twitch_seen(video['video_id'])
273 |     else:
274 |         mark_youtube_seen(video['video_id'])
275 | except Exception as e:
276 |     logging.error(f"[!] Upload failed for video {video['video_id']}: {e}")
277 |     upload_error = str(e)
278 |     cleanup_disk()
279 |
280 |     # Detect credentials, OAuth, or connection-related exceptions
281 |     err_str = str(e).lower()
282 |     is_auth_error = any(kw in err_str for kw in ["invalid_grant", "credentials", "token", "unauthorized"])
283 |     is_network_error = any(kw in err_str for kw in ["transport", "connection", "socket", "timeout"])
284 |
285 |     if is_auth_error or is_network_error:
286 |         logging.error("[!] Authentication or network error detected. Aborting channel pipeline loop.")
287 |         abort_pipeline = True
288 |         break
289 |
290 |     # For other transient or video-specific errors, mark seen and continue
291 |     if source_type == "twitch":
292 |         mark_twitch_seen(video['video_id'])
293 |     else:
294 |         mark_youtube_seen(video['video_id'])
295 |     continue
296 |
297 |     cleanup_disk()
298 |
299 |     time.sleep(5)
300 |
301 |     # =====
302 |     # Long-form Video Phase: N compilation videos per run
303 |     # (same Twitch source; each video = several clips stitched
304 |     # into one 16:9 1080p upload, scheduled like the shorts)
305 |     # =====
306 |     videos_per_run = channel.get("videos_per_run", 0)
307 |     clips_per_video = channel.get("clips_per_video", 5)
308 |     videos_created = 0
309 |
310 |     if videos_per_run > 0 and not abort_pipeline:
311 |         logging.info(f"[*] Video phase: targeting {videos_per_run} long-form video(s) ({clips_per_video} clips per video)")
312 |
313 |         if get_upload_count_today() >= config.MAX_UPLOADS_PER_DAY:
314 |             logging.info("[!] Daily quota limit reached. Skipping video phase.")
315 |         else:
316 |             video_candidates = find_twitch_clips(target_games=target_games)
317 |             cursor = 0
318 |
319 |             # Facebook videos must be > 3 minutes. Estimate how many clips are
320 |             # needed: Twitch clips are <= 60s, so use a generous 45s avg and add
321 |             # headroom. YouTube/other channels keep the configured clips_per_video.
322 |             if platform == "facebook":
323 |                 needed_clips = max(
324 |                     int(round(fb_min_video_seconds / 45.0)) + 1,
325 |                     clips_per_video
326 |                 )
327 |             else:
328 |                 needed_clips = clips_per_video
329 |
330 |             for vid_idx in range(videos_per_run):
331 |                 if get_upload_count_today() >= config.MAX_UPLOADS_PER_DAY:
332 |                     logging.info("[!] Daily quota limit reached during video phase. Stopping video creation.")
333 |                     break
334 |
335 |                 segments = []
336 |                 used_clips = []

```

```

337 |         while len(segments) < needed_clips and cursor < len(video_candidates):
338 |             clip = video_candidates[cursor]
339 |             cursor += 1
340 |             clip_id = clip["video_id"]
341 |
342 |             # Skip unreasonably short clips before wasting a download
343 |             if clip.get("duration", 30) < 8:
344 |                 mark_twitch_seen(clip_id)
345 |                 continue
346 |
347 |             clip_path = download_twitch_clip(clip)
348 |             if not clip_path or not os.path.exists(clip_path):
349 |                 mark_twitch_seen(clip_id)
350 |                 continue
351 |
352 |             seg_range = get_full_clip_range(clip_path)
353 |             if seg_range["duration"] < 8:
354 |                 mark_twitch_seen(clip_id)
355 |                 continue
356 |
357 |             segments.append((clip_path, seg_range["start"], seg_range["end"]))
358 |             used_clips.append(clip)
359 |
360 |         if len(segments) < 2:
361 |             logging.info("[-] Not enough clips available for another compilation video. Stopping video")
362 |             break
363 |
364 |         logging.info(f"\n>>> Building Long-form Video [{vidx + 1}/{videos_per_run}] from {len(segments)} clips")
365 |         out_filename = f"video_{datetime.now().strftime('%H%M%S')}_{vidx + 1}.mp4"
366 |         processed_path = process_compilation_video(segments, out_filename)
367 |
368 |         if not processed_path or not os.path.exists(processed_path):
369 |             logging.error("[!] Compilation video processing failed.")
370 |             for c in used_clips:
371 |                 mark_twitch_seen(c["video_id"])
372 |             cleanup_disk()
373 |             continue
374 |
375 |         # HARD GUARANTEE: every Facebook video must be > 3 minutes.
376 |         if platform == "facebook":
377 |             actual_dur = get_video_duration(processed_path)
378 |             if actual_dur < fb_min_video_seconds:
379 |                 logging.error(
380 |                     f"[!] Facebook video too short ({actual_dur:.1f}s < {fb_min_video_seconds}s). "
381 |                     f"Discarding this attempt and retrying next run to protect the >3min requirement."
382 |                 )
383 |                 for c in used_clips:
384 |                     mark_twitch_seen(c["video_id"])
385 |                 cleanup_disk()
386 |                 continue
387 |
388 |         metadata = generate_video_metadata(used_clips[0]["title"], niche=niche)
389 |         scheduled_time = base_publish_time + timedelta(hours=(uploaded_count + videos_created) * interval)
390 |
391 |         try:
392 |             if platform == "facebook":
393 |                 # Facebook-only channel: upload compilation to FB Page.
394 |                 fb_id = upload_fb_video(
395 |                     processed_path, metadata,
396 |                     schedule_time=scheduled_time,
397 |                     channel=channel, is_short=False
398 |                 )
399 |                 uploaded_video_id = fb_id
400 |                 videos_created += 1
401 |                 logging.info(f"[+] ({vidx + 1}/{videos_per_run}) Scheduled compilation video to Facebook")
402 |                 uploads_info.append({
403 |                     "video_id": uploaded_video_id,
404 |                     "title": metadata['title'],
405 |                     "publish_time": scheduled_time.strftime('%Y-%m-%dT%H:%M:%S.000Z'),
406 |                     "type": "video"
407 |                 })
408 |             else:
409 |                 uploaded_video_id = upload_video(
410 |                     processed_path,
411 |                     metadata,
412 |                     schedule_time=scheduled_time,
413 |                     token_info=token_info,
414 |                     token_path=token_path
415 |                 )
416 |                 videos_created += 1
417 |                 logging.info(f"[+] ({vidx + 1}/{videos_per_run}) Successfully uploaded & scheduled compilation video")
418 |                 uploads_info.append({
419 |                     "video_id": uploaded_video_id,
420 |                     "title": metadata['title'],
421 |                     "publish_time": scheduled_time.strftime('%Y-%m-%dT%H:%M:%S.000Z'),

```

```

422 |         "type": "video"
423 |     })
424 |
425 |     for c in used_clips:
426 |         mark_twitch_seen(c["video_id"])
427 | except Exception as e:
428 |     logging.error(f"[!] Video upload failed for compilation {out_filename}: {e}")
429 |     if not upload_error:
430 |         upload_error = str(e)
431 |     for c in used_clips:
432 |         mark_twitch_seen(c["video_id"])
433 |
434 |     # Detect credentials, OAuth, or connection-related exceptions
435 |     err_str = str(e).lower()
436 |     is_auth_error = any(kw in err_str for kw in ["invalid_grant", "credentials", "token", "unau
437 |     is_network_error = any(kw in err_str for kw in ["transport", "connection", "socket", "timeo
438 |
439 |     if is_auth_error or is_network_error:
440 |         logging.error("[!] Authentication or network error detected during video phase. Aborting
441 |         break
442 |
443 |     cleanup_disk()
444 |     time.sleep(5)
445 |
446 |     status = "Success"
447 |     if upload_error:
448 |         if uploaded_count > 0:
449 |             status = "Partial Success"
450 |         else:
451 |             status = "Failed"
452 |
453 |     return {
454 |         "channel_name": channel_name,
455 |         "shorts_created": uploaded_count,
456 |         "videos_created": videos_created,
457 |         "uploads": uploads_info,
458 |         "status": status,
459 |         "error": upload_error
460 |     }
461 |
462 | def print_report(report):
463 |     print("\n" + "=" * 60)
464 |     print("                                AUTOMATION BATCH RUN REPORT")
465 |     print("=" * 60)
466 |     for r in report:
467 |         print(f"Channel Name:    {r['channel_name']}")
468 |         print(f"Status:           {r['status']}")
469 |         print(f"Shorts Created: {r['shorts_created']}")
470 |         print(f"Videos Created: {r.get('videos_created', 0)}")
471 |         if r['uploads']:
472 |             print("Scheduled Uploads:")
473 |             for u in r['uploads']:
474 |                 print(f"    - [{u['video_id']}] ({u.get('type', 'short')}) {u['title']} (Publish: {u['publish_tim
475 |             if r['error']:
476 |                 print(f"Error:           {r['error']}")
477 |             print("-" * 60)
478 |         print("=" * 60 + "\n")
479 |
480 | def save_report(report):
481 |     report_dir = os.path.join(config.DATA_DIR, "reports")
482 |     os.makedirs(report_dir, exist_ok=True)
483 |     report_file = os.path.join(report_dir, f"report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json")
484 |     latest_report_file = os.path.join(config.DATA_DIR, "latest_run_report.json")
485 |
486 |     report_data = {
487 |         "timestamp": datetime.now(timezone.utc).isoformat(),
488 |         "summary": report
489 |     }
490 |
491 |     for fpath in [report_file, latest_report_file]:
492 |         with open(fpath, "w", encoding="utf-8") as f:
493 |             json.dump(report_data, f, indent=4)
494 |
495 |     logging.info(f"[+] Run report saved to {latest_report_file}")
496 |
497 | def run_multi_channel(channels_file="channels.json", default_count=6, default_gap=2, channel_filter=None):
498 |     if not os.path.exists(channels_file):
499 |         logging.error(f"Channels configuration file '{channels_file}' does not exist.")
500 |         sys.exit(1)
501 |
502 |     with open(channels_file, "r", encoding="utf-8") as f:
503 |         try:
504 |             channels = json.load(f)
505 |         except Exception as e:
506 |             logging.error(f"Failed to parse channels configuration file: {e}")

```

```

507 |         sys.exit(1)
508 |
509 |     if not isinstance(channels, list):
510 |         logging.error("Channels configuration must be a JSON array of channel objects.")
511 |         sys.exit(1)
512 |
513 |     # Optional single-channel filter (used by parallel matrix jobs)
514 |     if channel_filter:
515 |         channels = [c for c in channels if c.get("channel_name") == channel_filter]
516 |         if not channels:
517 |             logging.error(f"Channel '{channel_filter}' not found in configuration.")
518 |             sys.exit(1)
519 |
520 |     logging.info(f"[*] Loaded {len(channels)} channel configurations from '{channels_file}'.")
521 |
522 |     report = []
523 |
524 |     for channel in channels:
525 |         channel_name = channel.get("channel_name", "Unknown Channel")
526 |         try:
527 |             res = run_channel_pipeline(channel, default_count=default_count, default_gap=default_gap)
528 |             report.append(res)
529 |         except Exception as e:
530 |             logging.error(f"[!] Pipeline failed for channel '{channel_name}': {e}", exc_info=True)
531 |             report.append({
532 |                 "channel_name": channel_name,
533 |                 "shorts_created": 0,
534 |                 "uploads": [],
535 |                 "status": "Failed",
536 |                 "error": str(e)
537 |             })
538 |
539 |     print_report(report)
540 |     save_report(report)
541 |
542 |     # Send execution summary notification
543 |     try:
544 |         notify_report(report)
545 |     except Exception as e:
546 |         logging.error(f"[!] Failed to execute notification flow: {e}")
547 |
548 |
549 | if __name__ == "__main__":
550 |     parser = argparse.ArgumentParser(description="Multi-Channel YouTube Shorts Scheduled Automation Pipeline")
551 |     parser.add_argument("--channels", type=str, default="channels.json", help="Path to channels.json configurat")
552 |     parser.add_argument("--count", type=int, default=6, help="Default number of Shorts to generate and schedule")
553 |     parser.add_argument("--gap", type=int, default=2, help="Default minimum gap in hours between scheduled video")
554 |     parser.add_argument("--loop", action="store_true", help="Run continuously in 24/7 loop mode instead of sing")
555 |     parser.add_argument("--channel", type=str, default=None, help="Run only this single channel (used by parall")
556 |     args = parser.parse_args()
557 |
558 |     print("[Bot] Multi-Channel YouTube Shorts Scheduled Automation Bot Initiated [Bot]")
559 |
560 |     if args.loop:
561 |         logging.info(f"[*] Running in continuous loop mode...")
562 |         while True:
563 |             try:
564 |                 run_multi_channel(channels_file=args.channels, default_count=args.count, default_gap=args.gap,
565 |                                     logging.info(f"[*] Sleeping 24 hours until next daily batch cycle...")
566 |                                     time.sleep(86400)
567 |             except Exception as e:
568 |                 logging.error(f"[!] Pipeline crashed: {e}")
569 |                 time.sleep(300)
570 |         else:
571 |             logging.info(f"[*] Running multi-channel scheduled batch mode...")
572 |             run_multi_channel(channels_file=args.channels, default_count=args.count, default_gap=args.gap, channel_
573 |             sys.exit(0)
574 |

```

Explanation

Lines 1-22 — imports & logging

Imports every specialist module. Logging at INFO with timestamps.

Lines 24-39 — cleanup_disk()

Deletes temp files in raw_videos and processed (skips dotfiles) to keep the ephemeral runner's disk from filling up.

Lines 41-68 — run_channel_pipeline() header + pause gate

Sets the active channel, reads platform, and implements a `paused_until` date gate: if a channel is paused (date in the future) it is skipped entirely and reports 'Paused'. This lets you freeze a channel without editing code.

Lines 70-76 — Shorts count & FB minimum

Reads `shorts_per_run`. **Facebook override:** if platform is facebook, `shorts_per_run` = 0 (no Shorts). Reads `min_video_seconds` (default 195) — the >3-minute threshold for FB videos.

Lines 78-124 — scheduling, credentials, quota gate

Resolves `interval_hours`, sets the active channel path, resolves the YouTube token (inline dict/file/path), checks the daily upload quota and bails if reached.

Lines 126-156 — source dispatch

Picks the source: Twitch (default), YouTube search, or YouTube playlist. If no candidates are found it reports 'No candidate videos' and exits the channel run.

Lines 158-299 — SHORTS LOOP

- Computes `base_publish_time` = `now` + `interval_hours`.
- For each candidate up to `shorts_per_run`: download → analyze audio → get highlight → process_video → generate metadata → upload (YouTube or Facebook) with a staggered schedule.
- On download/process failure the clip is marked seen and skipped.
- On auth/network upload error it ABORTS the channel loop (these are fatal); other errors mark seen and continue.
- **For the Facebook channel this loop is skipped entirely** because `shorts_per_run=0`.

Lines 301-445 — LONG-FORM VIDEO LOOP (incl. the >3-min guarantee)

- Reads `videos_per_run` and `clips_per_video`.
- For Facebook it computes `needed_clips` = `max(round(195/45)+1, clips_per_video)` (~6) so enough Twitch clips are gathered to exceed 3 minutes (lines 319-328).
- Gathers segments, skipping clips <8s; builds the compilation via `process_compilation_video`.
- **HARD GUARANTEE (lines 375-386):** for Facebook it probes the rendered file with `get_video_duration`; if shorter than `min_video_seconds` it discards the attempt, marks clips seen, and continues to the next iteration — protecting the 'every video >3 min' rule.
- Then generates long-form metadata and uploads (YouTube or Facebook) with a staggered schedule, marking clips seen on success.

Lines 446-460 — status

Derives Success / Partial Success / Failed from counts and any error.

Lines 462-495 — reporting

`print_report()` prints a console summary; `save_report()` writes a JSON run report under `data/reports` and `latest_run_report.json`.

Lines 497-573 — run_multi_channel() & CLI

Loads `channels.json`, optionally filters to one `--channel` (used by the matrix), runs each channel (catching per-channel exceptions), prints/saves the report, and sends the notification. The `__main__` block parses CLI args and supports `--loop` (24/7) or single batch mode. The GitHub Actions job calls it with `--channel "<name>"`.

5.17 .github/workflows/upload_short.yml

5.x Source Code — .github/workflows/upload_short.yml

```

1 | name: Scheduled Twitch to YouTube Shorts Automation
2 |
3 | on:
4 |   schedule:
5 |     # Trigger ONCE daily at 00:30 UTC (06:00 AM IST)
6 |     - cron: '30 0 * * *'
7 |   workflow_dispatch:
8 |     inputs:
9 |       channel:
10 |         description: 'Run only this channel (leave empty to run ALL channels in parallel)'
11 |         required: false
12 |         default: ''
13 |         type: string
14 |
15 | permissions:
16 |   contents: write
17 |
18 | jobs:
19 |   run-automation:
20 |     # Run all channels in parallel; each gets its own 6h job limit so none is starved.
21 |     # If a single channel is requested via manual dispatch, run only that one.
22 |     strategy:
23 |       fail-fast: false
24 |       max-parallel: 4
25 |       matrix:
26 |         channel: ${ fromJSON(inputs.channel && format('["{}"]', inputs.channel)) || '["ExampleGamingChannel"] }
27 |     name: ${ matrix.channel }
28 |     runs-on: ubuntu-latest
29 |     timeout-minutes: 350
30 |
31 |     steps:
32 |       - name: Checkout Repository
33 |         uses: actions/checkout@v4
34 |
35 |       - name: Set up Python 3.11
36 |         uses: actions/setup-python@v5
37 |         with:
38 |           python-version: '3.11'
39 |
40 |       - name: Install System FFmpeg
41 |         run: |
42 |           sudo apt-get update
43 |           sudo apt-get install -y ffmpeg
44 |
45 |       - name: Install Dependencies
46 |         run: |
47 |           python -m pip install --upgrade pip
48 |           pip install -r requirements.txt
49 |
50 |       - name: Execute Shorts + Video Generation for ${ matrix.channel }
51 |         env:
52 |           TWITCH_CLIENT_ID: ${ secrets.TWITCH_CLIENT_ID }
53 |           TWITCH_CLIENT_SECRET: ${ secrets.TWITCH_CLIENT_SECRET }
54 |           YOUTUBE_API_KEY: ${ secrets.YOUTUBE_API_KEY }
55 |           GROQ_API_KEY: ${ secrets.GROQ_API_KEY }
56 |           YOUTUBE_CLIENT_SECRET_JSON: ${ secrets.YOUTUBE_CLIENT_SECRET_JSON }
57 |           # Channel dynamic tokens passed as environment variables:
58 |           YOUTUBE_TOKEN_JSON_EXISTINGCHANNEL: ${ secrets.YOUTUBE_TOKEN_JSON_EXISTINGCHANNEL }
59 |           YOUTUBE_TOKEN_JSON_EXAMPLEGAMINGCHANNEL: ${ secrets.YOUTUBE_TOKEN_JSON_EXAMPLEGAMINGCHANNEL }
60 |           YOUTUBE_TOKEN_JSON_GAMES: ${ secrets.YOUTUBE_TOKEN_JSON_GAMES }
61 |           YOUTUBE_TOKEN_JSON_MINECRAFTSHORTS: ${ secrets.YOUTUBE_TOKEN_JSON_MINECRAFTSHORTS }
62 |           # Facebook Page (Graph API) credentials for the FacebookGaming channel:
63 |           FACEBOOK_PAGE_ID: ${ secrets.FACEBOOK_PAGE_ID }
64 |           FACEBOOK_PAGE_ACCESS_TOKEN: ${ secrets.FACEBOOK_PAGE_ACCESS_TOKEN }
65 |           # Notification Configuration Secrets
66 |           NOTIFICATION_PROVIDER: ${ secrets.NOTIFICATION_PROVIDER }
67 |           TWILIO_ACCOUNT_SID: ${ secrets.TWILIO_ACCOUNT_SID }
68 |           TWILIO_AUTH_TOKEN: ${ secrets.TWILIO_AUTH_TOKEN }
69 |           TWILIO_FROM_NUMBER: ${ secrets.TWILIO_FROM_NUMBER }
70 |           USER_PHONE_NUMBER: ${ secrets.USER_PHONE_NUMBER }
71 |           TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
72 |           TELEGRAM_CHAT_ID: ${ secrets.TELEGRAM_CHAT_ID }
73 |         run: |
74 |           python pipeline.py --channels channels.json --channel "${ matrix.channel }"
75 |
76 |       - name: Commit & Push Updated History (best-effort, retry on conflict)
77 |         if: always()
78 |         run: |
79 |           git config user.email "actions@github.com"
80 |           git config user.name "github-actions[bot]"
81 |           for i in 1 2 3 4 5; do

```

```

82 |         find data/channels -type f \( -name "seen_videos.txt" -o -name "upload_log.json" \) -exec git add {} \;
83 |         if git diff --cached --quiet; then
84 |             echo "No history changes for ${matrix.channel}"
85 |             break
86 |         fi
87 |         git commit -m "auto: update ${matrix.channel} history [skip ci]" || break
88 |         git fetch origin main
89 |         git rebase -X ours origin/main || git rebase --abort || true
90 |         if git push origin main; then
91 |             echo "Pushed ${matrix.channel} history"
92 |             break
93 |         fi
94 |         echo "Push conflict, retrying ($i)..."
95 |         sleep 5
96 |     done
97 |

```

Explanation

Lines 1-13 — triggers

Scheduled cron 30 0 * * * (00:30 UTC daily) plus workflow_dispatch with an optional channel input (lets you run one channel manually).

Lines 15-29 — matrix job

fail-fast:false + max-parallel:4 so all channels run concurrently but no single one is starved; 350-minute timeout. The matrix is either a single channel (manual) or the full 5-channel list.

Lines 31-74 — steps & env

Checkout → Python 3.11 → install system ffmpeg → pip install -r requirements.txt. The big env: block injects every secret (Twitch, YouTube, Groq, the four per-channel YouTube tokens, Facebook Page creds, and notification secrets) into the job. Then runs python pipeline.py --channels channels.json --channel "<matrix.channel>".

Lines 76-96 — commit history back

A best-effort step (always runs) commits the updated seen_videos.txt and upload_log.json per channel, rebasing on origin/main with -X ours and retrying up to 5 times on push conflicts. **Why:** this persists dedup state across ephemeral runners so the same clips aren't re-uploaded next run.

5.18 README.md (summary)

5.x Source Code — README.md

```

1 | # Twitch to YouTube Shorts Automation Bot ■■
2 |
3 | An automated production pipeline that fetches trending gaming clips from the **Twitch Helix API**, processes the
4 |
5 | ---
6 |
7 | ## ■ Key Features
8 |
9 | - **Twitch Helix API Source**: Fetches trending clips across top game categories (GTA V, Fortnite, Minecraft, A
10 | - **Direct CDN Fast Downloader**: Downloads Twitch clip MP4 files directly from Twitch's CDN infrastructure with
11 | - **AI Audio & Highlight Analysis**: Detects human voice vs. game audio vs. silence and applies background musi
12 | - **Shorts Video Processor**: Crops, scales, sharpens, and speeds up 16:9 gameplay into pristine 9:16 vertical
13 | - **Groq Llama 3.3 Metadata**: Generates viral titles, SEO descriptions, and tags.
14 | - **YouTube OAuth2 Uploader**: Automatically uploads Shorts to YouTube with `#shorts` tags.
15 | - **Long-form Compilation Videos**: Also builds regular 16:9 1080p videos (several clips stitched with BGM) and
16 | - **GitHub Actions 24/7 Scheduler**: Automatically runs 6 times daily (`0 6,9,12,15,18,21 * * *` UTC).
17 |
18 | ---
19 |
20 | ## ■■ Prerequisites
21 |
22 | - Python 3.11+
23 | - FFmpeg installed
24 | - Twitch Developer Account (for Client ID & Client Secret)
25 | - YouTube Cloud Console Credentials (for OAuth2 token)
26 |
27 | ---
28 |
29 | ## ■ Environment Variables Setup
30 |
31 | Create a `.env` file in the root directory:
32 |
33 | ```env
34 | # Twitch Helix API Credentials (dev.twitch.tv)
35 | TWITCH_CLIENT_ID="your_twitch_client_id"
36 | TWITCH_CLIENT_SECRET="your_twitch_client_secret"
37 |
38 | # Optional YouTube Data API & Groq API
39 | YOUTUBE_API_KEY="your_youtube_api_key"
40 | GROQ_API_KEY="your_groq_api_key"
41 |
42 | # YouTube OAuth Secrets
43 | YOUTUBE_TOKEN_JSON='{"token": "...", "refresh_token": "..."}'
44 | YOUTUBE_CLIENT_SECRET_JSON='{"installed": {"client_id": "..."}}'
45 | ```
46 |
47 | ---
48 |
49 | ## ■ Local Usage
50 |
51 | 1. **Install dependencies**:
52 | ```bash
53 | pip install -r requirements.txt
54 | ```
55 |
56 | 2. **Run single trigger (1 Short upload)**:
57 | ```bash
58 | python pipeline.py
59 | ```
60 |
61 | 3. **Run in 24/7 loop mode**:
62 | ```bash
63 | python pipeline.py --loop
64 | ```
65 |
66 | ```json
67 | {
68 |   "channel_name": "MyChannel",
69 |   "niche": "Minecraft gaming highlights and funny moments",
70 |   "upload_schedule": { "interval_hours": 3 },
71 |   "shorts_per_run": 6,
72 |   "videos_per_run": 3,
73 |   "clips_per_video": 5,
74 |   "youtube_oauth_credentials": "data/channels/MyChannel/token.json",
75 |   "source_configuration": {
76 |     "source_type": "twitch",
77 |     "target_games": ["Minecraft"]
78 |   }
79 | }
80 | ```
81 | - `shorts_per_run` → how many Shorts to create per batch (default 6).

```

```

82 | - `videos_per_run` → how many long-form compilation videos to create per batch (default 0 = disabled; each vid
83 |
84 | ---
85 |
86 | ## ■ GitHub Actions Workflow Secrets
87 |
88 | Add the following Repository Secrets under **Settings** $↗ **Secrets and variables** $↗ *
89 |
90 | - `TWITCH_CLIENT_ID`
91 | - `TWITCH_CLIENT_SECRET`
92 | - `YOUTUBE_API_KEY`
93 | - `GROQ_API_KEY`
94 | - `YOUTUBE_TOKEN_JSON`
95 | - `YOUTUBE_CLIENT_SECRET_JSON`
96 |
97 | ---
98 |
99 | ## ■ Repository Architecture
100 |
101 | ```
102 | ███ config.py                # Central config & target game categories
103 | ███ twitch_finder.py         # Queries Twitch Helix API for trending game clips
104 | ███ twitch_downloader.py     # Direct Twitch CDN & yt-dlp clip downloader
105 | ███ audio_analyzer.py        # Voice detection & audio analysis
106 | ███ highlight_detector.py     # Detects exciting gameplay highlights
107 | ███ video_processor.py       # Crops 16:9 to 9:16 vertical Short (< 45s)
108 | ███ metadata_generator.py    # AI title, description & tag generation via Groq
109 | ███ youtube_uploader.py      # OAuth2 YouTube Shorts uploader
110 | ███ pipeline.py              # Main trigger controller & workflow executor
111 | ███ .github/workflows/       # Scheduled GitHub Actions cron pipeline
112 | ```
113 |

```

Explanation

What it is

A user-facing quick-start. It describes the Twitch→YouTube Shorts pipeline, key features, prerequisites, the `.env` setup, local usage, and the GitHub Actions secrets to add. It is a subset of this document's detail; the code is the source of truth. (Note: the README's cron example `0 6,9,...` is stale — the actual workflow uses `30 0 * * *` once daily, as documented in 5.17.)

5.19 data/ runtime artefacts

5.x Source Code — data/channels/GAMES/seen_videos.txt

```

1 | VastSmilingBottleBloodTrail-dsdxK5gWqNHC2WNo
2 | RelievedCrepuscularGerbilBrainSlug-GYx5Y2pZenJRV-rT
3 | PreciousBigWerewolf0Ssloth
4 | EmpathicProductivePizzaTooSpicy
5 | ObservantSucculentMangoDuDudu-AGJ0QmG4P4NaS_M9
6 | HyperCrowdedOxBudBlast-_M7MXAPboLoY9fg7
7 | AmericanBlushingDiscGingerPower-8fTDUmZ-MDdxH_dJ
8 | KindPowerfullardBCWarrior
9 | SassyLaconicTapirNotATK
10 | PatientAcceptableGuanacoHumbleLife-dDDh3PcIO5_5DnYN
11 | HorribleCorrectSowAsianGlow-dT4WSho0tAtKed6l
12 | LaconicSillySpaghettiDoggo-MyVlu4GLjonrefRI
13 | RelievedBashfulSoymilkAMPEnergyCherry
14 | BlueSnappyGerbilDoubleRainbow-h8INCGZDuTqfhdr4
15 | CharmingTriumphantSmoothieJonCarnage
16 | RudeBrainyNuggetsMcAT-m0eiOME0Q6QJs3Q0
17 | AdventurousImportantPresidentPipeHype-g93v5zbjs2PIKovv
18 | TacitAmusedBananaLitFam-skXxb8tsI6vQLRxB
19 | BlazingObliviousElkM4xHeh
20 | TriumphantFaithfulPrariedogHassaanChop
21 | SassyRelentlessTaroSquadGoals-GNsiuMgy1rU8E67P
22 | AstutePowerfulAirGuitarAsianGlow-yPB_g3LUQ9zgEL_2
23 | WimpyLittleBaguettePogChamp-iccitCdgt2NWH5k
24 | CleverTsundereDragonflySuperVinlin
25 | KitschyHandsomeBulgogiSquadGoals-e-wkl6P93IbZC_x9
26 | SweetBlushingGuanacoUnSane-qh7mY0hcqWtG0AB
27 | RespectfulBloodyPangolinPeteZaroll-9EkttYP6PtBC3SLn
28 | RacyLovelyWerewolfAMPEnergyCherry-s0AMRKKEhvhvFF7w2
29 | BashfulSmokyMoonMau5-8I2M8BvuGv0g-kuR
30 | OutstandingOddMilkDerp-F_gHLGJCuq6Fgasa
31 | PlayfulPunchyApeBudStar-7YeJAmok1HmR2L0E
32 | AmazingGiantFlyShadyLulu-DPef-sNnmk_EaAL
33 | OutstandingPunchyChowderSMOrc-qU-Dg_112k_8_5w0
34 | AntsyPlainSmoothieCoolCat-__WcrJx8L03egr00
35 | TallSpunkyBorkVoHiYo
36 | IntelligentSpeedyAnteaterNerfBlueBlaster-Gi0ShFXorfYUeI6
37 | SuspiciousCaringSandwichAMPTropPunch-YpU2knjWkVd-Z7Hw
38 | CarefulSmellyConsoleLeeroyJenkins-ayw7ThjCQeMk2BRx
39 | ApatheticTrustworthyAxeBloodTrail
40 | VivaciousNurturingSnakeDxBomb-Fwh7gJwQyOpIcLg1
41 | GoldenCoyArtichokeNerfBlueBlaster
42 | SuaveAmazonianDragonflyTwitchRPG-g5sY8s6H-db16TTE
43 | IntelligentMuddyHerringResidentSleeper-40C7-X-FUPhAWE36
44 | SparklyStylishZucchiniBatChest
45 | DelightfulGrossPigeonMrDestructoid-gQqlvUozqvrtU_rX
46 | BlightedGlamorousKuduWTRuck-57Q3YIsK16BFR00H
47 | FuriousSuccessfulRatPrimeMe-2A0M6jaXjdYYkDLB
48 | DrabRelatedHerringWTRuck
49 | EndearingSincereReubenJKanStyle-90nTy8A9s-GGMdb
50 | EvilPreciousTarsierWoofier-mWGy6Gh8Rc52PMSY
51 | FilthyFantasticRatSSSSss-NoFNa0sXdILA-UAi
52 | LachrymoseSquareDugongPrimeMe-ZAEE140sJkXPROI1
53 | FrozenVainHerdANELE-4nuYw98Bb0GGiQxk
54 | PliableTenderPidgeonShazBotstix-rh_6fLH7FCWAKEu4
55 | OptimisticUgliestGarlicMoreCowbell-nISfqK2EbWnbFwOK
56 | KawaiiEndearingAnacondaCopyThis
57 | CourageousFunnyFriesKappaWealth
58 | ClumsyGoldenNoodleTheTarFu-mQ70IbuahtJIXLLi
59 | SweetSpeedyKuduRalpherZ-ptFREXXZquuelQYr
60 | BoldSteamyAdminOneHand-FB1f4ZW0y1iJuMw
61 | RacySucculentPheasantBleedPurple-pCg8AxfgdIJZ0Snt
62 | LittleImpartialFriesBCWarrior-unyH2jXAN2fYwc3M
63 | FlaccidWimpyMosquito4Head-qu3J-MfiETpFs0x
64 | PolishedPoorSkunkCharlieBitMe-QFAwhV3TRAmVCPTL
65 | FriendlyFitPassionfruitArsonNoSexy-Y70k6xP-06JmHdNn
66 | SlickBoredPigeonLeeroyJenkins--wrXIhGT4cNA2QDc
67 | SoftOptimisticRaccoonPastaThat--vtUa66zinGn1fH6
68 | BraveWealthyEggplantTF2John-9C5X9KrZPws872XT
69 | MushyFunnyBarracudaSMOrc-wFTegatdQhQ0FbQ7
70 | PlainRefinedClint4Head-q6HTKcPyDqMHs8df
71 | CautiousCourageousMeerkatFeelsBadMan
72 | ClumsyTardyHedgehogStrawBeary-XwdUdQJQ0wpVtLRk
73 | HilariousDeafKaleTheRinger-IfLJi8ynfGsf6UXS
74 | SecretiveImportantCobblerTF2John-NjqA225cZhBM-7c-
75 | PiercingBrightCheeseStrawBeary
76 | CoyCourageousMagpieKappaWealth-aKSBUuemPINitqfE
77 | BoxySucculentWalrusDoritosChip-zuasDXn8swUpZoP6
78 | SwissSolidZebraDatSheffy-3rzUH_zucx0hCvRL
79 | RacyEnthusiasticHerringPipeHype-Cfk8z0a609eUjRB1
80 | AssiduousHyperCrocodileMrDestructoid-EIkeJlfxQhYGBvXU
81 | SparklingRockyMeatloafHumbleLife

```

82 | HorribleSourCrocodileAsianGlow-KBa9sS7Z-gzNjG5_
83 | FairRoundCrowTheThing-feEbDTfSexRPFipk
84 | ResoluteObservantMoonTBTacoLeft
85 | SoftFitMageFloof-W6aHXTvENa5YZrRU
86 | RoundSmellyRamenHassanChop-_zzCbKFIIdi83XD8Q
87 | FlaccidBeautifulSoybeanVoteYea
88 | TacitCrypticJackalSwiftRage-XoT092UYVf4vekDb
89 | AcceptablePlacidAmazonBCWarrior-mMZVy1zLm2aCrX2w
90 | ShinyBlindingAlmondKeyboardCat-1lVP8PsQ5RzbwJKR
91 | ApatheticDifferentNostrilAsianGlow-opD6aS6T-YojLbu_
92 | ViscousAcceptableLeopardDogFace-3E4vgT0yn54L6kvU
93 | ExcitedRelatedPhoneYee-Mc64DG4s36GKDKkT
94 | ZanyCrazyAniseCclamChamp-L5RcKFbew_-13qJz
95 | ProtectiveJollyPuddingCharlieBitMe-5yXuLR4gixUf2xha
96 | KathishFaintFriesGingerPower-IWV6AJSEbYlFzRA8
97 | SilkyDistinctDragonfruitGrammarKing-3t6V-01FR3Dx5Xw-
98 | TenderBreakableZebraKappaRoss-KR506SfmRJkIfZgE
99 | ElatedStrangeFalconTheThing
100 | RespectfulRichLobsterGivePLZ-XmcsWFB-DLHFwiU
101 | AdorableALiveApeRuleFive-DmEUBjjWtnHw8e5o
102 | HandsomeVibrantCobblerWow
103 | DarlingDepressedHornetPunchTrees-_Q843-0ip1eQb6Y3
104 | ExquisiteDepressedTurnipTriHard-2tc4CC7oFFDxbwTD
105 | TriangularEncouragingYogurtKAPOW-hvph934TFRC-U4k9
106 | BraveTacitCheesecakeBleedPurple-v4f-kcwJ9rjk8-nC
107 | SpotlessVainLionDansGame-ghcZQf7_T46tzEjI
108 | BlindingWonderfulAppleBCWarrior
109 | MoralSpotlessWormGivePLZ-lgCs9Lt8BnfsAP2k
110 | GrotesqueMiniatureLobsterTheRinger
111 | MildTiredUdonANELE-CCAXigydvTvm5yAR
112 | EnticingBusyCarabeefGivePLZ
113 | BreakableRepleteStapleSM0rc-26v64FR3GLV4pUW1
114 | LazyLaconicNostrilVoHiYo-9jX50XR8xuxP8-Yh
115 | HorribleBillowingRatThisIsSparta-uVCd_U651JtLLSD_
116 | AttractiveTastyMouseLitFam-zQIqktFBB85JVjUO
117 | InnocentInexpensiveTarsierDendiFace
118 | MagnificentBadTigerOneHand
119 | ZealousProtectiveCockroachNinjaGrumpy
120 | ResourcefulSuccessfulMartenPupper-UEZlmZu9in1YZVNE
121 | IgnorantTameMagpieFreakinStinkin
122 | AffluentNeighborlyDonkeyNerfBlueBlaster-V4l2pAIlp0z6aKrT
123 | IgnorantUgliestMonitorNinjaGrumpy-Gx08ATcentCxbG70
124 | EagerEagerBeefThunBeast-QzVSLqR1hL08kdvw
125 | TalentedTransparentKleeDancingBaby-wJkh7TvoVMqhGzq3
126 | AmazonianAnnoyingPeafowlFloof-8JSQHue33hAo_bk7
127 | InexpensiveCaImSrirachaPrimeMe
128 | RamshackleEnthusiasticStorkHeyGirl-bZUI1_Xe7u-Bg9uY
129 | MistyBeautifulVultureDerp-Y34KAGbhLUSOs66n
130 | SuccessfulAdventurousButterfly4Head-MBfx2QuX9XH7cGfY
131 | AntediluvianBusyPeanutRedCoat-I0bPSdeJExbacGS1
132 | ExpensiveAgitatedButterflyOMGScoots-04L4_A2No8kkkVrF
133 | ScaryGiftedCattleDoggo-oRHj9m0F9HhuCRwc
134 | AntediluvianShortCrabMVGame-UshWpqPvOndK0cXX
135 | RelatedAdorableBibimbapArsonNoSexy
136 | TrappedPlacidOctopusOSSloth-r9SaaaYlJwgtihWO
137 | AnimatedOptimisticWasabiVoteNay
138 | ObliviousEnjoyablePheasantFunRun-sL0uKfjEZiP682iq
139 | BitterDeafNarwhalMikeHogu-usvmRU4QMYMSDgR6
140 | TemperedViscousLouseYee-QHrXkgJyG0qqyLF6
141 | ModernVainRedpandaNotATK-0fSY4V8nJfXLL4J4
142 | HappyGeniusAxePMSTwin
143 | IronicProtectivePelicanCharlieBitMe-p7hapUoFqmTTMR2
144 | SaltyVivaciousWalletEleGiggle-TwakuttRgfjA87Df
145 | PlausibleMildLatteCclamChamp
146 | HorribleFreezingMooseDBstyle-e_EQb0H9v8NVkWIJ
147 | EnticingUglyClipzThisIsSparta-0EkgBAWjR9Y4jY3
148 | KawaiiBlazingChickenDogFace-NN8WNK-Y3hULF9BJ
149 | SparklyEntertainingAlmondCharlietheUnicorn-QLas0wdTECCzTsa
150 | ObliviousBrainyChamoisKippa-B-SqxwICbv80FDry
151 | BelovedUglyMagePastaThat
152 | UglyBelovedDragonfruitDuDudu-BZQ2sunt0YCeikYc
153 | SpoopyMoistPangolinPanicVis
154 | RelievedRespectfulAlligatorPlanking
155 | AcceptableMoistFennelPeteZarollTie-s5KmFPIx-S8rsfw
156 | SmokyDistinctTermiteRuleFive-Qv-fas6xokgg8_b6
157 | FrozenPhilanthropicFrieseSeemsGood-r2v80dCaInHaA30I
158 | FaintRichElephantYouWHY
159 | PowerfulBusyOtterKippa-ILi8VZZAWlJCvpUr
160 | DifficultRefinedHyenaChocolateRain-6XJNutXWIMGuNZ-q
161 | KindDeliciousTireDoritosChip-mB0ecSyywKpw_lkW
162 | TangentialOptimisticWatermelonPeteZaroll-ZyVgovL1MX-aVoyG
163 | ObedientThoughtfulEelCoolStoryBob
164 | AcceptableLachrymoseOpossumEagleEye-HdI2fGndtK3Mohn
165 | PliableDependableClamSmoocherZ-krbmialJtPwz1mDs
166 | HilariousTriumphantPidgeonRlyTho

167 | KawaiiSpookySalmonWow-VZ40zHiP83AkjAk_
168 | LazyMildDadAsianGlow-Qq9G_mIez98Fjx6_
169 | PuzzledMoralSnoodBudStar-WC_kMbSbK4_FCJMs
170 | SmallBreakableParrotANELE-LdZpex8OfG-rbnIP
171 | AnnoyingAnimatedTrollAsianGlow-pBCfe-lj2VGxMQt2
172 | ObedientDepressedBaguetteShazBotstix
173 | AmusedSillyPonyGivePLZ-YXrCHvuLycZSgG5C
174 | CooperativeTemperedMoonAsianGlow-TD2X0wzhbA-y-uxL
175 | DelightfulDaintyParrotTakeNRG-a_tfHw0lc7Lnfxv
176 | ColdPoliteLaptopStinkyCheese-1Tv5dnWe8DinehXv
177 | KnottyStupidWoodcockSaltBae-C4NUP8zDORxla_EA
178 | HilariousGiftedBisonAMPenergy-AR3J88kPNh7qdGFE
179 | AlluringAttractiveWebTinyFace
180 | YawningCheerfulArugulaNotATK-DWSoJvgm8g38g9KK
181 | PricklyGorgeousMallardWow
182 | HeartlessWildLampStinkyCheese-DjorYUPQRVXzcUWi
183 | FlaccidCooperativeClintmullinsDxCat-VY_eB1Y1F_ruUzTu
184 | SmellyMoralRamenDoggo-dGmRZXcPHFU46dzI
185 | RamshackleFineOwlBrokeBack
186 | DependableSavageRhinocerosPastaThat-g21hdQloCBNXkV6z
187 | CrypticMotionlessRutabagaPunchTrees-diTfLSbA0TazSTDE
188 | AgileEnthusiasticHamCoolCat-JvGzuQo51X12uuvF
189 | HomelyPunchyBeaverNononoCat-PRQqowJ9urW8s-Qh
190 | PoliteDrabParrotPupper-tXjLeM0vokTcPN-q
191 | GloriousFaithfulSangOssloth
192 | EphemeralAbrasiveCamelTebowing
193 | CoyEsteemedTrollMcAT-aE1FQMeVA2ULnJB5
194 | AlertChillyMinkPeoplesChamp
195 | DeterminedHeartlessMarrowFutureMan-FV087sFVlyekwSio
196 | BoredProtectiveGullKAPOW-m9t4MGrmIXLu29yZ
197 | WittyImpossibleToothMikeHogu-4zBDK_-CRTtLltVT
198 | DistinctWittyEchidnaRitzMitz-LC5uvL-6mJ5C12Na
199 | BlightedCleverBubbleteaPanicVis-iYgTGNn5U74NHVgN
200 | ReliableMuddySushiKeepo
201 | DirtySullenDolphinPRChase
202 | OptimisticArtisticToadUwot
203 | CrispyGracefulPeanutPicoMause
204 | KathishSleepyCheddarOhMyDog
205 | AntediluvianCarefulVelociraptorVoteNay-uUrZRdc99Ee_9qb2
206 | BreakableCourteousHedgehogDoubleRainbow-TlhrjrQzebAcY6Dy
207 | SquareBeautifulPotatoAsianGlow-wLeYhZI606QC747Q
208 | DarlingBetterCaterpillarTTours-1d9y044bEnpxU3vD
209 | CoweringLaconicWeaselDxAbomb-pOpiSsoqYLHVPRni
210 | CleverBigCheetahPunchTrees-EpyZ8B_1-EITsJ_z
211 | HomelyResourcefulCrowPrimeMe-sQmDfn3soaNEhMuA
212 | CovertDependableKleePeteZaroll-yyN97kCn0dlyL2zZ
213 | ObliqueOutstandingSparrowAMPenergyCherry-XpUtJpmByxb2C8fL
214 | AbrasiveGlamorousGrouseUnSane-VbEAMdRLWQHLX5K6
215 | AlluringBlitheEmuKAPOW-AeGLr9b0IEqmmCxc
216 | ZealousPrettiestWatercressCmonBruh-Z_cDh_eJFcpo0irs
217 | LachrymoseCautiousCattleM4xHeh-lAnkVQtHptvR1tTl
218 | KindBitterGoshawkBabyRage-ciQGBjKKgldNVQf3
219 | SucculentKnottySandwichThunBeast-PKsGRmX4AvQm7SnL
220 | InterestingBlitheMomDerp-SBUdPy7KgF3LDJoD
221 | MuddySpinelessPoultryCharlietheUnicorn-1CSQRdpVbqKwX4LU
222 | TriangularBlazingSowVoteNay-ZQaYn10PbG5bJiBb
223 | EsteemedSaltyGoshawkTBTacoLeft-ulJhIZRk6CvxdXYS
224 | SpookyNurturingOwlNotATK-9LKC5wi6BXh8fXuS
225 | GorgeousSavoryBisonRlyTho-jIdexJTK-D3dEVVd
226 | SpunkyBoxyTortoiseNononoCat-B4vYy559ShgrsG5M
227 | AttractiveTastyPieHeyGuys-63NDFdDMuxSgnVpq
228 | SpinelessGracefulGrouseNinjaGrumpy-mE4XpG0_3k0-Ssk0
229 | HelplessSmallBubbleteaBabyRage
230 | FuriousOutstandingSnoodFloop-rklibAqeSn9uC6Ki
231 | SecretiveAliveMosquitoShazBotstix-ZnG_ecJLWmjTrDrD
232 | AbrasiveUglySardineChefFrank-u2R0mhQFAuAMf2iT
233 | PlayfulAcceptableBearDatBoi-s6380yixbI5iUStR
234 | SingleHilariousRamenPunchTrees-NR0pJT1Lec5Q9E8p
235 | TrustworthyRespectfulRadishRiPepperonis
236 | ObeseWealthySkirretCoolStoryBob
237 | ThankfulHorribleCamelAMPenergyCherry
238 | GorgeousProudCougarItsBoshyTime-EvNh3-P9i4z4000A
239 | BitterStrongSquirrelRitzMitz-Qf6woiHoCb5hJm1Z
240 | BlazingSuccessfulGoshawkTheTarFu-1-UA3RntfvK5EY12
241 | QuaintMoralSkunkPeoplesChamp-Fv7FmmFSVpjI3c8E
242 | CuteSmoothFalconBloodTrail
243 | NastyNurturingSpiderRickroll
244 | AuspiciousEphemeralCobblerWutFace
245 | ArtisticTangibleTofuItsBoshyTime-FGG0gCG_2MR_UNAI
246 | PuzzledEnthusiasticOtterUwot-M-EUaIEqyxMDfz2R
247 | CoySuccessfulParrotSwiftRage-1abfi6U6cvGpv20n
248 | CalmPopularManateeBibleThump-OwNBtC2q8o7rKp5j
249 | HeartlessPricklyMartenPartyTime
250 | AmericanScrumptiousDonutGrammarKing
251 | PiliabLeSincereCodWutFace-iH__Dza8roosEqXV

252 | InexpensivePuzzledKittenDBstyle-lVrANPxXd_2c5FcQ
253 | GeniusSourLlamaCopyThis
254 | GloriousBombasticPancakeTBTacoRight-Cw0-xXvMAjUlnU7V
255 | ResoluteToughCatCoolStoryBob-2B3DE_CjkL4iuSHp
256 | WimpyToughSagePJSugar
257 | SpookySpinelessGnatHeyGirL-bvMcAU8GcmkC86NL
258 | ToughSingleDoveSpicyBoy
259 | WonderfulFreezingWatermelonPrimeMe
260 | FunCarefulElephantFeelsBadMan-L081d6wi1Y1f4enh
261 | BelovedColdBeaverAllenHuhu-mQh_TToZ8Yf7s0oz
262 | MotionlessDoubtfulChipmunkJKanStyle-vXl-7rG3bcxZNf6Q
263 | StylishRichWhalePooooound-X65xSueQ4hYyvvcX
264 | CredulousPeppyHabaneroTwitchRPG-9AcPimJPz8MQTvLF
265 | MushyResoluteCurryBatChest-c9UvJgkrzrBJD0yP
266 | IcyBoringAubergineCcIamChamp-Ic3syNN1KjsxLCTc
267 | AggressiveBrainyRedpandaVoteNay
268 | EvilPlausibleQuailPogChamp-2Pf0kTHwCValHmYa
269 | BovineBombasticFishShazBotstix-hG1PbdwLyEZoyzyE
270 | SparklingSucculentVampireSoBayed-Z4VFXk3jEpFx_PoA
271 | CrazyFriendlyClintWutFace-n4QWsr2EfDrVstP1
272 | BombasticGrossKuduBigBrother-0hhbwLHTCotZ2XeK
273 | TawdryGrossHareBabyRage-Z9ExWT6rcG5dfe9k
274 | ChillyBlueAirGuitarAliens-u1wVTnslg2ydqrc4
275 | GiftedFunnyDeerOhMyDog-lNuUM80Vhx1PGd0
276 | CreativeHotPrariedogNerfBlueBlaster
277 | TrustworthyBetterCobblerCopyThis-zIyPWQ9sYANNISmF
278 | SlickDifficultSparrowFunRun-1qOXmgox8j5MA00o
279 | InquisitiveSpicySnoodVoteYea-tne6epT9wBhgxAek
280 | AthleticGloriousWheelNononoCat
281 | BombasticTrustworthyEggplantCoolStoryBro-FwIGxRsmrqkh1kmp
282 | PeacefulCreativeRuffArgieB8-VS_Gtvp40EtyNXQ2
283 | CuteRamshackleSnailTebowing-hl6Mq4bnvkl_81JH
284 | VibrantCrackyBananaUnSane-Fy8fRig3VcEQAMSV
285 | HedonisticSincereFrogOMGScoots
286 | SplendidFantasticToadSpicyBoy-HVJLXR1lqddYQKjR
287 | SmallFancyWrenChefFrank
288 | WealthyIcyFlamingoSoonerLater-LtS9c69b72VwwtKP
289 | SpeedyStylishLouseTBTacoRight
290 | BovineWealthyAirGuitarHoneyBadger-yer4ffbbqQntewLtB
291 | SleepyShakingLasagnaVoteNay-KSqL5LkbcuWxfrJ
292 | SincerePreciousFlyWOOP-SYutoQupUKHLurXp
293 | ObedientLittleTrollAMPTropPunch-0EHAFL0FxjUlJjJS
294 | SingleSparklyAniseSMorc-637DQBB6cOpazSda
295 | ConfidentFastAxeTTours
296 | BlushingAlluringCrocodileTF2John-Wjo-4HSXE6StF9-7
297 | SparklyCulturedMouseTooSpicy-GorFp9G2qVc0ocu1
298 | FrigidBitterHamsterCharlieBitMe
299 | EntertainingBlatantTriangleResidentsSleeper-dMCpZoc6bEPBUK0_
300 | JazzyTriumphantDragonfruitLitty
301 | CleverRelievedCheetahHoneyBadger
302 | InventiveKindHeronBCouch-yj43DRKqfpWeYnu2
303 | DreamyBumblingPastaFutureMan-RFojH-a1JPWu6NLh
304 | SpinelessReliableFalconPlanking
305 | FlirtyTenaciousShingleANELE
306 | HelpfulCourteousCormorantOneHand-TRkw_F6f8334cUQK
307 | YummyLitigiousPonyBrokeBack-YUD0VPrIu1PHGEva
308 | RoundFitPelicanGingerPower-hvMaXD-CN0uTWJmX
309 | SpinelessOpenMushroomPastaThat
310 | PlainCrepuscularYakinikuDatBoi-6jJjEb2LERG9XXSa
311 | AuspiciousSingleLyrebirdTBcheesePull
312 | GracefulIntelligentElephantPeteZaroll-bbCbgtCr04swU113
313 | BigSuaveWormPraiseIt-fFtrouUnU5C4M41P
314 | ImpossiblePolitePorpoiseBrokeBack-8k8b0neLUgz1N3C
315 | FaithfulEnjoyablePotKappaRoss
316 | NastyExpensiveGorillaBlargNaut-7W3_1bPa0AlanbZb
317 | AdorableTangibleTaroBrokeBack-mKAvw9BmFb7zbgUV
318 | NurturingSpicyChoughPraiseIt-Sd01UmFgYGQR5qLd
319 | ImpartialHonestSaladKappawealth
320 | DreamyAnimatedPepperResidentsSleeper
321 | BlueShinySandstormCoolCat-PEmxRf06fol8IvTy
322 | ShyBreakableMacaroniKAPow-yoqwkUaVkyBzFYi0
323 | YummyAmazonianTrayKAPow
324 | DeliciousCharmingTubersFloof-f2BMeXvhrTctlsD9
325 | EsteemedDeadGrassYouWHY-wLCWYakiHsUFivZt
326 | DifferentBlindingFungusDxAbomb-bCR_-ck7aBzxvuEM
327 | DullStrongSkunkKeepo-2deXovBIzPECNIPH
328 | PlacidLaconicDugongSaltBae-rG0vzATbuLPVEvAQ
329 | AuspiciousEnchantingDaikonHassaanChop-ExDZRnjfQ0IXBxUL
330 | IntelligentAbstemiousBasenjiUwot-TGnSnnXjAdYpGMaa
331 | SarcasticEasyHamburgerLitty--C4L8T25fIvaK1GY
332 | VivaciousArtisticDootYouDontSay-buQ0c7boC4x52MUG
333 | PerfectBovineToothDogFace-e5e6HD07NoJ-I3gL
334 | TrustworthyPrettiestCurlewKippa
335 | TastyPeppyShieldNerfBlueBlaster
336 | RelievedCoyPieTooSpicy-aJBisysBi603r3P7


```

337 | DreamyFilthyWrenchPeoplesChamp-Zssdks_STyo7BRCp
338 | BoxyWiseMilkShazBotstix-_Rz0i3fHsrZaJj1Y
339 | FunnyNurturingButterTwitchRaid
340 | ChillyReliableDuckBleedPurple-l_kh02UIZEbwumc9
341 | CovertCuteFungusKippa
342 | InspiringGoodCheetahHumbleLife-taxZVDVMFfd7wd9A
343 | TubularAmericanWheelJonCarnage-mT0kXlIR0x34Xdng
344 | CuteRespectfulFerretTooSpicy-qcN2-Z0mCKJMH2e2
345 | OriginalCarefulPassionfruitPMSTwin-oP7PIyA1AkoYV8oq
346 | PeacefulGenerousBatteryMoreCowbell-JpC0VXZ_AXmEAlww
347 | SweetObedientMochaFreakinStinkin
348 | FamousHorribleBobaCoolStoryBob
349 | EndearingWildHerdTTours-gQgX2uJd09YSd50j
350 | SpinelessDoubtfulAlbatrossRaccAttack
351 | LitigiousOriginalRuffPermaSmug-xgBw21kWiZiL2-w5
352 | VainStrongStarLeeroyJenkins-3oGuhMMFCESFGiG3
353 | RichExquisiteSquirrelStinkyCheese-erg74FIWwaqxR4yJ
354 | CrispyGlutenFreeEndiveDuDudu-OyDu3iY0-a-FcQvC
355 | NeighborlySavageLardNerfBlueBlaster
356 | SingleFriendlyCiderPicoMause
357 | RespectfulBlightedVelociraptorRedCoat-t4UB-h0P0w1y4knC
358 | AgileSmokyCurlewTwitchRaid-veaQUARZhsgy2W5k
359 | SavageRoughStorkFailFish-ww2k8vPPp5QBzbJP
360 | AnimatedPricklyCaterpillarKappaWealth-W6Lw-wp8jSVysoMM
361 | OutstandingPlayfulYakAMPEnergyCherry-f00Dd1RckSh6-wbc
362 | GeniusAnimatedCrocodileAMPTropPunch
363 | NastyCooperativeLemurThisIsSparta-5fYVB00LY1DPW1iB
364 | SpinelessPlumpHareCoolStoryBro
365 | RockyEnticingOstrichNomNom-TqN6kBr9bUiY2DW1
366 | JazzyIntelligentPlumDatBoi-X4l2rEeW9ozkmCrn
367 | LittleAffluentFlamingoRaccAttack
368 | RudeTriangularDolphinTriHard
369 | VenomousAmorphousWrenchWoofer
370 | UnsilentlyRelievedSmoothie4Head
371 | BloodyUninterestedSnakeMcAT-bnPsWiKxdLLPqbY4
372 | PlausibleWiseSnoodFrankerZ-LP0r20dPmXOWBCJC
373 | SparklingRacyWaterFutureMan-M7kICbc9SpsSKHSh
374 | BashfulSmokyCaterpillarImGlitch-qb0IP5fZ34c2f4wK
375 | FilthyAmazingStrawberrySM0rc-pAWcMERQgh4gu5i9
376 | AmazonianDignifiedThymeHotPokket-kw1J7igL419tG4jo
377 | HappyAbstruseSushiNerfBlueBlaster-6cPiQ2o_HwfULWxH
378 | DignifiedTamePorpoiseBTacoRight-F_hIyz51X9XudfZD
379 | IronicPuzzledDootDendiFace-j7STApH43NNcHSH6
380 | HardPoliteYakDAESupply-aLCzqX9uonQ0Cuzb
381 | AntediluvianFrailPorcupineSoBayed-ntoYxqXRo-orzyqz
382 | StupidSaltyCaterpillarAMPTropPunch-Uhaov2NmWOLo6uCo
383 | RoundObliviousOcelotPJSugar-IhjoIMBK-oDPBMNE
384 | AmazingMoralTortoiseFreakinStinkin-BFNCUWDDa2ym2nyJ
385 | SuperAgreeablePeppermintRitzMitz--uuu6Bxz0KF0EkMF
386 | EasyRefinedTroutRitzMitz-mMteaPzZapdheHEH
387 | SecretiveDullAlfalfaJonCarnage-WSMwu0T-mRatJJMu
388 | ObeseScrumptiousSharkPJSugar-kCPqj_0xn_ILE7BA
389 | TrustworthyTameLeopardImGlitch-KwETvNjlaQ5roKsD
390 | GorgeousCharmingBeefBCouch-gMl30Y0aEYNTqT8i
391 | GlutenFreeSmokyCardPeoplesChamp-qe7bBG_DfJbM33_a
392 | AnimatedAbrasiveStinkbugFeelsBadMan-Um_wo1gLDDgw-WAd
393 | UgliestStylishPigMrDestructoid-0JmtVkuZujwV5Z-8R
394 | StupidVenomousAardvarkHotPokket-RKhZ9Piw9s93aTXQ
395 | CrypticBoldEagleFloof-ik_Me3eSFboiMVeG
396 | VivaciousPhilanthropicKoalaAllenHuhu--ITtHMEbu_AgY_P5
397 | BlindingEnchantingSlothPanicBasket-T3lJI8XhXzwM5RYi
398 | AntsyOpenNigiriTriHard-4GHDU-ArkA7uF9ld
399 | InventiveYummyLorisMau5-J_xh5UPS4Y0VFK09
400 | PhilanthropicSillyMartenHeyGuys-NiKFNO4r08kabHf1
401 | CourageousMiniatureSharkTriHard-uEQxYfWg8hHS-FzE
402 | AbstemiousSucculentLEDFutureMan-u8I7W5ywCaUZLiit
403 | SmoggyGoodMacaroniVoHiYo-iJQDnX9p40eM-715
404 |

```

Explanation

seen_videos.txt

One clip/video id per line. twitch_finder.mark_video_seen caps it at 500 entries; video_finder.mark_video_seen appends unbounded. It is the dedup ledger. Committed back to the repo each run so the next ephemeral runner starts where the last left off.

upload_log.json

JSON array of {video_id, date, time} per upload. youtube_uploader.get_upload_count_today() counts today's entries to enforce MAX_UPLOADS_PER_DAY (24).

token.json (per channel)

OAuth credential created by `gen_token.py` or materialised from the `YOUTUBE_TOKEN_JSON_<CHANNEL>` secret by `config.setup_secret_files()`. Gitignored; never committed.

6. End-to-End Execution Walkthrough (one full run)

Assume the daily cron fires at 00:30 UTC for the FacebookGaming matrix job. Sequence:

```

1 | 00:30 UTC  GitHub Actions starts the FacebookGaming matrix job (ubuntu-latest).
2 |  ■■ checkout repo
3 |  ■■ Python 3.11 + apt ffmpeg installed
4 |  ■■ pip install -r requirements.txt
5 |  ■■ env: FACEBOOK_PAGE_ID, FACEBOOK_PAGE_ACCESS_TOKEN, TWITCH_*, GROQ_* injected
6 |  ■■ python pipeline.py --channels channels.json --channel FacebookGaming
7 |
8 | pipeline.py:
9 |   run_multi_channel() -> filters to FacebookGaming
10 |   run_channel_pipeline():
11 |     platform='facebook' -> shorts_per_run forced to 0 (NO shorts phase)
12 |     videos_per_run=3, clips_per_video=6, min_video_seconds=195
13 |     source_type='twitch' -> find_twitch_clips(['Minecraft']) (recency windows)
14 |     for vidx in 0..2:
15 |       gather >=6 clips (each <60s) -> needed_clips=max(round(195/45)+1,6)=6
16 |       download each clip (CDN, fallback yt-dlp)
17 |       process_compilation_video(6 segments) -> one 16:9 ~3-6min MP4
18 |       get_video_duration(rendered) -> if <195s: DISCARD, mark seen, retry next
19 |       generate_video_metadata() (Groq, no #shorts)
20 |       upload_fb_video(is_short=False) -> RESUMABLE chunked Graph upload, scheduled
21 |       mark clips seen; cleanup_disk()
22 |     save_report() + notify_report() (Telegram if configured)
23 |
24 | Post-step: commit seen_videos.txt + upload_log.json back to repo (state persists).
```

The four YouTube channels follow the same flow but run the **Shorts loop** (18/15 shorts) AND the video loop (3 long-form), uploading via YouTube OAuth2 with #shorts tags and staggered publish times.

7. Interview Questions & Answers

These questions are derived directly from the codebase and are suitable for assessing understanding of the automation, the APIs used, and the operational fixes applied.

Q1. What is the overall purpose of this repository and what are its inputs/outputs?

It is an autonomous content pipeline that turns trending Twitch gaming clips into YouTube Shorts + long-form compilations (and Facebook videos for one channel). Inputs: Twitch Helix API clips, NCS background music, Groq for metadata, YouTube/Facebook credentials. Outputs: scheduled uploads on YouTube/Facebook and persisted run-state (seen lists, upload logs).

Q2. How is the project scheduled to run, and why GitHub Actions?

A cron in `.github/workflows/upload_short.yml` fires daily at 00:30 UTC, plus a manual dispatch with an optional single-channel input. GitHub Actions is used because it is free-tier, ephemeral (no server to maintain), runs the same Python on Linux with ffmpeg preinstalled via apt, and lets secrets live in the repo's encrypted Secrets store rather than on disk.

Q3. How does the system avoid re-uploading the same clip?

Each channel keeps a seen-list (`seen_videos.txt`). After a clip is downloaded and used (or after a download/processing failure), its id is appended via `mark_video_seen`. `find_twitch_clips` skips any id already in that set. The list is capped at 500 entries (Twitch path) so old clips rotate back and the candidate pool never empties.

Q4. Explain the Twitch clip discovery fix (the recency windows).

Previously the code only queried Twitch's current trending clips, which is a small fixed pool; once the seen-list covered it, `find_twitch_clips` returned `[]` forever. The fix walks back through 2-hour recency windows over the last ~2 days (RFC3339 UTC timestamps with seconds precision and a 'Z' suffix, which is exactly what Twitch's Get Clips API requires) and paginates up to 4 pages per window, collecting unseen clips. Combined with the 500-cap seen-list, content is now always available.

Q5. Why did Facebook uploads fail with HTTP 413 and how was it fixed?

Facebook's simple `/v/{page}/videos` multipart upload rejects large files with 413 'Request Entity Too Large', even at ~9-80MB. The fix routes long-form compilation videos through Facebook's RESUMABLE (chunked) upload endpoint: start -> transfer 4MB chunks in a loop tracking `end_offset` -> finish with metadata/schedule. Shorts still try the simple endpoint first and upgrade to resumable on 413.

Q6. How is the 'every Facebook video must be longer than 3 minutes' guarantee enforced?

In `pipeline.py`, for the Facebook channel `shorts_per_run` is forced to 0 (no Shorts), and `needed_clips` is raised so ~6 Twitch clips (each <=60s) are stitched. After rendering, `get_video_duration` (ffprobe) probes the actual file; if it is under `min_video_seconds` (195s, i.e. >3 min with headroom) the video is discarded, its clips marked seen, and the loop retries the next iteration. So a too-short video is never handed to the uploader.

Q7. How does YouTube authentication work and where are tokens stored?

OAuth2 with the `youtube.upload` scope. Locally, `gen_token.py` runs an `InstalledAppFlow` that opens a browser, writes `data/channels/token.json`. In CI, the token is supplied as the `YOUTUBE_TOKEN_JSON` secret, and `config.setup_secret_files()` writes it to the same path. The token is refreshed automatically when expired. Tokens are gitignored and never committed.

Q8. What is the daily upload quota guard and why is it needed?

`MAX_UPLOADS_PER_DAY = 24` caps uploads per channel per UTC day, tracked via `upload_log.json` (`get_upload_count_today`). It prevents blowing YouTube's daily upload quota and stops runaway loops from flooding a channel.

Q9. Describe the Groq metadata generation and its self-healing behaviour.

`metadata_generator` calls Groq to produce title/description/hashtags/tags as JSON. `_available_chat_models` queries Groq's live `/models` list (filtering speech models, ordering preferred families) and caches it, so decommissioned models are never used. `_call_groq` tries each model; if Groq returns `json_validate_failed` (HTTP 400) it retries the SAME model without forced json mode before moving on. `_extract_json` defensively strips ````json` fences. If everything fails, templated fallback metadata is used.

Q10. How is audio handled differently for voice vs silence vs game audio?

audio_analyzer computes RMS and vocal-band energy and optionally runs SpeechRecognition. If voice is detected the original audio is muted and BGM added; if silent, BGM is added; if clean game audio, original audio is kept with low-volume BGM. This keeps uploads copyright/quality-safe.

Q11. What is the difference between process_video and process_compilation_video?

process_video renders ONE Short: crops 16:9 to 9:16 (1080x1920), 1.2x speedup, capped at 45s, with BGM. process_compilation_video stitches several full clip segments into one 16:9 1080p long-form video (graded, original audio preserved, BGM underlaid), capped at 10 minutes, used for both YouTube long-form and Facebook videos.

Q12. How are secrets kept out of the codebase?

All credentials are environment variables / GitHub Actions secrets. config.py reads them at runtime; setup_secret_files() materialises token files from env only inside CI. token.json and client_secret.json are gitignored. This document contains no live secrets.

Q13. What happens if a YouTube upload hits an auth/network error mid-run?

pipeline.py classifies the error; if it matches auth/network keywords (invalid_grant, credentials, token, unauthorized, transport, connection, socket, timeout) it sets abort_pipeline and breaks the loop, so the channel stops cleanly instead of retrying doomed uploads. Other errors mark the clip seen and continue.

Q14. How does state persist across ephemeral GitHub Actions runners?

After each run the workflow commits the per-channel seen_videos.txt and upload_log.json back to the repo (with a rebase -X ours and up to 5 push retries). The next runner checks out the updated files, so dedup and quota continue correctly.

Q15. What is the purpose of the 'paused_until' field in channels.json?

If a channel has paused_until set to a future date, run_channel_pipeline skips it entirely (no sourcing, no uploads) and reports 'Paused'. It auto-resumes once the date passes. Useful for freezing a channel without code changes.

Q16. Why does the Facebook channel strip '#shorts' from metadata?

Facebook has no #shorts convention (Shorts are a YouTube-only feature). upload_fb_video removes '#shorts' from the title, description, and hashtags and adds neutral gaming tags instead, keeping Facebook posts clean.

Q17. How are multiple channels run in parallel without interfering?

The workflow uses a matrix strategy (max-parallel:4). Each job runs pipeline.py with --channel . config.set_active_channel isolates every file path under data/channels/, so concurrent jobs never share raw_videos, processed dirs, seen lists, or token files.

Q18. What is the role of ffmpeg-python vs the ffmpeg binary?

ffmpeg-python is a Python wrapper used to BUILD filter graphs declaratively (crop, scale, speedup, amix, concat). The actual encoding is performed by the external ffmpeg binary, whose path is resolved by config.get_ffmpeg_path() (system PATH first, bundled fallback). ffprobe (same family) is used for duration probing.

Q19. How would you add a brand-new YouTube channel?

1) Create its OAuth token via gen_token.py and store as secret YOUTUBE_TOKEN_JSON_. 2) Append a channel object to channels.json with channel_name, niche, upload_schedule, videos_per_run, shorts_per_run, youtube_oauth_credentials path, and source_configuration (twitch + target_games). 3) Add the channel to the workflow matrix list and its token secret mapping. No Python changes needed.

Q20. What are the main failure modes observed and their fixes?

a) 'No candidate videos' (seen-list exhaustion) -> recency windows + 500-cap. b) Twitch 400 on clip query -> RFC3339 'Z' seconds-precision windows. c) Facebook 413 -> resumable upload. d) Groq json_validate_failed -> retry without json mode + live model list. e) YouTube invalid_grant -> re-auth token via gen_token.py and update secret. f) Daily quota hit -> MAX_UPLOADS_PER_DAY guard.

8. Environment Variables & Secrets Reference

Name	Used by	Purpose
TWITCH_CLIENT_ID / TWITCH_CLIENT_SECRET	video_finder	Twitch App Access Token (Helix API).
YOUTUBE_API_KEY	video_finder	YouTube Data API for search/playlist source.
GROQ_API_KEY	metadata_generator	Groq model access for AI metadata.
YOUTUBE_CLIENT_SECRET_JSON	config / youtube_uploader	OAuth client secret (root).
YOUTUBE_TOKEN_JSON	config	Legacy/global YouTube token.
YOUTUBE_TOKEN_JSON_<CHANNEL>	config	Per-channel YouTube token (e.g. _GAMES).
FACEBOOK_PAGE_ID	facebook_uploader	Target Facebook Page id.
FACEBOOK_PAGE_ACCESS_TOKEN	facebook_uploader	Long-lived Page Access Token (graphs).
FACEBOOK_GRAPH_VERSION	facebook_uploader	Graph API version (default v19.0).
NOTIFICATION_PROVIDER	notifier	TELEGRAM / TWILIO / NONE.
TELEGRAM_BOT_TOKEN / TELEGRAM_CHANNEL_ID	notifier	Telegram report delivery.
TWILIO_ACCOUNT_SID / AUTH_TOKEN / FROM / USER_PHONE	notifier	SMS report delivery.
COOKIES_PATH	downloader	Optional YouTube cookie file for downloads.

All values are provided through GitHub Actions Secrets (or a local .env). None are stored in the repo.

9. Troubleshooting & Operational Notes

Facebook upload says token expired (code 190/subcode 463)

The FACEBOOK_PAGE_ACCESS_TOKEN is a short-lived token. Regenerate a LONG-LIVED Page Access Token (~60 days) in the Facebook Graph API Explorer / your app, and update the GitHub secret FACEBOOK_PAGE_ACCESS_TOKEN.

YouTube upload returns invalid_grant

The channel's OAuth refresh token is revoked/expired. Re-run ``python gen_token.py`` locally, then paste the new JSON into the YOUTUBE_TOKEN_JSON_secret.

'No candidate videos' for a channel

The seen-list covered the trending pool. The recency-window fix mitigates this; the 500-cap on seen_videos.txt also lets old clips rotate back. If still stuck, trim seen_videos.txt.

Twitich API 400 Bad Request on clip query

Caused by >2h windows or non-RFC3339 timestamps (microseconds/offset). The code now uses 2h windows with '%Y-%m-%dT%H:%M:%SZ' format.

Groq model errors / json_validate_failed spam

Handled by self-healing model selection and the no-json-mode retry. If all models fail, templated fallback metadata is used so runs still complete.

Compilation video under 3 minutes on Facebook

The ffmpeg gate discards it and retries next iteration; ensure clips_per_video (6) and min_video_seconds (195) are set, and that enough distinct Minecraft clips are available.

Disk full on runner

cleanup_disk() removes raw/processed temp files after each clip; intermediates in process_compilation_video are removed in finally.

Push conflict on history commit

The workflow rebases with -X ours and retries up to 5 times; concurrent channel jobs may race but the rebase keeps each channel's own files.

— End of documentation. This PDF was generated programmatically from the repository source at generation time; every code block reflects the files as they exist on disk. No secrets or tokens are included.